itemize

v0.3.0 MIT

An easy way to customize enum and list.

TIANYI-SMILE

Table of Contents

Overview 3
I.1. Basic usage 3
I.2. Features 4
I.3. Override #enum and #list 5
I.4. Model 7
Detailed Features 11
II.1. Support for styling by level and item 11
II.2. Separating setting of #enum and #list 15
II.3. Individual setting (#item) 16
II.3.1. Mark item 17
II.3.2. Terms-like list 19
II.4. Item label formatting 19
II.4.1. Text style 19
II.4.2. Label alignment 20
II.4.3. Label width 20
II.4.4. Sharing the maximum label width 22
II.4.5. Label baseline (Vertical movement) 25
II.4.6. Label inset (Horizontal movement) 29
II.4.7. Formating in any way 30
II.4.8. Step for enum number 33
II.4.9. Enhancement of #list.marker 33
II.5. Item body formatting 35
II.6. Horizontal spacing setting 39
II.6.1. Indentation of item 39
II.6.2. Indentation of item body . 40
II.6.3. Indentation of item label . 40
II.6.4. Body margin and item width 41
II.7. Vertical spacing setting 44
II.7.1. Spacing of the whole item 44
II.7.2. Tight mode 45
II.7.3. Spacing of each item 47
II.7.4. Item tight mode 48
II.8. Reference 50
II.8.1. Reference enum number . 50
II.8.2. Configuration of reference 50
II.8.3. Reference item label 56
II.9. Resuming enum number 56
II.9.1. Resuming by level or by specifying #enum 56
II.9.2. Isolating enum's resuming 59
II.9.3. Resuming for all 60
II.10. Body paragraph setting 61
II.11. Checklist 63
II.11.1. Configuration of checklist 64
II.12. Description list 67
II.12.1. Configuration of description list 69
II.13. Set-Rule 76
API Reference 78

III.1. Main methods 78
III.2. Tool methods 124

Part I.
Overview

I.1. Basic usage

The `itemize` package allows users to easily customize `#enum` and `#list`. To use this package, include the following at the beginning of your document:

```
#import "@preview/itemize:0.3.0" as el
```

Use the method `#default-enum-list` to override the native behavior of `#enum` and `#list` by adding the following at the beginning of your document:

```
#show: el.default-enum-list
```

to apply all `#enum` and `#list` in your document. Or use

```
#el.default-enum-list()[
  /* content */
]
```

to only apply the `#enum` and `#list` in `/* content */`.

⇄ Changed in 0.3.0

`itemize:0.3.0` requires at least Typst 0.15.1.

Now you can use `#enum` and `#list` as usual. `itemize` revises some default behaviors of `#enum` and `#list` (of Typst 0.15.1):

- Labels on the same line are now aligned with the list;
- If the label has no baseline, bottom alignment is used by default (which better fits common usage).

Below is a comparison.

Example

```
#let test = [
  + + $vec(1, 1, 1)$ #lorem(2)
  #set list(marker: circle(width: 2em))
  - #lorem(2)
]
#table(
  columns: (1fr, 1fr),
  table.header([*Native*], [*Itemize*]),
  [#test],
  [
    #show: el.default-enum-list
    #test
  ],
)
```

Native	**Itemize**
1. 1. $\begin{pmatrix}1\\1\\1\end{pmatrix}$ Lorem ipsum. ◯ Lorem ipsum.	1. 1. $\begin{pmatrix}1\\1\\1\end{pmatrix}$ Lorem ipsum. ◯ Lorem ipsum.

I.2. Features

NEW **New features** in version 0.3.0.
Enhancements in version 0.3.0.
Breaking changes in version 0.3.0.
Minor changes in version 0.3.0.

Using `itemize`, you can customize the *label*s and *bodie*s of `#enum` and `#list` for each **level** and each **item**. Currently, the package offers the following features:

- **Item label formatting**
  - Setting the *text style* of item label: `#default-enum-list.args`
  - Setting the indentation for the label: `#default-enum-list.label-indent`
  - Label alignment: `#default-enum-list.label-align`
  - NEW Horizontal movement of the label: `#default-enum-list.label-inset`
  - Vertical movement of the label: `#default-enum-list.label-baseline`
  - Setting the label width: `#default-enum-list.label-width`
    - Allow different `#enum` and `#list` to share the maximum label width: `#default-enum-list.auto-label-width`
  - Formatting the label in any way: `#default-enum-list.label-format`
    - NEW Support setting the *border* of the label.
  - NEW Setting the step size for `#enum` numbers: `#default-enum-list.step`
- **Item body formatting**
  - Setting the indentation between the item label and the item body: `#default-enum-list.body-indent`
  - NEW Setting the item body margin: `#default-enum-list.body-margin`
    - Removed `enum-margin` and `is-full-width` (the latter can be achieved with `body-format: (whole: (width: 100%))`).
  - Formatting the body in any way: `#default-enum-list.body-format`
    - `item-format` is removed; use `body-format` instead.
- **Horizontal spacing setting**
  - The indentation of the item: `#default-enum-list.indent`
    - The border of the whole item will be affected by this setting.
- **Vertical spacing setting**
  - Setting the above and below spacing of whole `#enum` or `#list`: `#default-enum-list.whole-spacing`
    - NEW Configure compact mode: `#default-enum-list.tight-mode`
    - NEW Use the configuration method `#config.auto-detect-tight` to enable the compact mode between the list and the paragraph below it.
    - `enum-spacing` is renamed to `whole-spacing`.
  - Setting the above and below spacing of each item: `#default-enum-list.item-spacing`
    - NEW Configure compact mode: `#default-enum-list.tight-item-mode`
- **Body paragraph setting**
  - NEW Setting the indentation of the first line for first paragraph in item body: `#default-enum-list.first-line-inset`
  - Setting the line indentation and hanging indentation for paragraphs in item body: `#default-enum-list.line-indent`, `#default-enum-list.hanging-indent`
    - NEW For the `default-*` methods, add `#default-enum-list.hanging-type` to set the hanging indent type.

  - The `line-indent` and `hanging-indent` parameters are independent of `par.first-line-indent`, `par.hanging-indent`.
  - Paragraphs inside `block-level` elements that appear in the item body are not processed.
- **Referencing `#enum` number and item label**
  - Enable references to `#enum` numbers by configuring through the method `#config.ref`.
    - NEW Support relative form references.
  - NEW Allow using `#default-enum-list.ref-numbering` and `#default-enum-list.supplement` to set citation styles for different `#enum`s.
  - NEW Support referencing item labels for `#enum` and `#list`.
- **Resuming `#enum` number**
  - Enable through the parameter `#default-enum-list.auto-resuming`, and use in conjunction with the methods `#resume`, `#resume-label`, `#resume-list`, `#auto-resume-enum`.
- **Different styles of description list**
  - Checklist: Enable through the parameter `#default-enum-list.checklist`, and configure through the method `#config.checklist`.
  - Terms-like description list: Change the item label of the current item (`#list` or `#enum`) using the `#item` method.
  - NEW Horizontal description list, hanging description list, stacked description list, plain description list
    - Using the syntax `+ / term: description` and `- / term: description`
    - Configure through the argument `#default-enum-list.description-config`.

NEW **Inline list** Interpret the list content as paragraphs: `#inline-enum-list` (`#inline-enum`, `#inline-list`)

- **Individual setting**
  - NEW Set the style for the current item separately through the `#item` method.
  - NEW Use `#item` to set `tag` and `elem-tag` attributes, then format items or lists with these attributes.

NEW **Initial support for RTL typesetting**

I.3. Override `#enum` and `#list`

The `itemize` package primarily provides the following methods to override the native `#enum` and `#list`:

- `default`-style (Typst's native style): `#default-enum-list`, `#default-enum`, `#default-list`
- `paragraph`-style: `#paragraph-enum-list`, `#paragraph-enum`, `#paragraph-list`
  - This is equivalent to using `default-enum-list.with(hanging-type: "paragraph")`.
- `inline`-style: `#inline-enum-list`, `#inline-enum`, `#inline-list`
  - Any `block-level` elements appearing in the item body are laid out flush left.
  - Contents at every level of the list span the full width of the parent container.
  - Because of this paragraph behavior, some features of `default-enum-list` (`default-enum`, `default-list`) no longer take effect (`label-baseline`, `border` of `body-format`, `body-margin`, `auto-label-width`, and `auto-resuming`).

You can freely customize the styles of `#enum` and `#list` through the parameters provided by these methods.

By default, the `default-*` methods indent paragraphs after the item label, while the `paragraph-*` methods align paragraphs with the item label. `inline-*` methods interpret the list content as paragraphs.

`itemize` currently does not support exporting to HTML format (see https://github.com/tianyi-smile/itemize/issues/18).

See the examples below:

default-style

```
#set enum(numbering: "(A).(I).(i)")
#show: el.default-enum-list
```

(A) Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

(I) Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

(II) Lorem ipsum dolor sit amet, consectetur adipiscing elit.

(B) Lorem ipsum dolor sit amet, consectetur adipiscing elit.

- Lorem ipsum dolor sit amet, consectetur adipiscing elit.
  - Lorem ipsum dolor sit amet, consectetur adipiscing elit.

(A) • (I) ‣ Lorem ipsum dolor sit amet, consectetur adipiscing elit.

paragraph-style

```
#set enum(numbering: "(A).(I).(i)")
#show: el.paragraph-enum-list
```

(A) Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

(I) Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

(II) Lorem ipsum dolor sit amet, consectetur adipiscing elit.

(B) Lorem ipsum dolor sit amet, consectetur adipiscing elit.

- Lorem ipsum dolor sit amet, consectetur adipiscing elit.
  - Lorem ipsum dolor sit amet, consectetur adipiscing elit.

(A) • (I) ‣ Lorem ipsum dolor sit amet, consectetur adipiscing elit.

default-inline

```
#set enum(numbering: "(A).(I).(i)")
#show: el.inline-enum-list
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

$$a^2 + b^2 = c^2$$

(A) Lorem ipsum dolor sit amet, consectetur adipiscing elit.

$$a^2 + b^2 = c^2$$

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

paragraph-inline

```
#set enum(numbering: "(A).(I).(i)")
#show: el.inline-enum-
list.with(hanging-type: "paragraph")
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

$$a^2 + b^2 = c^2$$

(A) Lorem ipsum dolor sit amet, consectetur adipiscing elit.

$$a^2 + b^2 = c^2$$

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

(I) Lorem ipsum dolor sit amet, consectetur adipiscing elit. Lorem ipsum dolor sit amet, consectetur adipiscing elit. (II) Lorem ipsum dolor sit amet, consectetur adipiscing elit.	(I) Lorem ipsum dolor sit amet, consectetur adipiscing elit. Lorem ipsum dolor sit amet, consectetur adipiscing elit. (II) Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Block: Lorem ipsum dolor sit amet, consectetur adipiscing elit.	**Block:** Lorem ipsum dolor sit amet, consectetur adipiscing elit.
(B) Lorem ipsum dolor sit amet, consectetur adipiscing elit.	(B) Lorem ipsum dolor sit amet, consectetur adipiscing elit.
• Lorem ipsum dolor sit amet, consectetur adipiscing elit. ‣ Lorem ipsum dolor sit amet, consectetur adipiscing elit.	• Lorem ipsum dolor sit amet, consectetur adipiscing elit. ‣ Lorem ipsum dolor sit amet, consectetur adipiscing elit.
(A) • (I) ‣ Lorem ipsum dolor sit amet, consectetur adipiscing elit.	(A) • (I) ‣ Lorem ipsum dolor sit amet, consectetur adipiscing elit.

The differences between `*-enum-list` (`#default-enum-list`, `#paragraph-enum-list`, `#inline-enum-list`), `*-enum` (`#default-enum`, `#paragraph-enum`, `#inline-enum`), and `*-list` (`#default-list`, `#paragraph-list`, `#inline-list`) are:

- `*-enum-list` shares the same level counter of `#enum` and `#list`, and can uniformly configure both `#enum` and `#list`, while allowing separate configuration through parameters:
  - `enum-config` for `#enum` (`#default-enum-list.enum-config`);
  - `list-config` for `#list` (`#default-enum-list.list-config`).
- `*-enum` only configures `#enum`, leaving nested `#list` styles unchanged.
- `*-list` only configures `#list`, leaving nested `#enum` styles unchanged.

I.4. Model

The diagram below illustrates the model for `#enum` and `#list` in `itemize`.

- `#enum` and `#list` consist of multiple items.
- Each item is composed of a `label` and a `body`:
  - `label` (`item label`): The enum number or list marker
    - Also, including the spacings: `label-indent`, `body-indent`.
  - `body` (`item body`): The content following the label.
- Level (`level`): Starts from 1, and increments if an item contains nested items.
- Item index (`n`): Starts from 1. The index of items in the same level.

⇄ Changed in 0.3.0

Now (since version 0.3.0), each of the `default-*`, `paragraph-*`, and `inline-*` methods will have a separately maintained level counter. When the method is used, its level will start counting from 1 again. If you want to use the level counting from `itemize 0.2.0` (where a single level counter is maintained throughout the entire document), you need to set the parameter `auto-base-level` (`#default-enum-list.auto-base-level`) to `false`.

In `itemize`, the **level**s of `#enum` and `#list` are divided into the following concepts:

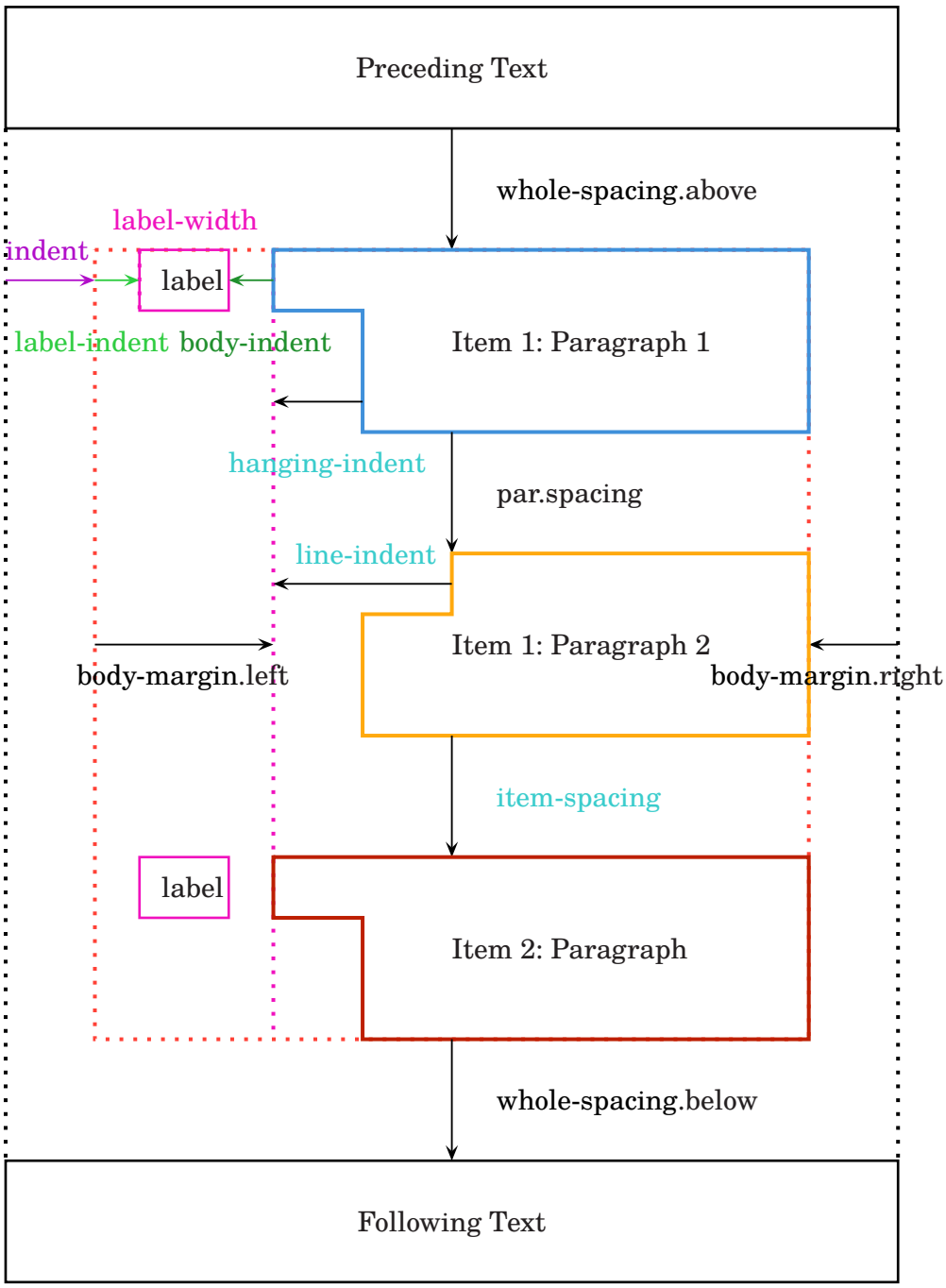
- `absolute level`: includes both `#enum` and `#list` each level of nesting increases the level by 1, for example:

```
1  + level one
2    - level two
3      + level three
4    Paragraph
5    + level two
6  - level one
```

- `relative enum level`: only counts the nesting levels of `#enum`
- `relative list level`: only counts the nesting levels of `#list`.

```
1  + enum level one
2    - list level one
3      + enum level two
4      - list level two
5    Paragraph
6    + enum level two
7      - list level one
8  - list level one
9    + enum level one
```

- Each **`*-enum-list`** method independently maintains the `absolute level`, `relative enum level`, and `relative list level`. The levels within the method use the `absolute level`; additionally, the `enum-config` parameter uses the `relative enum level`, and the `list-config` parameter uses the `relative list level`.
- The implementation does not use Typst's `counter` or `state` (to avoid one extra layout iteration).
- Each **`*-enum`** method independently maintains the `relative enum level`; the levels within the method use the `relative enum level`.
- Similarly, each **`*-list`** method independently maintains the `relative list level`; the levels within the method use the `relative list level`.



#default-enum-list

Code Example:

Example

```
#set align(left)
#show: el.default-enum-list.with(
  indent: 1em,
  label-align: right,
  label-width: 3em,
  label-format: (stroke: red),
  body-indent: 1em,
  hanging-indent: 2em,
  line-indent: 4em,
  body-margin: (right: 1em),
  whole-spacing: 2.5em,
  item-spacing: 2em,
  body-format: (
    whole: (stroke: blue, inset: 5pt),
    inner: (stroke: green, inset: 2pt),
  )
)
Preceding Text. #lorem(10)
1. Item 1: Paragraph 1. #lorem(10)

  Paragraph 2. #lorem(10)

100. Item 2. #lorem(10)
Following Text. #lorem(10)
```

Preceding Text. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

1. Item 1: Paragraph 1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

   Paragraph 2. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

100. Item 2. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Following Text. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Part II.
Detailed Features

II.1. Support for styling by level and item

In the **`*-enum-list`** (**`*-enum`**, **`*-list`**) methods, most parameters support being set per level and per item, which is achieved by passing an `array` or a `function`. In the API Reference (Section III), these are marked as `array` (level-property) and `function` (level-property).

> The parameter types of the methods in `itemize` can be divided into the following three kinds:
>
> - `base-property`: Not settable per level or per item
> - `level-property`: Settable per level, but not per item
> - `level-index-property`: Settable per level and per item; most parameters are of this type.

- `array` (level-property): Each element of the `array` sets the style of the item at the corresponding level; that is, the value at array index `l` sets the style of the item at level `l + 1`.
  - If the last element of the array is `LOOP`, the values in the array will be used cyclically;
  - otherwise, the last element's value applies to subsequent levels.
  - The value of each element of the array is one of the values allowed by the parameter.
  - If the parameter allows setting the style of an individual item (i.e., it is a `level-index-property`), then an element of the array may itself be an `array`, used to set the style of the items at the current level: for example, if the element at index `l` is an array `arrayItem`, then the element at index `n` of `arrayItem` sets the style of the `(n + 1)`-th item at level `l + 1`. Similarly,
    - If the last element of `arrayItem` is `LOOP`, the values in the array will be used cyclically for other indexes of items;
    - otherwise, the last element's value applies to subsequent indexes of items.

For example: the `fill` parameter in `#default-enum-list.args` sets the style of the item label, and it is `level-index-property`.

Example

```
#let test = block(stroke: red, inset: 5pt)[
  + Level1 One
  + Level1 Two
    + Level2-1
    + Level2-2
      + Level3
        + Level4
    + Level2-3
    + Level2-4
]
#[
  #show: el.default-enum-list.with(
    fill: (red, green, blue), // level 1: red, level 2: green, subsequent levels: blue
  )
  #test
]
#[
  #show: el.default-enum-list.with(
    fill: (red, green, blue, el.LOOP), // level 1: red, level 2: green, level 3: blue, level 4: red (cyclically used)
  )
  #test
]
#[
  #show: el.default-enum-list.with(
    fill: (red, (purple, green, el.LOOP), blue), // level 1: red, level 2 (index 1: purple, index 2: green, index 3: purple, index 4: green, cyclically used), subsequent levels: blue
  )
  #test
]
```

1. Level1 One
2. Level1 Two
   1. Level2-1
   2. Level2-2
      1. Level3
         1. Level4
   3. Level2-3
   4. Level2-4

1. Level1 One
2. Level1 Two
   1. Level2-1
   2. Level2-2
      1. Level3
         1. Level4
   3. Level2-3
   4. Level2-4

1. Level1 One
2. Level1 Two
   1. Level2-1
   2. Level2-2
      1. Level3
         1. Level4
   3. Level2-3
   4. Level2-4

- `function` (level-property): If it is a `function`, its form is `it => value`, where `value` is one of the values allowed by the parameter (in most cases).
  - If the parameter allows setting the style of an individual item (i.e., it is a `level-index-property`), then `value` may be an `array`, where each element sets the style of the item at the corresponding index.
  - You can usually access the relevant values provided by `itemize` through the keys available on `it`, for example:
    - `it.level`: The level of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.e`: The field of the current construct element (`#enum` or `#list`), but excluding the `children`.
    - `it.n`: The index of the item, starting from 1, when the parameter is `level-index-property`.

    … (For more accessible values, see the API Reference (Section III).)

Example

```
#show: el.default-enum-list.with(
  fill: it => {
    if it.level == 1 {
      red // level 1: red
    } else {
      (purple, blue, el.LOOP) // subseqence levels (index 1: purple, index 2: green, index 3: purple, index 4: green, cyclically used)
    }
  },
)
+ Level1 One
+ Level1 Two
  + Level2-1
  + Level2-2
    + Level3-1
    + Level3-2
    + Level3-3
  + Level2-3
  + Level2-4
```

1. Level1 One
2. Level1 Two
   1. Level2-1
   2. Level2-2
      1. Level3-1
      2. Level3-2
      3. Level3-3
   3. Level2-3
   4. Level2-4

If the parameter is `level-property`, the `function` (level-property) can generally access:

- `it.level`: The level of the item, starting from 1.
- `it.n-last`: The index of the last item in the current level.
- `it.elem-tag` (since ver0.3.0): The tag of the current enum or list.
- `it.e` (since ver0.3.0): The field of the current construct element (`#enum` or `#list`), but excluding the `children`.
- `it.hanging-type` (since ver0.3.0): The hanging type of the current level.

If it is `level-index-property`, the following can also be accessed:

- `it.n`: The index of the item, starting from 1.
- `it.tag` (since ver0.3.0): The tag of the current item.
- `it.marker` (since ver0.3.0): The marker of the current label if it is a `#list`.
- `it.number` (since ver0.3.0): The number of the current label if it is a `#enum`.

For horizontal spacing parameters

- NEW `it.parent` (since ver0.3.0): the parameter info of the parent containing the current list — the parent's construction element `e`, the item `n` where the parent is located, and the horizontal margin parameters: `label-indent`, `label-inset`, `body-indent`, `indent`, `body-margin`, `label-width` (`max`, `current`)
- ! `it.get` (since ver0.3.0): access the parameter info of the current list's ancestors by level
- NEW For `label-indent`, `label-inset`, `body-indent`, `indent`, `body-margin`: you can access `label-width` (`max`, `current`)
- NEW For `first-line-inset`, `line-indent`, `hanging-indent`: you can access `label-indent`, `label-inset`, `body-indent`, `indent`, `body-margin`, `label-width` (`max`, `current`)

Using a `function` (level-property), you can implement some complex style settings. For example:

Example

```
#show: el.default-enum-list.with(
  fill: it => {
    let last-e(color) = if it.n == it.n-last
    { red } else { color }
    if it.e.tight {
      last-e(purple)
    } else {
      last-e(blue)
    }
  },
)
*tight*
+ tight #lorem(2)
+ tight #lorem(2)
+ tight #lorem(2)

*not-tight*
+ tight #lorem(2)

+ tight #lorem(2)
+ tight #lorem(2)
```

tight

1. tight Lorem ipsum.
2. tight Lorem ipsum.
3. tight Lorem ipsum.

not-tight

1. tight Lorem ipsum.
2. tight Lorem ipsum.
3. tight Lorem ipsum.

If a parameter itself allows passing an `array` or a `function` (marked in the API Reference (Section III) as `array` (base-property) and `function` (base-property)), then to resolve the conflict, wrap these values in an `array`.

For example: the `font` parameter in `#default-enum-list.args` sets the font of the item label. According to `#text.font`, this parameter allows an `array`. To set the desired font correctly, note the difference between the following three examples.

- In the following, the first level of labels use the font `(name: "DejaVu Sans Mono", covers: "latin-in-cjk")`, and other levels use the font `"Noto Serif CJK SC"`.

Example

```
#let label-font = ((name: "DejaVu Sans Mono", covers:
"latin-in-cjk"), "Noto Serif CJK SC")
#show: el.default-enum-list.with(font: label-font)
#set enum(numbering: " (1) ")
+ #lorem(2)
+ #lorem(2)
  + #lorem(2)
  + #lorem(2)
```

(1) Lorem ipsum.
(2) Lorem ipsum.
    (1) Lorem ipsum.
    (2) Lorem ipsum.

- In the following, for each level, the first index of label uses the font `(name: "DejaVu Sans Mono", covers: "latin-in-cjk")`, and other indices use the font `"Noto Serif CJK SC"`.

Example

```
#let label-font = ((name: "DejaVu Sans Mono", covers:
"latin-in-cjk"), "Noto Serif CJK SC")
#show: el.default-enum-list.with(font: (label-font,))
#set enum(numbering: " (1) ")
+ #lorem(2)
+ #lorem(2)
  + #lorem(2)
```

(1) Lorem ipsum.
(2) Lorem ipsum.
    (1) Lorem ipsum.
    (2) Lorem ipsum.

```
7    + #lorem(2)
```

- In the following, all the levels of labels use the font `((name: "DejaVu Sans Mono", covers: "latin-in-cjk"), "Noto Serif CJK SC")`.

Example

```
#let label-font = ((name: "DejaVu Sans Mono", covers: "latin-in-cjk"), "Noto Serif CJK SC")
#show: el.default-enum-list.with(font: ((label-font,),))
#set enum(numbering: " (1) ")
+ #lorem(2)
+ #lorem(2)
  + #lorem(2)
  + #lorem(2)
```

(1) Lorem ipsum.
(2) Lorem ipsum.
  (1) Lorem ipsum.
  (2) Lorem ipsum.

II.2. Separating setting of #enum and #list

↑ Introduced in 0.2.0

`#default-enum-list` is used to uniformly set the styles of `#enum` and `#list`. You can also set the styles of `#enum` and `#list` separately through the parameters `#default-enum-list.enum-config` and `#default-enum-list.list-config`.

- The level used in `enum-config` and `list-config` is the relative level of `#enum` (i.e., ignore `#list`) and `#list` (i.e., ignore `#enum`), respectively.
- The values allowed to be passed in the keys are consistent with the corresponding parameters in the method `#default-enum-list`.

Rules: If both `#default-enum-list` and `#default-enum-list.enum-config` have the same property set, the settings in `enum-config` take precedence.

- For `..args`, `label-format`, `body-format`, and `description-config`, the settings are folded.

Example

```
#show: el.default-enum-list.with(
  enum-config: (fill: (red, auto)), // For level 1, enum's labels are colored by red
  list-config: (fill: (blue, auto)), // For level 1, list's labels are colored by blue
  fill: green, // For subsequent levels, all labels are colored by green
)
+ enum level one
  - list level one
    + enum level two
    - list level two
  Paragraph
  + enum level two
    - list level one
- list level one
  + enum level one
```

1. enum level one
   - list level one
     1. enum level two
     - list level two

   Paragraph
   1. enum level two
      - list level one
- list level one
  1. enum level one

II.3. Individual setting (#item)

↑ Introduced in 0.3.0

For most parameters of #default-enum-list, you can use #item to set the style of a specific item. It must be placed between the label and the body, for example:

```
+ #el.item(...) body
```

> The values allowed to be passed in the keys of #item.args are consistent with the corresponding parameters in the method #default-enum-list.

> Rules: If #item.args, #default-enum-list and #default-enum-list.enum-config have the same property set, the settings in #item.args take precedence.
>
> - For ..args, label-format, body-format, and description-config, the settings are folded.

NEW The style of the whole item (#enum or #list)

- It must be placed in the first item of the whole item to take effect
- If the parameter is `level-property`, it takes effect directly. For example, #default-enum-list.whole-spacing and #default-enum-list.body-format.whole are `level-property`,

Example

```
#show: el.default-enum-list
Paragraph
+ #lorem(2)
+ #lorem(2)
Paragraph
+ #el.item(whole-spacing: 2em) #lorem(2)
+ #lorem(2)
Paragraph
```

Paragraph
1. Lorem ipsum.
2. Lorem ipsum.

Paragraph

1. Lorem ipsum.
2. Lorem ipsum.

Paragraph

Example

```
#show: el.default-enum-list
Paragraph
+ #lorem(2)
+ #lorem(2)
Paragraph
+ #el.item(body-format: (whole: (stroke: red)))
#lorem(2)
+ #lorem(2)
Paragraph
```

Paragraph
1. Lorem ipsum.
2. Lorem ipsum.

Paragraph
1. Lorem ipsum.
2. Lorem ipsum.

Paragraph

- If the parameter is `level-index-property`, to make its effect apply to the whole item, set `for-elem` in #item to `true`. For example:

Example

```
#show: el.default-enum-list
Paragraph
+ #el.item(fill: red, for-elem: true) #lorem(2)
+ #lorem(2)
Paragraph
+ #el.item(fill: red) #lorem(2)
+ #lorem(2)
Paragraph
```

Paragraph
1. Lorem ipsum.
2. Lorem ipsum.

Paragraph
1. Lorem ipsum.
2. Lorem ipsum.

Paragraph

NEW The style of a single item. The parameter must be `level-index-property`.

Example

```
#show: el.default-enum-list
Paragraph
+ #lorem(2)
+ #el.item(body-format: (inner: (emph,)), fill: red) #lorem(2)
+ #lorem(2)
```

Paragraph
1. Lorem ipsum.
2. *Lorem ipsum.*
3. Lorem ipsum.

Note that `#item` only affects the item at the current level and does not affect its nested items (unless the effect itself affects the style inside the item body). For example:

Example

```
#show: el.default-enum-list
Paragraph
+ #lorem(2)
+ #el.item(fill: red) #lorem(2)
  + #lorem(2)
+ #el.item(body-format: (style: (fill: blue))) #lorem(2)
  + #lorem(2)
  #lorem(2)
+ #lorem(2)
```

Paragraph
1. Lorem ipsum.
2. Lorem ipsum.
   1. Lorem ipsum.
3. Lorem ipsum.
   1. Lorem ipsum.

   Lorem ipsum.
4. Lorem ipsum.

II.3.1. Mark item

↑ Introduced in 0.3.0

You can use the parameters `elem-tag` and `tag` of `#item` to mark the whole item and a single item respectively, and then add styles to them or implement other complex customizations.

- The parameters `elem-tag` and `tag` accept any type

The parameter `elem-tag` must be placed in the first item of the whole item to take effect.

- For parameters that are `level-property` and `level-index-property`, you can use a `function` (level-property) to access the parameters `elem-tag` and `tag`.

Example

```
#show: el.default-enum-list.with(
  fill: it => {
    if it.elem-tag == "important" { red } else
    {
      if it.tag == "important" { blue } else
      { auto }
    }
  }
)
+ #lorem(2)
  + #el.item(elem-tag: "important") #lorem(2)
  + #lorem(2)
  Paragraph
  + #lorem(2)
+ #el.item(tag: "important") #lorem(2)
+ #lorem(2)
```

1. Lorem ipsum.
   1. Lorem ipsum.
   2. Lorem ipsum.

   Paragraph
   1. Lorem ipsum.
2. Lorem ipsum.
3. Lorem ipsum.

By parsing the values `elem-tag` and `tag`, you can mark items and customize their styles. For example:

Example

```
#show: el.default-enum-list.with(
  body-format: (
    inner: (
      format: it => {
        if it.tag == "emph" {
          set text(fill: red)
          show: emph
          it.body
        } else {
          it.body
        }
      },
    ),
    whole: (
      stroke: it => if it.elem-tag ==
      "stroke:red" { red } else { auto },
    ),
  ),
  label-format: it => {
    if type(it.tag) == array {
      let fmt-strong = body => {
        if "strong" in it.tag {
          set text(fill: orange)
          show: strong
          body
        } else {
          body
        }
      }
      let fmt-highlight = body => {
        if "highlight" in it.tag {
          show: highlight
          it.body
        } else {
```

1. Lorem ipsum.
2. *Lorem ipsum.*
3. Lorem ipsum.
   1. Lorem ipsum.
   2. Lorem ipsum.
   3. Lorem ipsum.

```
34            body
35          }
36        }
37        fmt-strong(fmt-highlight(it.body))
38      } else {
39        it.body
40      }
41    },
42  )
43  + #lorem(2)
44  + #el.item(tag: "emph") #lorem(2)
45  + #lorem(2)
46    + #el.item(tag: ("strong",), elem-tag:
    "stroke:red") #lorem(2)
47    + #el.item(tag: ("strong", "highlight"))
    #lorem(2)
48    + #lorem(2)
```

II.3.2. Terms-like list

↑ Introduced in 0.2.0

You can pass a positional argument (`content`) to #item:

- Change the current #list marker

Example

```
1 #show: el.default-enum-list
2 - #el.item[#sym.section] #lorem(2)
3 - #lorem(2)
```

§ Lorem ipsum.
- Lorem ipsum.

↑ Introduced in 0.3.0

NEW Change the current #enum number
- The parameter `skipped` of #item sets whether to skip counting the current #enum number; in this case the current #enum number will be the same as the previous item

Example

```
1 #show: el.default-enum-list.with(label-align:
  left)
2 + #lorem(2)
3 + #el.item[#sym.section] #lorem(2) Current enum
  number is 2.
4 + #lorem(2)
5 + #el.item(skipped: true, [#sym.section])
  #lorem(2) Current enum number is 3.
6 + #lorem(2)
```

1. Lorem ipsum.
§ Lorem ipsum. Current enum number is 2.
3. Lorem ipsum.
§ Lorem ipsum. Current enum number is 3.
4. Lorem ipsum.

II.4. Item label formatting

II.4.1. Text style

↑ Introduced in 0.1.0

#default-enum-list.args can pass any #text named arguments (like: `fill`, `size`, `font`, etc.) to format the current item label. Unless `auto` is used, the text style of `label` cannot be changed via `#set text`.

Example

```
#show: el.default-enum-list.with(
  fill: (red, blue, auto),
  size: (18pt, 15pt, auto),
  stroke: (none, none, purple + 0.05em),
)
+ #lorem(2)
  + #lorem(2)
  + #lorem(2)
+ #lorem(2)
  + #lorem(2)
    + #lorem(2)
    + #lorem(2)
```

1. Lorem ipsum.
   1. Lorem ipsum.
   2. Lorem ipsum.
2. Lorem ipsum.
   1. Lorem ipsum.
      1. Lorem ipsum.
      2. Lorem ipsum.

All the `#text` named arguments support `level-index-property`.

II.4.2. Label alignment

↑ Introduced in 0.2.0

`#default-enum-list.label-align` (`level-index-property`) is usually used to set *horizontal* alignment for label.

If you want 2D alignment, see `#default-enum-list.label-baseline`.

This differs from `enum.number-align` and `list.marker-align` (the latter align whole enum numbers and list markers). We suggest keeping `enum.number-align` and `list.marker-align` at their defaults when using `itemize`.

Example

```
#show: el.default-enum-list.with(
  label-align: (start, end),
)
+ #lorem(2)
  + #lorem(2)
  100. #lorem(2)
100. #lorem(2)
```

1. Lorem ipsum.
   1. Lorem ipsum.
   100. Lorem ipsum.
100. Lorem ipsum.

II.4.3. Label width

↑ Introduced in 0.2.0

`#default-enum-list.label-width` (`level-index-property`) allows re-rendering the width of the current label as a given value. Its complete form is `(`

`(amount): 0pt` `auto` | `"max"` | `relative`

`(style): "constant"` `"default"` | `"constant"` | `"auto"` | `"native"`

`)`

Notation:

- `real-width`: The real width of the current label's content (calculated by `measure`).
- `max-width`: The maximum value of all `real-widths` in the current level.

⇄ Changed in 0.3.0

When `amount` is

🡅 `relative`: The value is `amount.length + amount.ratio × real-width`.

- `auto`: `real-width`.
- `"max"`: `max-width`.

> `amount` allows to pass `ratio`, which is relative to the real width of the item label. So, `(amount: 100%)` is the same as `(amount: auto)`.

When `style` is

- `"default"`: If `real-width` is less than `amount`, the label's width is rendered as `amount`; otherwise, the label's width is rendered as `real-width`.

Example

```
#set par(justify: true)
#el.default-enum-list(
  label-width: (amount: 2em, style: "default"),
  label-format: (stroke: red),
)[
  1. #lorem(10)
  11. #lorem(10)

      #lorem(15)
  1111. #lorem(10)
  111111. #lorem(10)
]
```

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
11. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

1111. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

111111. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

- `"constant"`: The label's width is rendered as `amount`.

Example

```
#set par(justify: true)
#el.default-enum-list(
  label-width: (amount: 2em, style:
  "constant"),
  label-format: (stroke: red),
)[
  1. #lorem(10)
  11. #lorem(10)

      #lorem(15)
  1111. #lorem(10)
  111111. #lorem(10)
]
```

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
11. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

1111. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

111111. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

- `"auto"`: The label's width is rendered as `real-width`.

Example

```
#set par(justify: true)
#el.default-enum-list(
  label-width: (amount: 2em, style: "auto"),
  label-format: (stroke: red),
)[
  1. #lorem(10)
  11. #lorem(10)

      #lorem(15)
  1111. #lorem(10)
  111111. #lorem(10)
]
```

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
11. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

1111. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
111111. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

- `"native"`: The label's width is rendered as `max-width`.
  ⚠ Also `amount` is set to `max-width`.

⇄ Changed in 0.3.0

Example

```
#set par(justify: true)
#el.default-enum-list(
  label-width: (style: "native"),
  label-format: (stroke: red),
)[
  1. #lorem(10)
  11. #lorem(10)

      #lorem(15)
  1111. #lorem(10)
  111111. #lorem(10)
]
```

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
11. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

1111. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
111111. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Shorthand rules:

`label-width` can also take

- `auto`: Use the native behavior. That is, the current width of the label will be set to `max-width`. This is the same as `(style: "native")`.
- `relative`: This is the same as `(amount: label-width, style: "default")`.
- `"max"`: This is the same as `auto`.

> In order to make the layout more in line with expectations, the `start-margin` (i.e., `body-margin.left` if `text.dir` is `ltr`, `body-margin.right` if `text.dir` is `rtl`) will be set to `amount + body-indent` (if `start-margin` is `auto`).

II.4.4. Sharing the maximum label width

In Typst, the maximum widths of the item labels of different `#enum`s and `#list`s are computed separately. For example:

Example

```
1. enum A
100. #lorem(2)
Paragraph
1. enum B
Paragraph
100. enum C
```

1. enum A
100. Lorem ipsum.

Paragraph

1. enum B

Paragraph

100. enum C

You will see that the two items in `enum A` share the same maximum width, but `enum B` and `enum C` have two different widths.

↑ Introduced in 0.2.0

By configuring `#default-enum-list.auto-label-width` (`level-property`), `itemize` allows different (but at the same level) `#enum`s and `#list`s to share the maximum item label width so that they are aligned. For example:

Example

```
#el.default-enum-list(
  auto-label-width: "each"
)[
  1. enum A
  100. #lorem(2)
  Paragraph
  1. enum B
  Paragraph
  100. enum C
]
```

1. enum A
100. Lorem ipsum.

Paragraph

1. enum B

Paragraph

100. enum C

If you need to specify which `#enum`s and `#list`s share the maximum item label width, there are several ways as follows.

- Set `#default-enum-list.auto-label-width` to `auto`, then use the method `#auto-label-item`. In this case, in the `body` of the method `auto-label-item(body)`, the `#enum` or `#list` share the same maximum label width.

Example

```
/// for debug
#let v-line = context {
  let height = if page.height == auto {
    12cm // debug
  } else {
    page.height
  }
  box(outset: (top: height, bottom: height), stroke: 1pt + yellow)
}
#show: el.default-enum-list.with(auto-label-width: auto)
+ #lorem(2)
Paragraph
#el.auto-label-item[
  + #lorem(2)
```

1. Lorem ipsum.

Paragraph

1. Lorem ipsum.
    1. Lorem ipsum.
    2. Lorem ipsum.

Paragraph

10. Lorem ipsum.
    100. Lorem ipsum.

Paragraph

1. Lorem ipsum.

```
    + #lorem(2)
    + #lorem(2)
  Paragraph
  10. #v-line#lorem(2)
    100. #v-line#lorem(2)
]
Paragraph
+ #lorem(2)
```

- `#auto-label-item.form` (`level-property`) is used to specify the `#enum` and `#list` at different levels.
  - **Note**: The method `#auto-label-item` cannot be nested.
- For `#default-enum-list.auto-label-width`, use a `function` (level-property) together with the `#item` method to mark `elem-tag`. In the following, for the same level of `#enum`s or `#list`s, if they are marked as `"auto-label"` (with the help of `el.item(elem-tag: ...)`), then they will share the same maximum label width.

Example

```
#show: el.default-enum-list.with(auto-label-
width: it => {
  if it.elem-tag == "auto-label" { auto }
})
+ Item one
  + #el.item(elem-tag: "auto-label") Sub Item
one
  + Sub item two
+ Item one
  10. #el.item(elem-tag: "auto-label") Sub Item
one
  + Sub item two
```

1. Item one
   1. Sub Item one
   2. Sub item two
2. Item one
   10. Sub Item one
   11. Sub item two

Both `#auto-label-item.form` and `#default-enum-list.auto-label-width` can take the following values:

- `"each"`: shares the maximum item label width of `#enum` and `#list` separately.
- `"all"`: shares the maximum item label width of `#enum` and `#list` together.
- `"enum"`: shares only the maximum item label width of `#enum`.
- `"list"`: shares only the maximum item label width of `#list`.

When `auto-label-width` is not `auto` or `none`, the method `default-enum-list` cannot be used in a nested manner. For example: both of the following two usages are incorrect.

```
#show: el.default-enum-list.with(auto-label-width: "all")
#show: el.default-enum-list.with(auto-label-width: "each")
...
```

or

```
#el.default-enum-list(auto-label-width: "all")[
  #show: el.default-enum-list.with(auto-label-width: "each")
  ...
]
```

However, the following usage is not affected:

```
#[
  #show: el.default-enum-list.with(auto-label-width: "all")
  ...
]
#[
  #show: el.default-enum-list.with(auto-label-width: "each")
]
#el.default-enum-list(auto-label-width: "each")[
  ...
]
```

II.4.5. Label baseline (Vertical movement)

↑ Introduced in 0.2.0

By setting `#default-enum-list.label-baseline` (`level-index-property`), you can vertically translate the item label. It can also be used to set the alignment between the item label and the item body.

In Typst 0.15.1, if the item label and the item body have a `baseline`, then the item label and the item body will be aligned by their baselines by default; otherwise they are top-aligned.

Note: the `#enum` or `#list` as a whole has no baseline. In particular, if multiple item labels are displayed on the same line, their baselines are not aligned. For example:

```
1  + + $vec(1,1,1)$
```

is rendered as

1. 1. $\begin{pmatrix}1\\1\\1\end{pmatrix}$

In `itemize`, we modify this behavior by default to align their baselines. At the same time, more alignment options are provided.

Its complete form is: (

(amount): 0pt `auto` | `relative` | `"center"` | `"top"` | `"bottom"` | `"baseline"` | `"top-item"` | `"horizon-item"` | `"bottom-item"`

(at): auto `auto` | `top` | `bottom` | `horizon`

(relative-to): auto `auto` | `"bottom"` | `"center"` | `"top"` | `"baseline"` | `"top-item"` | `"horizon-item"` | `"bottom-item"` | `array` | (

  (to): [A] `auto` | `content`

  (at): auto `auto` | `top` | `bottom` | `horizon`

  (shift): 0pt `relative`

)

(same-line-style): `auto` | `"top"` | `"center"` | `"bottom"` | `"baseline"` | `"top-item"` | `"horizon-item"` | `"bottom-item"` | `"top-label"` | `"center-label"` | `"bottom-label"` | `"baseline-label"` | (

```
    (amount): 0pt  auto | relative
    (at): auto  auto | top | bottom | horizon
    (relative-to): "baseline"  "bottom"
      | "center" | "top" | "baseline" | "top-item" | "horizon-item" | "bottom-item" | "top-label" |
      "center-label" | "bottom-label" | "baseline-label" | relative | content | array | (
       (to): [A]  auto | content
       (at): auto  auto | top | bottom | horizon
       (shift): 0pt  relative
      )
  )
 (impact-first-line): auto  auto | bool
)
```

↑ Introduced in 0.3.0 NEW The three keys `(amount, at, relative-to)` set the alignment between the current item label and the item body. Their meaning is: take the position at `at` of the item label, move it by `amount`, and align it with the position set by `relative-to`.

- `amount` may be a `ratio`, which is relative to the height of the current item label.
- For the meaning of `at`, see also #box.baseline.
- The alignment position set by `relative-to: (to, at, shift)` means: the position of the content `to` at `at`, moved by `shift`.
  - The content `to` should usually appear in the first line of the item body.
  - **Note**: `itemize` uses the height of the first character (i.e., [A]) in the first line of the item body. If you manually set the font style of the paragraph's text, this alignment may not be accurate. It is recommended to use the `style` parameter in `body-format` for adjustments.
  - `shift` is relative to the height of `to`.
- Only when `amount` is `auto` or `relative`, the keys `at` and `relative-to` can work.

1. Shorthand rules for `label-baseline`:
   - When `label-baseline` takes `auto` | `relative` | `"center"` | `"top"` | `"bottom"` | `"baseline"` | `"top-item"` | `"horizon-item"` | `"bottom-item"`

   it is equivalent to setting `amount` to the corresponding value.
2. Shorthand rules for `amount`:
   - When `amount` takes `"center"` | `"top"` | `"bottom"` | `"baseline"` it aligns the `horizon`, `top`, `bottom`, `baseline` of the item label and the item body (first line). which are respectively equivalent to `(at: horizon, relative-to: "center")`, `(at: top, relative-to: "top")`, `(at: bottom, relative-to: "bottom")`, `(at: auto, relative-to: "baseline")`.
   - When `amount` takes `"top-item"` | `"horizon-item"` | `"bottom-item"`, it aligns the `top`, `horizontal`, `bottom` of the item label to the `top`, `horizontal`, `bottom` of the whole item body. This is equivalent to setting `enum.number-align` or `list.marker-align` to `top`, `horizon`, `bottom`. In this case `at` will be set to `top`, `horizon`, `bottom` and can no longer be set by the user.
3. Shorthand rules for `relative-to`.
   - When `relative-to` takes `"center"` | `"top"` | `"bottom"` | `"baseline"`, it is equivalent to aligning the item body at the `horizon`, `top`, `bottom`, `baseline` of the first line. That is, equivalent to `(to: [A], at: horizon)`, `(to: [A], at: top)`, `(to: [A], at: bottom)`, `(to: [A], at: auto)`.
   - When `relative-to` takes `"top-item"` | `"horizon-item"` | `"bottom-item"`, it is equivalent to setting `enum.number-align` or `list.marker-align` to `top`, `horizon`, `bottom`. In this case `at` will be set to `top`, `horizon`, `bottom` and can no longer be set by the user.

- When `relative-to` takes `content`, it is equivalent to `(at: auto, to: relative-to, shift: 0pt)`.
- When `relative-to` takes `length`, it is equivalent to `(at: auto, to: [A], shift: relative-to)`,
- When `relative-to` takes `array`, its form is: `(content | relative, "center" | "top" | "bottom" | "baseline")`
  - If the first element is a `content`, then `to` is set to `relative-to`; if it is `relative`, then `shift` is set to `relative-to`.
  - The second element is equivalent to setting `at` to `horizon`, `top`, `bottom`, `auto`, respectively.

Example

```
#show: el.default-enum-list.with(
  label-baseline: (amount: -50%, at: bottom), // Here, the radio is relative to the label height
  fill: (red, auto),
)
#set enum(numbering: "A.")
+ #lorem(10)
```

A. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Example

```
#let cap-L = text(size: 3em)[L]
#show: el.default-enum-list.with(
  label-baseline: (at: horizon, relative-to: (cap-L, "center")),
  size: (2em, auto),
  label-format: (stroke: red),
  fill: (red, auto),
)
- #cap-L #lorem(2)
```

L Lorem ipsum.

Example

```
#set list(marker: [#box(stroke: red, height: 1em, width: 1em)])
#show: el.default-enum-list.with(
  size: (1.5em, auto, 2em),
  label-baseline: "center",
)
- #lorem(5)
  - - #lorem(5)
- #lorem(5)
```

Lorem ipsum dolor sit amet.

Lorem ipsum dolor sit amet.

Lorem ipsum dolor sit amet.

Removed `alone` in `itemize0.2.0`.

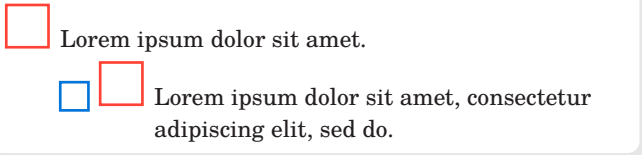
⇄ Changed in 0.3.0

`same-line-style`: sets the alignment of labels on the same line

- If it is `auto` (default), the current setting of `label-baseline` is used, which aligns to the baseline of the item body by default.
- Sets whether to align the labels on the same line to the last label, the first line of the item body, or the whole item body.
  - Shorthand rules for `same-line-style`.
    - `"center"` | `"top"` | `"bottom"` | `"baseline"` : the first line of the item body

Example

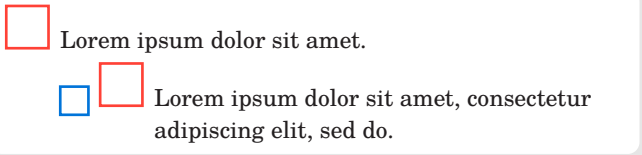
```
#set list(marker: [#context box(stroke:
text.fill, height: 1.2em, width: 1.2em)])
#show: el.default-enum-list.with(
  size: (1.5em, auto, 1.5em),
  fill: (red, blue, red),
  label-baseline: (same-line-style: "center"),
)
- #lorem(5)
  - - #lorem(10)
```



- `"top-item"` | `"horizon-item"` | `"bottom-item"` : the whole item body

Example

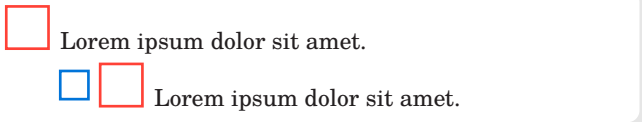
```
#set list(marker: [#context box(stroke:
text.fill, height: 1.2em, width: 1.2em)])
#show: el.default-enum-list.with(
  size: (1.5em, auto, 1.5em),
  fill: (red, blue, red),
  label-baseline: (same-line-style: "horizon-
  item"),
)
- #lorem(5)
  - - #lorem(10)
```



- `"top-label"` | `"horizon-label"` | `"bottom-label"` | `"baseline-label"`: the last label

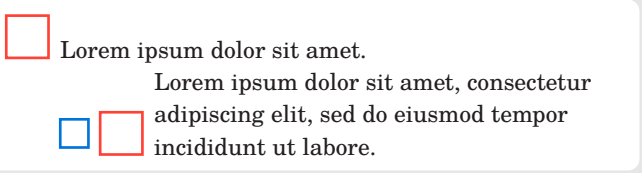
Example

```
#set list(marker: [#context box(stroke:
text.fill, height: 1.2em, width: 1.2em)])
#show: el.default-enum-list.with(
  size: (1.5em, auto, 1.5em),
  fill: (red, blue, red),
  label-baseline: (same-line-style: "center-
  label"),
)
- #lorem(5)
  - - #lorem(5)
```



Example

```
#set list(marker: [#context box(stroke:
text.fill, height: 1.2em, width: 1.2em)])
#show: el.default-enum-list.with(
  size: (1.5em, auto, 1.5em),
  fill: (red, blue, red),
  label-baseline: (auto, (same-line-style:
  "center-label", amount: "bottom-item")),
)
- #lorem(5)
  - - #lorem(15)
```



– More detailed settings can be made via `(amount, at, relative-to)`, similar to `#default-enum-list.label-baseline`.

- In `itemize0.2.0`, by default, the baselines of labels on the same line are all aligned to the baseline of the last label.
- In `itemize0.3.0`, by default, the baselines of labels on the same line are all aligned to the baseline of the item body. You can set `same-line-style` to `"baseline-label"` to restore the default behavior of `itemize0.2.0`.
- Note: in `itemize0.2.0`, the values `"top"`, `"center"`, `"bottom"` are equivalent to the values `"top-label"`, `"center-label"`, `"bottom-label"` in `itemize0.3.0`.

↑ Introduced in 0.3.0

NEW `impact-first-line`: Whether to impact the first line of the label body by `label-baseline`.

- if `auto`, then if `#default-enum-list.hanging-type` is `"paragraph"`, then `true`, else `false`.

Example

```
#let test = [
  + #el.item([$vec(a, b, c)$]) #lorem(10)
]
#table(
  columns: (1fr, 1fr),
  table.header([`impact-first-line: false`],
  [`impact-first-line: true`]),
  [
    #show: el.default-enum-list
    #test
  ],
  [
    #show: el.default-enum-list.with(label-
    baseline: (impact-first-line: true))
    #test
  ],
)
```

`impact-first-line: false`	`impact-first-line: true`
$\begin{pmatrix} a \\ b \\ c \end{pmatrix}$ Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.	$\begin{pmatrix} a \\ b \\ c \end{pmatrix}$ Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

In `itemize0.2.0`, there is no `impact-first-line` property, and by default the item label impacts the first line of the label body.

II.4.6. Label inset (Horizontal movement)

↑ Introduced in 0.3.0

`#default-enum-list.label-inset` (`level-index-property`) is used to horizontally move the item label, without affecting the label width and the inset of the first line.

Example

```
#let test = [
  + #lorem(15)
    + #lorem(2)
  100. #lorem(2)
]
#[
  #show: el.default-enum-list.with(label-inset:
  -.5em, label-width: 2em)
  #test
```

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.
    1. Lorem ipsum.
100. Lorem ipsum.

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.
    1. Lorem ipsum.
100. Lorem ipsum.

```
9  ]
10 #[
11   #show: el.default-enum-list.with(label-width:
     2em)
12   #test
13 ]
```

II.4.7. Formating in any way

⇄ Changed in 0.3.0

`#default-enum-list.label-format` is used to customize item labels in any way. Its complete form is: `(`

```
(format): none   none | content | function | array
(stroke):
(radius):
(outset):
(inset):
(clip):
(align): start   auto | alignment | array | function
(width-style): "real-width" | "label-width" | relative | (
    (amount): "label-width"   "real-width" | "label-width" | relative
    (stretched): false   bool
  )
(height-style): "real-height" | relative | (
    (amount): "real-height"   "real-height" | relative
    (stretched): false   bool
  )
)
```

- All the keys are `level-index-property`.

↑ Introduced in 0.2.0

- Shorthand rules for `label-format`: if `label-format` has no other keys, then `format` does not need to be written. That is, `(label-format: func)` is equivalent to `(label-format: (format: func))`, where `func` is `none` | `content` | `function` | `array`.
  - `content`: displays the content of the current label as `content`.

Example

```
1 #show: el.default-enum-list.with(label-format:
  [#emoji.alien])
2 + #lorem(2)
3 + #lorem(2)
```

👽 Lorem ipsum.
👽 Lorem ipsum.

  - `array`: elements can be a `content`, a `function` (in the form `body => content`, meaning the content of the item label is applied to the function), or an `array` (the `(n-1)`-th element of the array applies to the `n`-th item label).

Example

```
#show: el.default-enum-list.with(label-format:
(emph, [#emoji.alien], ([#emoji.butterfly],
[#emoji.book.orange], auto), auto))
+ #lorem(2)
  + #lorem(2)
    + #lorem(2)
    + #lorem(2)
    + #lorem(2)
+ #lorem(2)
```

1. Lorem ipsum.
   - Lorem ipsum.
     - Lorem ipsum.
     - Lorem ipsum.
       3. Lorem ipsum.
2. Lorem ipsum.

- `function` (level-property): in the form `it => content`, meaning the content of the item label is applied to the function.

Example

```
#show: el.default-enum-list.with(
  label-format: it => {
    if it.tag == "strong" {
      set text(fill: red, weight: "bold")
      it.body
    } else {
      it.body
    }
  })
+ #el.item(tag: "strong") #lorem(2)
+ #lorem(2)
  + #lorem(2)
  + #el.item(tag: "strong") #lorem(2)
```

1. Lorem ipsum.
2. Lorem ipsum.
   1. Lorem ipsum.
   2. Lorem ipsum.

See also the usage of the `#item` method and Section II.3.1.

↑ Introduced in 0.3.0 NEW `stroke`, `radius`, `outset`, `fill`, `inset`, `clip`: The *border* of the label, which are the same as `#box` arguments.

Example

```
#show: el.default-enum-list.with(
  size: 1.5em,
  fill: red,
  label-format: (
    stroke: red,
    fill: (gray, yellow, auto)
))
+ #lorem(2)
  + #lorem(2)
    + #lorem(2)
    + #lorem(2)
+ #lorem(2)
```

1. Lorem ipsum.
   - Lorem ipsum.
     - Lorem ipsum.
     - Lorem ipsum.
2. Lorem ipsum.

↑ Introduced in 0.3.0 NEW `align`, `width-style`, `height-style`: can be used to set the width and height of the current item label, and align within this range according to `align`.

- In `width-style` and `height-style`, you can set `stretched: true` to make the set width and height affect the layout.

Example

```
#show: el.default-enum-list.with(
  fill: red,
  label-format: (
    stroke: red,
    inset: 2pt,
    radius: 1em,
))
+ #lorem(2)
  + #lorem(2)
    + #lorem(2)
    + #lorem(2)
+ #lorem(2)
```

1. Lorem ipsum.
    1. Lorem ipsum.
        1. Lorem ipsum.
        2. Lorem ipsum.
2. Lorem ipsum.

Example

```
#show: el.default-enum-list.with(
  fill: red,
  label-format: (
    stroke: red,
    inset: 2pt,
    radius: 1em,
    width-style: (stretched: true)
))
+ #lorem(2)
  + #lorem(2)
    + #lorem(2)
    + #lorem(2)
+ #lorem(2)
```

1. Lorem ipsum.
    1. Lorem ipsum.
        1. Lorem ipsum.
        2. Lorem ipsum.
2. Lorem ipsum.

Example

```
#show: el.default-enum-list.with(
  label-format: (
    stroke: (red, blue),
    align: center + horizon,
    width-style: (amount: 1.4em, stretched: true),
    height-style: (amount: 1.4em, stretched: true),
    radius: 1em,
  ),
)
#set enum(numbering: "1.A")
+ #lorem(15)
  + #lorem(15)
  + #lorem(6)
+ #lorem(6)
```

1 Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.
    A Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.
    B Lorem ipsum dolor sit amet, consectetur.
2 Lorem ipsum dolor sit amet, consectetur.

Note: the width set here does not affect the maximum width of the item label at the current level (e.g., set via `#item`).

Example

```
#show: el.default-enum-list.with(fill: red,)
+ #lorem(2)
  + #el.item(label-format: (
      inset: 2pt,
      stroke: red,
      width-style: (amount: "label-width",
      stretched: true)
    ))
    #lorem(2)
    + #lorem(2)
    + #lorem(2)
  + #lorem(2)
+ #lorem(2)
```

1. Lorem ipsum.
   1. Lorem ipsum.
      1. Lorem ipsum.
      2. Lorem ipsum.
   2. Lorem ipsum.
2. Lorem ipsum.

II.4.8. Step for enum number

↑ Introduced in 0.3.0

`#default-enum-list.step` (`level-property`) is used to set the step size for `#enum`'s number.

Example

```
#let factorial-step = (..nums) => {
  let numbers = nums.pos()
  let n = numbers.len()
  if n >= 2 {
    return numbers.at(n - 2) + numbers.at(n - 1)
  }
}
#[
  Make enum numbers like factorial: \
  $a_n = a_(n - 2) + a_(n - 1)$, $a_1 = 1, a_2 = 1$.
  #show: el.default-enum-list.with(step: (factorial-step,))
  1. #lorem(2)
  1. #lorem(2)
  + #lorem(2)
  + #lorem(2)
  + #lorem(2)
  + #lorem(2)
]
#[
  Increment enum numbers by a step of 2:
  #show: el.default-enum-list.with(step: 2)
  1. #lorem(2)
  + #lorem(2)
  + #lorem(2)
  + #lorem(2)
]
```

Make enum numbers like factorial:
$a_n = a_{n-2} + a_{n-1}, a_1 = 1, a_2 = 1.$

1. Lorem ipsum.
1. Lorem ipsum.
2. Lorem ipsum.
3. Lorem ipsum.
5. Lorem ipsum.
8. Lorem ipsum.

Increment enum numbers by a step of 2:

1. Lorem ipsum.
3. Lorem ipsum.
5. Lorem ipsum.
7. Lorem ipsum.

II.4.9. Enhancement of `#list.marker`

↑ Introduced in 0.1.0

The `itemize` package improves the parsing of `#list.marker` as a `function` type (supporting label control for each item). It takes the following form:

- `level => (n => content) | array`

Here:

- `level`: The relative nesting level of the `list`, cyclically used.
  - For compatibility with the native `list.marker` behavior, unlike the methods provided by `itemize`, the level here starts from 0 (see `issue#3`).
  - Also, the item index starts from 0.

Example

```
#let test-list = [
  - #lorem(5)
    - #lorem(5)
    - #lorem(5)
  - #lorem(5)
]
#[
  #show: el.default-enum-list.with(label-format: it => [#it.level - #it.n])
  #test-list
]
#[
  #set list(marker: level => n => [#level - #n])
  #show: el.default-enum-list.with(auto-base-level: true)
  #test-list
]
```

1 - 1 Lorem ipsum dolor sit amet.
    2 - 1 Lorem ipsum dolor sit amet.
    2 - 2 Lorem ipsum dolor sit amet.
1 - 2 Lorem ipsum dolor sit amet.

0 - 0 Lorem ipsum dolor sit amet.
    1 - 0 Lorem ipsum dolor sit amet.
    1 - 1 Lorem ipsum dolor sit amet.
0 - 1 Lorem ipsum dolor sit amet.

- `n => content`, `array`: The `marker` for each `item` in the same level.

Example

```
#show: el.default-enum-list
  #let marker = level => {
    if level == 0 { // stand for level 1!!!
      ([#sym.suit.club.filled],
      [#sym.suit.spade.filled],
      [#sym.suit.heart.filled], el.LOOP)
    } else if level == 1 { // stand for level 2!!! The first item's number starts from 0
      n => rotate(24deg * n, box(rect(stroke: 1pt + blue, height: 1em, width: 1em)))
    } else { // others
      [#sym.ballot.check.heavy]
    }
  }
  #set list(marker: marker)
  - #lorem(5)
  - #lorem(5)
    - #lorem(5)
    - #lorem(5)
      - #lorem(5)
      - #lorem(5)
    - #lorem(5)
    - #lorem(5)
```

♣ Lorem ipsum dolor sit amet.
♠ Lorem ipsum dolor sit amet.
  ☐ Lorem ipsum dolor sit amet.
  ◇ Lorem ipsum dolor sit amet.
    ☑ Lorem ipsum dolor sit amet.
    ☑ Lorem ipsum dolor sit amet.
  ◇ Lorem ipsum dolor sit amet.
  ◇ Lorem ipsum dolor sit amet.
  ☐ Lorem ipsum dolor sit amet.
♥ Lorem ipsum dolor sit amet.
  ☐ Lorem ipsum dolor sit amet.
♣ Lorem ipsum dolor sit amet.

```
    - #lorem(5)
  - #lorem(5)
    - #lorem(5)
  - #lorem(5)
```

II.5. Item body formatting

↑ Introduced in 0.2.0

#default-enum-list.body-format is used to customize the item body in any way. Its complete form is (

```
  (style): (:)  dictionary
  (whole): (:)  function | array | (
      (format): none  none | function | array
      (stroke):
      (radius):
      (outset):
      (fill):
      (inset):
      (clip):
      (breakable):
      (sticky):
      (width):
    )
  (outer): (:)
  (inner): (:)
)
```

⇄ Changed in 0.3.0

The item-format in itemize0.2.0 has been moved to the format in body-format. Therefore, item-format is removed in itemize0.2.0.

Unlike the handling in itemize0.2.0, this setting also applies to the styles of the item labels of lists nested inside the item body.

- style: Any #text named arguments (like: fill, size, font, etc.). The *text-style* of the item body.
  - These keys are `level-index-property`. See also #default-enum-list.args.

Example

```
#show: el.default-enum-list.with(body-format: (
  style: (
    fill: (red, blue, auto),
    size: (1.2em, 12pt, auto),
  )
))
+ #lorem(2)
  + #lorem(2)
  + #lorem(2)
    + #lorem(2)
+ #lorem(2)
```

1. Lorem ipsum.
   1. Lorem ipsum.
   2. Lorem ipsum.
      1. Lorem ipsum.
2. Lorem ipsum.

- whole, outer, inner: Format the item body.
  - The keys in outer, inner are `level-index-property`, and in whole are `level-property`.

- `format` (`none`, `function`): The format of the item body.
  - See the `format` in `#default-enum-list.label-format`.

⇄ Changed in 0.3.0

ⓘ **Note**: Use the `format` function (since ver0.3.0) to replace the `item-format` (in ver0.2.0) parameter .

- `stroke`, `radius`, `outset`, `fill`, `inset`, `clip`, `breakable`, `sticky`, `width`: The *border* of the item body, which are the same as the `#block` named elements.
  - **Note**: `width` only works for `whole`.
- Shorthand rules: if there is only `format`, then `format` can be omitted.
  - `whole`: Wrap the entire `#enum` or `#list`

Example

```
#show: el.default-enum-list.with(body-format:
(whole: (stroke: red)))
+ #lorem(2)
+ #lorem(2)
```

1. Lorem ipsum.
2. Lorem ipsum.

  - `outer`: Wrap the item (including the label)

Example

```
#show: el.default-enum-list.with(body-format:
(outer: (stroke: red)))
+ #lorem(2)
+ #lorem(2)
```

1. Lorem ipsum.
2. Lorem ipsum.

  - `inner`: Wrap the item body (excluding the label)

Example

```
#show: el.default-enum-list.with(body-format:
(inner: (stroke: red)))
+ #lorem(2)
+ #lorem(2)
```

1. Lorem ipsum.
2. Lorem ipsum.

Example

```
#show: el.default-list.with(
  label-format: (
    inset: ((left: 0.5em), auto),
    width-style: ((stretched: true), auto),
    format: (strong, auto)
  ),
  body-indent: (2em, auto),
  item-spacing: (0pt, auto),
  body-format: (
    whole: (
      fill: (gray, auto)
    ),
    inner: (
      fill: (white, auto),
```

Item one	Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
Item two	Lorem ipsum.

```
      stroke: (gray.lighten(50%), auto),
      inset: (5pt, auto),
    )
  ),
)
#set par(justify: true)
- #el.item([Item one]) #lorem(10)
- #el.item([Item two]) #lorem(2)
```

- Shorthand rules for `body-format`
  - If `whole`, `outer`, and `inner` are omitted, the default is `outer`.
  - If there is only `format`, then `format` can also be omitted.

Example

```
#show: el.default-enum-list.with(body-format:
(emph, strong, auto))
+ #lorem(2)
  + #lorem(2)
+ + #lorem(2)
```

1. *Lorem ipsum.*
   1. ***Lorem ipsum.***
2. 1. ***Lorem ipsum.***

Example

```
#show: el.default-enum-list.with(body-format:
it => {
  if it.e.tight {
    show: highlight
    set text(fill: red)
    it.body
  } else {
    it.body
  }
})
+ #lorem(2)
+ #lorem(2)
Paragraph
+ #lorem(2)

+ #lorem(2)
```

1. Lorem ipsum.
2. Lorem ipsum.

Paragraph

1. Lorem ipsum.
2. Lorem ipsum.

Example

```
#set enum(numbering: "(a)(i)(1)")
#show: el.default-enum-list.with(
  body-format: (
    outer: (
      fill: (orange.lighten(80%), auto),
      inset: (5pt, 5pt, auto)
    ),
    whole: (
      stroke: (auto, (left: 4pt + orange, rest: 1pt + orange), auto),
      inset: (auto, (bottom: 1pt, left: 2pt), auto),
      outset: (auto, (left: -2pt), auto),
      format: it => {
        if it.level == 2 {
          set text(fill: orange)
          place(right+bottom)[#sym.suit.club.filled]
          it.body
        } else {
          it.body
        }
      }
    )
  ),
)
+ #lorem(15)
  + #lorem(15)
  + + #lorem(15)
+ #lorem(15)
  + #lorem(15)
  + #lorem(15)
```

(a) Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

(i) Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

(ii) (1) Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

(b) Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

(i) Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

(ii) Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Example

```
#show: el.default-enum-list.with(
  label-baseline: ("horizon-item",auto),
  fill: (white, auto),
  weight: ("bold", auto),
  size: (1.2em, auto),
  label-format: (
    width-style: ((stretched: true, amount: 2em), auto),
    height-style: ((stretched: true, amount: 2em), auto),
    fill: (green, auto),
    radius: (2em, auto),
    align: (center + horizon, auto),
  ),
  body-format: (
    inner: (
      stroke: ((left: 3pt + green, rest: .5pt + green), auto),
      inset: ((left: 0.5em, y: 1em), auto),
    ),
```

1. Lorem ipsum dolor sit amet.
2. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.
3. Lorem ipsum dolor sit amet.

```
  ),
)
#set enum(numbering: "1")
+ #lorem(5)
+ #lorem(15)
+ #lorem(5)
```

Example

```
#show: el.default-enum-list.with(
  fill: (white, auto),
  weight: ("bold", auto),
  size: (1.2em, auto),
  label-format: (
    width-style: ((stretched: true, amount: 2em), auto),
    height-style: ((stretched: true, amount: 2em), auto),
    fill: (blue, auto),
    radius: (2em, auto),
    align: (center + horizon, auto),
  ),
  body-format: (
    outer: (
      stroke: ((left: 3pt + blue, rest: .5pt + blue), auto),
      inset: ((left: 2pt), auto),
      outset: ((left: -2em - 0.5em), auto),
    ),
  ),
)
#set enum(numbering: "1")
+ #lorem(5)
+ #lorem(25)
+ #lorem(5)
```

1. Lorem ipsum dolor sit amet.
2. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus.
3. Lorem ipsum dolor sit amet.

II.6. Horizontal spacing setting

II.6.1. Indentation of item

↑ Introduced in 0.1.0

`#default-enum-list.indent` (`level-index-property`) is used to set the indentation of the item.

⇄ Changed in 0.3.0

Now the setting of `#default-enum-list.indent` affects the position of the whole border.

Example

```
#let test = [
  + #lorem(2)
  + #lorem(2)
]
#table(columns: (1fr, 1fr),
table.header([v0.3.0], [v0.2.0]))[
```

v0.3.0	v0.2.0
1. Lorem ipsum. 2. Lorem ipsum.	1. Lorem ipsum. 2. Lorem ipsum.

```
    #show: el.default-enum-list.with(indent: 1em,
    body-format: (whole: (stroke: red)))
    #test
  ][
    #import  "@preview/itemize:0.2.0" as el2
    #show: el2.default-enum-list.with(indent:
    1em, body-format: (whole: (stroke: red)))
    #test
  ]
```

II.6.2. Indentation of item body

↑ Introduced in 0.1.0

`#default-enum-list.body-indent` (`level-index-property`) is used to set the spacing between the item label and body.

Example

```
#let test = [
  + #lorem(10)
    + #lorem(5)
  + #lorem(10)
]
#table(columns: (1fr, 1fr),
table.header([default], [paragraph]))[
  #show: el.default-enum-list.with(body-indent:
  1em, body-format: (whole: (stroke: red)))
  #test
][
  #show: el.paragraph-enum-list.with(body-
  indent: 1em, body-format: (whole: (stroke:
  red)))
  #test
]
```

default	paragraph
1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do. 1. Lorem ipsum dolor sit amet. 2. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.	1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do. 1. Lorem ipsum dolor sit amet. 2. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

II.6.3. Indentation of item label

↑ Introduced in 0.2.0

`#default-enum-list.label-indent` (`level-index-property`) is used to set the the indentation of the item label.

Example

```
#let test = [
  + #lorem(10)
    + #lorem(5)
  + #lorem(10)
]
#table(columns: (1fr, 1fr),
table.header([default], [paragraph]))[
  #show: el.default-enum-list.with(label-
  indent: 1em, body-format: (whole: (stroke:
  red)))
  #test
][
```

default	paragraph
1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do. 1. Lorem ipsum dolor sit amet. 2. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.	1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do. 1. Lorem ipsum dolor sit amet. 2. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

```
    #show: el.paragraph-enum-list.with(label-
10  indent: 1em, body-format: (whole: (stroke:
    red)))
11  #test
12 ]
```

II.6.4. Body margin and item width

↑ Introduced in 0.3.0

`#default-enum-list.body-margin` (`level-index-property`) is used to set the (left and right) margin of the item body.

Based on the `itemize` design, `label-indent`, the label's content, and `body-indent` belong to the label's body.

- The left margin of the item body starts from the left of the label's body.
- The right margin of the item body ends at the right of the item body. This is the same as the `enum-margin` in `itemize0.2.0`.

⇄ Changed in 0.3.0

ⓘ `enum-margin` in `itemize0.2.0` is removed.

⇄ Changed in 0.3.0

ⓘ `is-full-width` in `itemize0.2.0` is also removed.

In Typst 0.15.1, some `block-level` elements (like `#enum`, `#list`, `#terms`, `#pad`) are always expanded to the full width of the parent element if the parent element is like `page`.

If you want to set `#enum` or `#list` be full width in any parent element, you can set `body-format: (whole: (width: 100%))`.

Example

```
1  #[
2    #show: el.default-enum-list.with(body-format:
     (whole: (width: 100%, stroke: red)))
3    *full width*:
4    #box([
5      + #lorem(5)
6      + #lorem(2)
7    ])
8  ]
9  #[
10   #show: el.default-enum-list.with(body-format:
     (whole: (stroke: red)))
11   *auto width*:
12   #box([
13     + #lorem(5)
14     + #lorem(2)
15   ])
16 ]
```

full width:

1. Lorem ipsum dolor sit amet.
2. Lorem ipsum.

auto width:

1. Lorem ipsum dolor sit amet.
2. Lorem ipsum.

`body-margin` can control the starting alignment position of the `block-level` element in the item body and the actual `100%` width.

- **Note**: In general, the layout of `inline-level` elements in the item body are not affected by `body-margin` (except for the actual `100%` width).
- The layout of `inline-level` elements usually are affected by `line-indent` and `hanging-indent`.

For example, for `#paragraph-enum-list`, the default value of `body-margin` is the same as `#default-enum-list`, but the `inline-level` elements in `#paragraph-enum-list` are aligned to the left of the item label. See the example below.

Example

```
#show: el.paragraph-enum-list.with(
  body-format: (
    whole: (stroke: red, inset: 2pt),
    inner: (stroke: blue, inset: 2pt),
  ),
)
+ #lorem(15)

  #lorem(15)

  #block(stroke: 1pt, inset: 2pt, width: 100%)[
    #lorem(15)
  ]
+ #lorem(2)
```

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

   Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

   Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

2. Lorem ipsum.

Its complete form is: (

(left): auto `auto` | `relative`

(right): 0pt `auto` | `relative`

)

- Shorthand rules: if `body-margin` takes `auto` | `relative`, then both its `left` and `right` values are set to `body-margin`.
- `left` sets the left margin of the item body. Its `ratio` part is relative to the whole label's body width, i.e., the rendered width of the label (`#default-enum-list.label-width`) + `label-indent` + `body-indent`.
  - If `body-margin.left` is `auto`, its default value is the label's body width; in addition, it is also affected by `#default-enum-list.label-width`.
  - Note: if the value set for `body-margin.left` is less than the label's body width, the item label and the item body may overlap (Typst does not yet support z-axis layout). However, as long as no background color is added, this is visually invisible.

Example

```
#show: el.paragraph-enum-list.with(
  body-format: (
    whole: (stroke: red, inset: 2pt),
    inner: (stroke: blue, inset: 2pt),
  ),
  body-margin: (0pt, auto)
)
+ #el.item(body-format: (inner: (fill:
gray.lighten(70%)))) #lorem(15)

  #lorem(15)

  #block(stroke: 1pt, inset: 2pt)[
    #lorem(15)
  ]
+ #lorem(2)
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

2. Lorem ipsum.

- `right` sets the right margin of the item body. Its `ratio` part is relative to the width of its parent container.

⇄ Changed in 0.3.0

Now the setting of `body-margin.right` affects the position of the whole and outer borders.

Example

```
#show: el.default-enum-list.with(
  body-margin: ((right: 2em), auto),
  body-format: (whole: (stroke: red, inset: 2pt), outer: (stroke: blue))
)
+ #lorem(10)
+ #lorem(3)
```

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
2. Lorem ipsum dolor.

With this attribute, it is now very easy to implement some lists with very special layouts.

1. The body of all nested lists is aligned to the left.

Example

```
#show: el.default-enum-list.with(
  body-margin: 0pt,
  indent: (2.5em, auto),
  label-indent: it => { -it.label-width.current - it.e.body-indent },
)
#set enum(full: true)
+ Level One
  + Level Two
    + #lorem(2)
  + Level Two
+ Level One
```

1. Level One
1.1. Level Two
1.1.1. Lorem ipsum.
1.2. Level Two
2. Level One

2. Align the (first-level) item body and the first line of the paragraph, e.g., indent both by 2em.

Example

```
#let indent = 2em
#set par(first-line-indent: (amount: indent, all: true))
#show: el.default-enum-list.with(
  body-margin: (0pt, auto),
  indent: (indent, auto),
  label-indent: it => {
    if it.level == 1 {
      -it.label-width.current - it.e.body-indent
    } else {
      auto
    }
  },
)
#lorem(15)
+ #lorem(15)
  + #lorem(15)
+ #lorem(15)
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.
   1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.
2. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

3. Align the (first level) item body and the first line of the paragraph with the `paragraph` hanging style.

Example

```
#let indent = 2em
#set par(first-line-indent: (amount: indent, all: true))
#show: el.default-enum-list.with(
  body-margin: (0pt, auto),
  indent: (0pt, indent, auto),
  label-indent: it => {
    if it.level == 1 {
      indent - it.label-width.current - it.e.body-indent
    } else {
      auto
    }
  },
)
#lorem(15)
+ #lorem(15)
  + #lorem(15)
+ #lorem(15)
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.
    1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.
2. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

II.7. Vertical spacing setting

II.7.1. Spacing of the whole item

↑ Introduced in 0.3.0

`#default-enum-list.whole-spacing` (`level-property`) is used to set the above and below spacing of the current `#enum` or `#list`. Its complete form is: `(`

`(above):` **`auto`** `auto` | `relative` | `fraction`

`(below):` **`auto`** `auto` | `relative` | `fraction`

`)`

⇄ Changed in 0.3.0

> This is a renamed argument of `enum-spacing` in `itemize0.2.0`.

- Shorthand rules: if `whole-spacing` takes `auto` | `relative` | `fraction`, then both its `above` and `below` values are set to `whole-spacing`.

Example

```
#show: el.default-enum-list.with(whole-spacing: 2em)
Paragraph
+ #lorem(2)
+ #lorem(2)
Paragraph
```

Paragraph

1. Lorem ipsum.
2. Lorem ipsum.

Paragraph

> For the meaning of the `spacing` (`above` and `below`) of `block-level` elements and the meaning of setting it to `auto`, see `#block.above` and `#block.below`.

II.7.2. Tight mode

↑ Introduced in 0.3.0

When `whole-spacing` is `auto`, the `above` and `below` spacing of the whole item can be controlled by `#default-enum-list.tight-mode` (`level-property`).

> **Background**: For the current `#enum` or `#list`, the above spacing (denoted as `above-spacing`) and the below spacing (denoted as `below-spacing`) are determined by the following cases:
>
> 1. The current `#enum` or `#list` is `tight` (i.e. `enum.tight` or `list.tight` is `true`, see `#default-enum-list.tight-item-mode`);
> 2. The current `#enum` or `#list` is not `tight` (i.e. `enum.tight` or `list.tight` is `false`, see `#default-enum-list.tight-item-mode`);
> 3. There is a `parbreak()` between the above paragraph and the current `#enum` or `#list`: like
>
> ```
> 1 The preceding paragraph.  // below has a `parbreak()`
> 2
> 3 + items
> ```
>
> 4. There is no `parbreak()` between the above paragraph and the current `#enum` or `#list`: like
>
> ```
> 1 The preceding paragraph.
> 2 + items
> ```
>
> - Combining these cases, we call
>   - case 1 with case 3 **par-tight** (i.e. `tight` with a `parbreak()` above),
>   - case 2 with case 3 **par-not-tight**,
>   - case 1 with case 4 **tight**,
>   - case 2 with case 4 **not-tight**.
> - Note: In native Typst behavior, cases **par-not-tight**, **par-tight** and **not-tight** are the same.

Its complete form is: `(`

`(tight):` **`auto`** `auto` | `relative` | `fraction` | `array`
`(not-tight):` **`auto`** `auto` | `relative` | `fraction` | `array`
`(par-tight):` **`auto`** `auto` | `relative` | `fraction` | `array`
`(par-not-tight):` **`auto`** `auto` | `relative` | `fraction` | `array`

`)`

- Here, if `tight` is an `array`, then the array needs 2 elements representing the values of the `above` and `below` spacing respectively; if it is `auto` | `relative` | `fraction`, then the values of the `above` and `below` spacing are both set to `tight`. `not-tight`, `par-tight`, `par-not-tight` follow the same rule.
- `tight`, `not-tight`, `par-tight`, `par-not-tight` represent the **tight** case, **not-tight** case, **par-tight** case, **par-not-tight** case, respectively.
- **Note**: In order to set `below` spacing from `par-tight` and `par-not-tight`, you need to add in the document:

```
1 #show: el.config.auto-detect-tight
```

Example

```
#let first-level-tight-mode = (
  tight: (auto, 0.65em),
  par-tight: (auto, auto),
  not-tight: (1.5em, 1.5em),
  par-not-tight: (2em, 2em),
)
#show: el.default-enum-list.with(
  tight-mode: (first-level-tight-mode, auto),
  body-format: (whole: (stroke: red)),
)
#show: el.config.auto-detect-tight

*config: tight-mode*

*Tight*: #lorem(2) *with parbreak*

+ #lorem(2) *without parbreak*
+ #lorem(2)
End paragraph

#lorem(2) *without parbreak*
+ #lorem(2) *without parbreak*
+ #lorem(2)
End paragraph
#divider()
*Not Tight*: #lorem(2) *with parbreak*

+ #lorem(2) *with parbreak*

+ #lorem(2)
End paragraph

#lorem(2) *without parbreak*
+ #lorem(2) *with parbreak*

+ #lorem(2)
End paragraph
```

config: tight-mode

Tight: Lorem ipsum. **with parbreak**

1. Lorem ipsum. **without parbreak**
2. Lorem ipsum.

End paragraph

Lorem ipsum. **without parbreak**

1. Lorem ipsum. **without parbreak**
2. Lorem ipsum.

End paragraph

Not Tight: Lorem ipsum. **with parbreak**

1. Lorem ipsum. **with parbreak**
2. Lorem ipsum.

End paragraph

Lorem ipsum. **without parbreak**

1. Lorem ipsum. **with parbreak**
2. Lorem ipsum.

End paragraph

- Shorthand rules
  - `auto`: Use the native Typst behavior of `#enum` or `#list`.
    - for the **tight** case: `above` spacing is `#enum.spacing` or `#list.spacing` (if `auto`, representing `#par.leading`); for the other cases it is `auto` (representing `#par.spacing`).
    - `below` spacing is `auto` (representing `#par.spacing`).
  - `"always-tight"`: `above` and `below` spacing are `#par.leading` and `auto`.
  - `"never-tight"`: `above` and `below` spacing are `#par.spacing` and `auto`
  - `"compact-tight"`: `above` and `below` spacing are both `#par.spacing`.

Example

```
#show: el.default-enum-list.with(
  tight-mode: ("compact-tight", auto),
  body-format: (whole: (stroke: (red, blue, auto)))
)
Paragraph
+ #lorem(2)

  Paragraph // below has a `parbreak()`

  + #lorem(2)
  + #lorem(2)
  End Paragraph
+ #lorem(2)
End Paragraph
```

Paragraph

1. Lorem ipsum.

   Paragraph

   1. Lorem ipsum.
   2. Lorem ipsum.

   End Paragraph
2. Lorem ipsum.

End Paragraph

II.7.3. Spacing of each item

↑ Introduced in 0.2.0

`#default-enum-list.item-spacing` (`level-index-property`) is used to set the spacing between each item. This parameter can take `auto` | `relative` | `fraction` or `(`

`(above): auto` `auto` | `relative` | `fraction`

`(below): auto` `auto` | `relative` | `fraction`

`)`

↑ Introduced in 0.3.0

For a value of type `dictionary`

- The `above` spacing is ignored for the first item, but the `below` spacing is added to the last item.
- The spacing between the previous and current items is the previous `below` plus the current `above`.
  - **Note**: This differs from setting `item-spacing` to a concrete value.

Example

```
#show: el.default-enum-list.with(item-spacing: (below: 1fr))
#lorem(10)
#block(stroke: red, height: 3cm, width: 100%)[
  + Answer one
  + Answer two
  + Answer three
]
#lorem(10)
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

1. Answer one
2. Answer two
3. Answer three

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Example

```
#show: el.default-enum-list.with(item-spacing: 1fr)
#lorem(10)
#block(stroke: red, height: 3cm, width: 100%)[
  + Answer one
  + Answer two
  + Answer three
]
#lorem(10)
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

1. Answer one
2. Answer two
3. Answer three

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

- For values of type `auto` | `relative` | `fraction`, the meaning is the same as `block.spacing`.

II.7.4. Item tight mode

↑ Introduced in 0.3.0

When `item-spacing` is `auto`, the spacing can be controlled by `#default-enum-list.tight-item-mode` (`level-index-property`).

Background: The spacing between items is determined by the following cases:

1. **tight**: There is no `parbreak()` between items, like:

```
+ item one // no `parbreak()` below
+ item two // no `parbreak()` below
+ item three
```

2. **not-tight**: There is a `parbreak()` between items, like:

```
+ item one // has a `parbreak()` below

+ item two
+ item three
```

Its complete form is: (

(tight): auto `auto` | `relative` | `fraction`
(not-tight): auto `auto` | `relative` | `fraction`

)

- If `tight` is `auto`, use `#par.leading`
- If `not-tight` is `auto`, use `#par.spacing`
- Use the values of `tight` and `not-tight` for the **tight** case and the **not-tight** case, respectively.

Example

```
#show: el.default-enum-list.with(
  tight-item-mode: ((tight: auto, not-tight: 2em), auto),
  body-format: (whole: (stroke: red)),
)
*Tight*:
+ #lorem(2)
```

Tight:

1. Lorem ipsum.
2. Lorem ipsum.

Not-tight:

1. Lorem ipsum.
2. Lorem ipsum.

```
  + #lorem(2)
*Not-tight*:
  + #lorem(2)

  + #lorem(2)
```

- Shorthand rules
  - `auto`: Use the native Typst behavior of `#enum` or `#list`.
    - The spacing uses `#enum.spacing` or `#list.spacing` (if `auto`, representing `#par.leading` for the **tight** case and `#par.spacing` for the **not-tight** case).
  - `"always-tight"`: The spacing is `#par.leading`.
  - `"never-tight"`: The spacing is `#par.spacing`.

Example

```
#show: el.default-enum-list.with(
  tight-item-mode: "always-tight",
  body-format: (whole: (stroke: red)),
)
*Tight*:
+ #lorem(2)
+ #lorem(2)
*Not-tight*:
+ #lorem(2)

+ #lorem(2)
```

Tight:

1. Lorem ipsum.
2. Lorem ipsum.

Not-tight:

1. Lorem ipsum.
2. Lorem ipsum.

`#default-enum-list.tight-item-mode` can also be implemented through the `e.tight` of the `function` (level-property) of `#default-enum-list.item-spacing`.

Example

```
#show: el.default-enum-list.with(
  item-spacing: it => {
    if it.level == 1 {
      if not it.e.tight {
        (below: 1fr)
      } else { auto }
    } else { auto }
  },
)
#lorem(10)
#block(stroke: red, height: 3.5cm, width: 100%)[
  *Tight*:
  + Answer one
  + Answer two
  + Answer three
  #lorem(5)
]
#block(stroke: red, height: 3.5cm, width: 100%)[
  *Not tight*:
  + Answer one

]
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Tight:

1. Answer one
2. Answer two
3. Answer three

Lorem ipsum dolor sit amet.

Not tight:

1. Answer one

2. Answer two

3. Answer three

Lorem ipsum dolor sit amet.

```
22   + Answer two
23   + Answer three
24   #lorem(5)
25 ]
```

II.8. Reference

↑ Introduced in 0.1.0

To enable this feature, use the method `#config.ref` like the following at the beginning of the document:

```
1 #show: el.config.ref
```

II.8.1. Reference enum number

In the enum item you want to reference, label it with `<some-label>`, and then use `@<some-label>` to reference the enum number of that item.

> If the enum number cannot be retrieved correctly, use the method `#elabel` to label the enum item.

Example

```
1 #show link: set text(fill: orange)
2 #show: el.config.ref
3 #set enum(numbering: "(1).(a)")
4 #show: el.default-enum-list
5 + #lorem(5)
6   + #lorem(5)
    + $a^2+b^2 = c^2$ #el.elabel(<item:eq>) // use
7   `elabel`, otherwise it will label the
    equation.
8 + #lorem(5) <item:one>
9 The Conclusion @item:one and the equation in
  @item:eq[Item] are important.
```

(1) Lorem ipsum dolor sit amet.
  (a) Lorem ipsum dolor sit amet.
  (b) $a^2 + b^2 = c^2$
(2) Lorem ipsum dolor sit amet.

The Conclusion (2) and the equation in Item (b) are important.

II.8.2. Configuration of reference

The `ref-numbering` and `supplement` in `#default-enum-list` and `#item`, as well as `#config.ref.numbering` and `#config.ref.supplement`, can be used to set the format of references.

↑ Introduced in 0.3.0

The complete form of `ref-numbering` (`base-property`) is: `(`

`(numbering): none` `none` | `str` | `function`

`(full): auto` `auto` | `bool` | `"rel"`

`)`

- For the meaning of `numbering`, see `#enum.numbering`. Its purpose is to apply the `numbering` to the referenced enum number (or the full enum numbers, i.e., including parent enum numbers). `none` means it is not applied.
- `full`. Determine to display the full enum numbers.
  - `true` displays the full number (including parent levels).

Example

```
#show: el.default-enum-list.with(ref-numbering:
(numbering: "(1).(a)", full: true))
#show: el.config.ref
#set enum(numbering: "1.a.")
+ #lorem(2)
+ #lorem(2)
  + #lorem(2)
  + #lorem(2) <item:full>

Test: @item:full.
```

1. Lorem ipsum.
2. Lorem ipsum.
   a. Lorem ipsum.
   b. Lorem ipsum.

Test: (2).(b).

- `false` displays only the current item's number.
- `"ref"` displays the reference number items in a relative manner (i.e., if the current item and the reference item have the same parent level (in the same parent enum), the same parent level is not displayed.)

Example

```
#show: el.default-enum-list
#show: el.config.ref.with(full: "rel")
#set enum(numbering: "1.a.i.")
#show link: set text(fill: orange)
+ #lorem(5) <p:a>
  + #lorem(5) <p:b>
  + #lorem(5)
    + #lorem(5) <p:c>
    + #lorem(5) Rel-ref: @p:a, @p:b and @p:c
  + #lorem(5) Rel-ref: @p:a, @p:b and @p:c
    + #lorem(5) Rel-ref: @p:a, @p:b and @p:c
  #lorem(10)
  + Rel-ref: @p:a, @p:b and @p:c
+ #lorem(5) Rel-ref: @p:a, @p:b and @p:c
  + Rel-ref: @p:a, @p:b and @p:c
Paragraph
+ #lorem(5) Rel-ref: @p:a, @p:b and @p:c
+ #lorem(5)
```

1. Lorem ipsum dolor sit amet.
   a. Lorem ipsum dolor sit amet.
   b. Lorem ipsum dolor sit amet.
      i. Lorem ipsum dolor sit amet.
      ii. Lorem ipsum dolor sit amet. Rel-ref: 1., a. and i.
   c. Lorem ipsum dolor sit amet. Rel-ref: 1., a. and b.i.
      i. Lorem ipsum dolor sit amet. Rel-ref: 1., a. and b.i.

   Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
   a. Rel-ref: 1., a. and b.i.
2. Lorem ipsum dolor sit amet. Rel-ref: 1., 1.a. and 1.b.i.
   a. Rel-ref: 1., 1.a. and 1.b.i.

Paragraph
1. Lorem ipsum dolor sit amet. Rel-ref: 1., 1.a. and 1.b.i.
2. Lorem ipsum dolor sit amet.

- Shorthand rules: `ref-numbering` can take `none` | `str` | `function`, which means `numbering` is `ref-numbering`.

↑ Introduced in 0.3.0

`supplement` (`level-index-property`) can take

- `auto`: No supplementary content will be added.
- `content`: Uses `content` as supplementary content. The effect is that when referencing `#enum` or `#list` labels, then content is added before the label.
- `dictionary`: (

  `(prefix): []` `content`

  `(suffix): []` `content`

)

  The effect is that when referencing enum (list) labels, `prefix` content is added before the label, and `suffix` content is added after the label.

Application rule (`ref-numbering`): consider, in order, the `ref-numbering` in `#item.args`, the `ref-numbering` in `#default-enum-list.enum-config`, and `#default-enum-list.ref-numbering`:

- `numbering`: if their `numbering` are all `none`, then `#config.ref.numbering` is used, and if its value is `auto`, then `enum.numbering` is used.
- `full`: if their `full` are all `auto`, then `#config.ref.full` is used, and if its value is `auto`, then `enum.full` is used.

Application rule (`supplement`):

- The native Typst `#ref.supplement` is always displayed to the left of the referenced content.
- Then the content of `#config.ref.supplement` is considered.
- Finally, consider, in order, the `supplement` in `#default-enum-list.enum-config` and `#default-enum-list.supplement`; if the former is `auto` or `none`, then the latter is used. If the `label` is placed after `#item`, then the `supplement` in `#item.args` is used with priority. For example:

```
+ #el.item(supplement: [...]) <some-label> body <other-label>
```

For `@some-label`, the content of `[...]` is used with priority (as long as it is not `auto` or `none`); for `@other-label`, the `supplement` in `#item.args` is ignored.

- **Note**: this content is passed as a whole to `#config.ref.supplement`. In particular, if `#config.ref.supplement` is a `function` (level-property), then for `it => content`, this part of the content is `it.body`.

You can set the reference format and supplement for each `#enum` in the following ways.

↑ Introduced in 0.3.0

NEW Use `#default-enum-list.ref-numbering` and `#default-enum-list.supplement` to set the reference format and supplement for each `#enum`.

Example

```
#show link: set text(fill: orange)
#set enum(numbering: "1.a.")
#[
  #show: el.default-enum-list.with(
    ref-numbering: "(1).(a)",
    supplement: [Assumption]
  )
  + #lorem(2) <item:ass>
    + #lorem(2)
    + #lorem(2) <item:ass-s>
  + #lorem(2)
]
#[
  #show: el.default-enum-list.with(
    ref-numbering: "1.a",
    supplement: [#sym.section]
  )
  + #lorem(2)
  + #lorem(2) <item:sec>
    + #lorem(2) <item:sec-s>
    + #lorem(2)
]
Test: @item:ass, @item:ass-s.

Test: @item:sec, @item:sec-s.
```

1. Lorem ipsum.
   a. Lorem ipsum.
   b. Lorem ipsum.
2. Lorem ipsum.

1. Lorem ipsum.
2. Lorem ipsum.
   a. Lorem ipsum.
   b. Lorem ipsum.

Test: Assumption (1), Assumption (b).

Test: § 2, § a.

↑ Introduced in 0.2.0

Use `#config.ref.numbering` and `#config.ref.supplement` to set the reference format and supplement for all current references.

Note: this method applies to the format used when referencing (i.e., using `@<some-label>` or `#ref(<some-label>)`), regardless of the location of the `#enum`.

Example

```
#show link: set text(fill: orange)
#set enum(numbering: "1.a.")
#show: el.default-enum-list
+ #lorem(2) <item:ass1>
  + #lorem(2)
  + #lorem(2) <item:ass1-s>
+ #lorem(2)
Paragraph

+ #lorem(2)
+ #lorem(2) <item:sec1>
  + #lorem(2) <item:sec1-s>
  + #lorem(2)
#[
  #show: el.config.ref.with(
    numbering: "(1).(a)",
    supplement: [Assumption]
  )
  - Test: @item:ass1, @item:ass1-s.
  - Test: @item:sec1, @item:sec1-s.
]
Paragraph

#[
  #show: el.config.ref.with(
    numbering: "1.a",
    supplement: [#sym.section]
  )
  - Test: @item:ass1, @item:ass1-s.
  - Test: @item:sec1, @item:sec1-s.
]
Paragraph
// default
- Test: @item:ass1, @item:ass1-s.
- Test: @item:sec1, @item:sec1-s.
```

1. Lorem ipsum.
   a. Lorem ipsum.
   b. Lorem ipsum.
2. Lorem ipsum.

Paragraph

1. Lorem ipsum.
2. Lorem ipsum.
   a. Lorem ipsum.
   b. Lorem ipsum.

- Test: Assumption (1), Assumption (b).
- Test: Assumption (2), Assumption (a).

Paragraph

- Test: § 1, § b.
- Test: § 2, § a.

Paragraph

- Test: 1., b..
- Test: 2., a..

↑ Introduced in 0.3.0

With `#default-enum-list.supplement` and `#config.ref.supplement` (`function` (level-property)), you can customize the reference format of `#enum`.

- The function form: `it => content`.
- Access the following values available on `it`:
  - `it.body`: The referenced `#enum` labels.
  - `it.level`: The level of the item, starting from 1.
  - `it.n`: The index of the item, starting from 1.
  - `it.n-last`: The index of the last item in the current level.
  - `it.tag`: The tag of the item.
  - `it.elem-tag`: The tag of the `#enum` or `#list`.

- `it.e`: The field of the current construct element (`#enum` or `#list`), but excluding the `children`.
- `it.hanging-type`: The hanging type of the current level.
- `it.number`: The number of the current label if it is a `#enum`.
- `it.target`: The target of the referenced label.
- `it.numbers`: The full numbers of the referenced item (including the parent `#enum` numbers).

Example

```
#show: el.config.ref
#let theorem(doc) = {
  let number = context counter(figure.where(kind: "thm")).display("1")
  figure(
    kind: "thm",
    supplement: "Theorem",
    block(
      stroke: (left: 4pt + blue, rest: 1pt + blue),
      inset: 5pt,
      width: 100%,
      {
        set align(left)
        [
          #set text(fill: blue)
          #strong[Theorem #number.]
        ]
        h(.65em, weak: true)
        set enum(numbering: "1).a)")
        show: el.default-enum-list.with(
          supplement: it => if it.level in (1, 2) {
            [Theorem #number #it.body]
          } else { it.body },
          ref-numbering: (numbering: "(1).(a)",
          full: true),
        )
        doc
      },
    ),
  )
}
#theorem([
  #lorem(5) The following conclusions are hold.
  + + #lorem(2) <item:thmA1>
    + #lorem(2)
  + #lorem(2)
])
See @item:thmA1.
```

Theorem 1. Lorem ipsum dolor sit amet. The following conclusions are hold.
1) a) Lorem ipsum.
   b) Lorem ipsum.
2) Lorem ipsum.

See Theorem 1 (1).(a).

Example

```
#show link: set text(fill: orange)
#set enum(numbering: "1.a.A.", full: true)
#show: el.default-enum-list
+ #lorem(2) <item:default0>
  + #lorem(2)
  + #lorem(2)
+ #lorem(2)
Paragraph
+ + #el.item(elem-tag: "ref:mark") #lorem(2) <item:mark0>
  + #lorem(2)
    + #lorem(2) <item:default1>
    + #lorem(2)
  + #lorem(2) <item:mark1>

#[
  #show: el.config.ref.with(
    numbering: "(1).(a).(A)",
    supplement: it => if it.at("elem-tag", default: none) == "ref:mark" {
      set text(style: "italic", fill: red)
      [#sym.section #std.numbering("-1-a-A", ..it.numbers)]
    } else {
      it.body // use default
    }
  )
  Customize:
  - Test (default): @item:default0, @item:default1
  - Test (ref-mark): @item:mark0, @item:mark1.
]
Default:
- Test (default): @item:default0, @item:default1
- Test (ref-mark): @item:mark0, @item:mark1.
```

1. Lorem ipsum.
   1.a. Lorem ipsum.
   1.b. Lorem ipsum.
2. Lorem ipsum.

Paragraph

1. 1.a. Lorem ipsum.
   1.b. Lorem ipsum.
      1.b.A. Lorem ipsum.
      1.b.B. Lorem ipsum.
   1.c. Lorem ipsum.

Customize:

- Test (default): (1), (1).(b).(A)
- Test (ref-mark): *§ -1-a*, *§ -1-c*.

Default:

- Test (default): 1., 1.b.A.
- Test (ref-mark): 1.a., 1.c..

Example

```
#let my-counter = counter("my")
#show: el.default-enum-list.with(
  supplement: it => {
    [#std.numbering("1.", ..(my-counter).at(it.target))#it.body]
  },
)
#set enum(numbering: "i.a.", full: true)
Paragraph
#my-counter.step()
+ #lorem(2)
  + #lorem(5)
  + #lorem(4) <item:testa>
+ #lorem(3)

Paragraph
#my-counter.step()
+ #lorem(2)
+ #lorem(3)
  + #lorem(5) <item:testb>
```

Paragraph

i. Lorem ipsum.
   i.a. Lorem ipsum dolor sit amet.
   i.b. Lorem ipsum dolor sit.
ii. Lorem ipsum dolor.

Paragraph

i. Lorem ipsum.
ii. Lorem ipsum dolor.
   ii.a. Lorem ipsum dolor sit amet.

Test: 1.i.b.

Test: 2.ii.a.

```
20
21 Test: @item:testa
22
23 Test: @item:testb
```

↑ Introduced in 0.3.0

NEW Use the `ref-numbering` and `supplement` of `#item.args` to set the reference format for the current `#enum` and the reference supplement for the items of the current `#enum`.

- The `ref-numbering` of `#item.args` must be placed in the first item of the whole item to take effect, and it only applies to the current `#enum`; nested `#enum`s do not use this setting.
- For the `supplement` of `#item.args` to take effect, the `label` needs to be placed after `#item`.

Example

```
1  #show: el.config.ref
2  #show link: set text(fill: orange)
3  #set enum(numbering: "1.a.i.")
4  #show: el.default-enum-list
5  + #lorem(5)
6    + #el.item(ref-numbering: "(a)", supplement: [Euler]) <item:a> *Euler*  #lorem(5)
7    + #lorem(5) <item:b>
8      + #el.item(ref-numbering: (full: true)) #lorem(5)
9      + #lorem(5) <item:b-s>
10   + #lorem(5)
11 Test: @item:a, @item:b[Item], @item:b-s.
```

1. Lorem ipsum dolor sit amet.
   a. **Euler** Lorem ipsum dolor sit amet.
   b. Lorem ipsum dolor sit amet.
      i. Lorem ipsum dolor sit amet.
      ii. Lorem ipsum dolor sit amet.
   c. Lorem ipsum dolor sit amet.

Test: Euler (a), Item (b), 1.b.ii..

II.8.3. Reference item label

↑ Introduced in 0.3.0

If you use `#item` to change the content of the current item label (Section II.3.2), referencing that content is now also supported, but the `label` needs to be placed after `#item`.

Example

```
1 #show link: set text(fill: orange)
2 #show: el.config.ref
3 #show: el.default-enum-list
4 - #el.item([*#sym.section*], supplement: [Assumption])<l:my> #lorem(15)
5 - #lorem(2)
6 See: @l:my.
```

§ Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

- Lorem ipsum.

See: Assumption §.

II.9. Resuming enum number

In `itemize`, it is allowed to specify which `#enum` numbers continue to use the numbers from other `#enum`s.

II.9.1. Resuming by level or by specifying `#enum`

To enable this feature, set `#default-enum-list.auto-resuming` (`level-property`) to `auto`. Then you can use the following methods:

↑ Introduced in 0.2.0

1. `#resume` is to continue using the enum numbers from the previous enum at the same level.

Example

```
#show: el.default-enum-list.with(auto-resuming: auto)
#let emph-enum = el.item(body-format: (whole: (stroke: red, inset: 1pt)))
+ #lorem(5)
  + #emph-enum #lorem(5)
  + #lorem(5)
+ #lorem(5)
  #el.resume() // -> 3
  + #emph-enum #lorem(5) (resume)
  + #lorem(5)
+ #lorem(5)
  + #lorem(5)
```

1. Lorem ipsum dolor sit amet.
   1. Lorem ipsum dolor sit amet.
   2. Lorem ipsum dolor sit amet.
2. Lorem ipsum dolor sit amet.
   3. Lorem ipsum dolor sit amet. (resume)
   4. Lorem ipsum dolor sit amet.
3. Lorem ipsum dolor sit amet.
   1. Lorem ipsum dolor sit amet.

- You can also use `resume[...]` to explicitly continue using the enum numbers from the previous level (especially in ambiguous cases) and treat the `[...]` as a new `#enum`.

Example

```
#show: el.default-enum-list.with(auto-resuming: auto)
#let emph-enum = el.item(body-format: (whole: (stroke: red, inset: 1pt)))
+ Item one
+ Item two
  + Sub item one
Paragraph
#el.resume[
  + #emph-enum New list item (resume) // resume
    + Sub item // not resume
  + New list item
]
+ New list item one // not resume
```

1. Item one
2. Item two
   1. Sub item one

Paragraph

3. New list item (resume)
   1. Sub item
4. New list item

1. New list item one

↑ Introduced in 0.2.0

2. Use the method `#resume-label` to label the enum you want to resume, and then use `#resume-list` in the desired enum to continue using the labelled enum numbers.

If you use the following in your document:

```
#show: el.config.ref-resume
```

then you can use `@some-label` instead of `resume-list(<some-label>)`.

Example

```
#show: el.default-enum-list.with(auto-resuming: auto)
+ Item one
+ Item two
  + Sub item one
  + Sub item Two
  + Sub item Three (label) #el.resume-label(<resume:sub>) // The enum you want to resume
Paragraph
#el.resume-list(<resume:sub>)
+ New Item (resume) // resume
```

1. Item one
2. Item two
   1. Sub item one
   2. Sub item Two
   3. Sub item Three (label)

Paragraph

4. New Item (resume)

↑ Introduced in 0.2.0

3. Use the method `#auto-resume-enum` and set `#auto-resume-enum.auto-resuming` (`level-property`) to `true` to automatically resume the enum numbers for `#enum`s in `#auto-resume-enum.doc`.

Example

```
#show: el.default-enum-list.with(auto-resuming: auto)
+ Item one
+ Item two
  + Sub item one
Paragraph
#el.auto-resume-enum(auto-resuming: true)[
  + New list item one
    + Sub item
    + Sub item
  + New list item two
    + Sub item (resume) // resume
  Paragraph
  + New list item (resume) // resume
]
+ New list item // not resume
```

1. Item one
2. Item two
   1. Sub item one

Paragraph

1. New list item one
   1. Sub item
   2. Sub item
2. New list item two
   3. Sub item (resume)

Paragraph

3. New list item (resume)

1. New list item

- Using `array` to specify that, for which level, the `#enum` continues to use the previous enum's numbers.

Example

```
#show: el.default-enum-list.with(auto-resuming: auto)
+ Item one
+ Item two
  + Sub item one
Paragraph
#el.auto-resume-enum(auto-resuming: (false, true, false))[
  // only for level 2 to resume
  + New list item one
    + Sub item
    + Sub item
  + New list item two
    + Sub item (resume) // resume
```

1. Item one
2. Item two
   1. Sub item one

Paragraph

1. New list item one
   1. Sub item
   2. Sub item
2. New list item two
   3. Sub item (resume)

Paragraph

1. New list item

1. New list item

```
  Paragraph
  + New list item // not resume
]
+ New list item // not resume
```

↑ Introduced in 0.3.0

NEW Using `function` to specify which `#enum` continues to use the previous enum's numbers.

Example

```
#show: el.default-enum-list.with(auto-resuming: auto)
+ List one
#el.auto-resume-enum(auto-resuming: it => {
  if it.elem-tag == "resume" { true } else { false }
})[
  + Item one
  + Item two
    + Sub item one
    + Sub item two
  New paragraph
  + Item one // not resume
    + #el.item(elem-tag: "resume") Sub item // resume
]
+ List two // not resume
```

1. List one

1. Item one
2. Item two
   1. Sub item one
   2. Sub item two

New paragraph

1. Item one
   3. Sub item

1. List two

The method `#auto-resume-enum` cannot be nested.

Application priority: `#resume-list` > `#auto-resume-enum` > `#resume`.

II.9.2. Isolating enum's resuming

↑ Introduced in 0.2.0

The method `#isolated-resume-enum` allows the `#enum`s in `#isolated-resume-enum.doc` to be treated as new enums with independent numbering, without affecting other enums out of `doc`.

Example

```
#let emph-enum(color) = el.item(body-format: (whole: (stroke: color, inset: 1pt)))
#table(
  columns: (1fr, 1fr),
  table.header[Case 1][Case 2],
  [
    #show: el.default-enum-list.with(auto-resuming: auto)
    + enumA #lorem(2)
      + enumB #lorem(1)
    Paragraph
    + #emph-enum(blue) enumC #lorem(1)
    + #lorem(2)
```

Case 1	Case 2
1. enumA Lorem ipsum. 1. enumB Lorem. Paragraph 1. enumC Lorem. 2. Lorem ipsum. 1. enumD Lorem. 2. Lorem ipsum. Paragraph 3. **resume enumC** 3. **resume enumD**	1. enumA Lorem ipsum. 1. enumB Lorem. Paragraph 1. enumC Lorem. 2. Lorem ipsum. 1. enumD Lorem. 2. Lorem ipsum. Paragraph 2. **resume enumA** 2. **resume enumB**

```
    + enumD #lorem(1)
    + #lorem(2)
  Paragraph
  #el.resume() // -> 3
  + *resume enumC*
    #el.resume() // -> 3
    + *resume enumD*
],
[
  #show: el.default-enum-list.with(auto-resuming: auto)
  + enumA #lorem(2)
    + enumB #lorem(1)
  Paragraph
  #el.isolated-resume-enum()[
    // isolated enum
    + #emph-enum(red) enumC #lorem(1)
    + #lorem(2)
      + enumD #lorem(1)
      + #lorem(2)
  ]
  Paragraph
  #el.resume() // -> 2
  + *resume enumA*
    #el.resume() // -> 2
    + *resume enumB*
],
)
```

II.9.3. Resuming for all

↑ Introduced in 0.2.0

If `#default-enum-list.auto-resuming` (`level-property`) is set to `true`, then all enum numbers will continue from the previous ones.

Example

```
#show: el.default-enum-list.with(auto-resuming: true)
+ #lorem(2)
  + #lorem(2)
  + #lorem(2)
Paragraph
+ #lorem(2)
  + #lorem(2)
```

1. Lorem ipsum.
   1. Lorem ipsum.
   2. Lorem ipsum.

Paragraph

2. Lorem ipsum.
   3. Lorem ipsum.

↑ Introduced in 0.3.0

Also, using `function` (level-property), you can specify which `#enum` continues to use the previous enum's numbers.

Example

```
#show: el.default-enum-list.with(
  fill: it => {
    if it.elem-tag == "resume" { red } else
    { auto }
  },
  auto-resuming: it => {
  if it.elem-tag == "resume" { true } else
  { false }
})
+ #lorem(2)
  + #lorem(2)
  + #lorem(2)
Paragraph
+ #lorem(2)
  + #el.item(elem-tag: "resume") #lorem(2)
+ #lorem(2)
  + #el.item(elem-tag: "resume") #lorem(2)
  + #lorem(2)
```

1. Lorem ipsum.
   1. Lorem ipsum.
   2. Lorem ipsum.

Paragraph

1. Lorem ipsum.
   3. Lorem ipsum.
2. Lorem ipsum.
   4. Lorem ipsum.
   5. Lorem ipsum.

If `auto-resuming` is not `none` or `false`, the method `default-enum-list(auto-resuming: ...)` cannot be used nested.

II.10. Body paragraph setting

↑ Introduced in 0.3.0

`#default-enum-list.first-line-inset` (`level-index-property`) is to add an inset to the first line of the item body. If there is no first line, it is ignored.

Example

```
#show: el.default-enum-list.with(first-line-
inset: 1em)
+ #lorem(15)

  Paragraph two: #lorem(15)
+ $ a^2 + b^2 = c^2 $ // no first line
  #lorem(15)
```

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

   Paragraph two: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.
2. $$a^2 + b^2 = c^2$$

   Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

↑ Introduced in 0.2.0

`#default-enum-list.line-indent` (`level-index-property`) is to set line indentation for paragraphs in item body.

Example

```
#show: el.default-enum-list.with(line-indent:
1em)
+ #lorem(15)

  Paragraph two: #lorem(15)
+ $ a^2 + b^2 = c^2 $
  #lorem(15)
```

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

   Paragraph two: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.
2. $$a^2 + b^2 = c^2$$

   Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

↑ Introduced in 0.2.0

`#default-enum-list.hanging-indent` (`level-index-property`) is to set hanging indentation for paragraphs in item body.

Example

```
#show: el.default-enum-list.with(hanging-indent: 1em)
+ #lorem(15)

  Paragraph two: #lorem(15)
+ $ a^2 + b^2 = c^2 $
  #lorem(15)
```

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

   Paragraph two: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.
2. $$a^2 + b^2 = c^2$$

   Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

⇄ Changed in 0.3.0

ⓘ Paragraphs inside `block-level` elements that appear in the item body are not processed.

Example

```
#show: el.default-enum-list.with(
  line-indent: 1em,
  hanging-indent: 2em
)
+ #block(stroke: 1pt)[
    #lorem(10)

    #lorem(10)
  ]
  #lorem(10)
+ #lorem(10)
  #table([#lorem(10)], [#lorem(10) $ a $ #lorem(10)])
```

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

   Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

   Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
2. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do. a Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

- The `inline` elements in the item body are processed as `par` (but the `inline` elements inside `block-level` elements are not processed), which is different from native `#enum` and `#list`.

Example

```
#set par(first-line-indent: (amount: 1em, all: true), hanging-indent: 1em)
#table(
  columns: (1fr, 1fr),
  table.header([*itemize*], [*native*]),
  [
    #show: el.default-enum-list.with()
    + #lorem(10)

      #lorem(10)
    + #lorem(10)
  ],
  [
    #enum(
      enum.item[
        #lorem(10)
```

itemize	**native**
1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do. 2. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.	1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do. 2. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

```
17          #lorem(10)
18        ],
19        enum.item[#lorem(10)] // not apply, since it
          is not `par`
20      )#el.itemize-prevent-label
21
22    ]
23  )
```

- The `line-indent` and `hanging-indent` parameters are independent of `par.first-line-indent`, `par.hanging-indent`.

Example

```
1  #set par(first-line-indent: (amount: 1em, all: true),
   hanging-indent: .5em)
2  #show: el.default-enum-list.with(
3    line-indent: 1em,
4    hanging-indent: -2em,
5  )
6  + #lorem(10)
7
8    #lorem(10)
9  + #lorem(10)
```

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

   Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
2. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

`#default-enum-list.hanging-type` (`level-property`) is to set the hanging type of the paragraphs inside the item body.

- `"classic"`: The native behavior of `#enum` or `#list`.
- `"paragraph"`: For each level, paragraphs inside the item body are not indented; they align to the current `body-margin`'s beginning margin (the same as the `paragraph-*` methods).
  - **Note**: But the current `body-margin`'s beginning margin is the same as the `"classic"` one.

Example

```
1  #show: el.default-enum-list.with(hanging-type:
   ("classic", "paragraph"))
2  + #lorem(10)
3    + #lorem(10)
4
5      #lorem(10)
6    + #lorem(10)
7  + #lorem(10)
```

1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
   1. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

   Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
   2. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
2. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

II.11. Checklist

↑ Introduced in 0.2.0

To enable this feature, set `#default-enum-list.checklist` (`level-property`) to `true`.

Example

```
#show: el.default-enum-list.with(checklist: true)
- [x] checked #lorem(2)
- [ ] unchecked #lorem(2)
- [/] incomplete #lorem(2)
- [-] canceled #lorem(2)
```

- ☑ checked Lorem ipsum.
- ☐ unchecked Lorem ipsum.
- ◧ incomplete Lorem ipsum.
- ⊟ canceled Lorem ipsum.

⇄ Changed in 0.3.0

Note: after `itemize0.3.0`, this feature can no longer be enabled through `#config.checklist`, and we have removed the `enable` parameter.

II.11.1. Configuration of checklist

↑ Introduced in 0.2.0

Using `#config.checklist`, you can configure the checklist.

- `baseline` (`auto`, `"center"`, `"top"`, `"bottom"`, `"baseline"`): The baseline of the checkbox. Default is `auto`, controlled by `#default-enum-list.label-baseline`.
- `extras` (`bool`): Whether to enable additional commands, i.e., use the following:

```
">": "➡",
"<": "🗓",
"?": "❓",
"!": "❗",
"*": "⭐",
"\"": "“",
"l": "📍",
"b": "⚡",
"i": "i",
"S": "💰",
"I": "💡",
"p": "👍",
"c": "👎",
"f": "🔥",
"k": "🔑",
"w": "🏆",
"u": "🔼",
"d": "🔽",
```

Example

```
#show: el.config.checklist.with(extras: true)
#show: el.default-enum-list.with(checklist: true)
- [>] checked #lorem(2)
- [?] unchecked #lorem(2)
- [f] incomplete #lorem(2)
- [d] canceled #lorem(2)
```

- ➡ checked Lorem ipsum.
- ❓ unchecked Lorem ipsum.
- 🔥 incomplete Lorem ipsum.
- 🔽 canceled Lorem ipsum.

- `fill` (`auto`, `color`): Border color. Default is `auto`, use the current label's text fill.
- `radius` (`length`): Border radius. Default is `0.1em`.

- `solid` (`color`, `none`): Solid color. Default is `none`.

Example

```
#show: el.config.checklist.with(
  fill: (blue, auto),
  radius: (.5em, 0em),
  solid: gray.lighten(50%),
)
#show: el.default-enum-list.with(checklist: true)
- [x] checked #lorem(2)
- [ ] unchecked #lorem(2)
  - [x] checked (nested) #lorem(2)
- [/] incomplete #lorem(2)
- [-] canceled #lorem(2)
```

- checked Lorem ipsum.
- unchecked Lorem ipsum.
  - checked (nested) Lorem ipsum.
- incomplete Lorem ipsum.
- canceled Lorem ipsum.

- `enable-character` (`bool`): When set to `true`, if the character in `[...]` is not among `x`, ` `, `-`, `/` or the extras characters (if `extras` is `true`), the character in `[...]` will be displayed.

Example

```
#show: el.default-enum-list.with(checklist: true)
#show: el.config.checklist.with(enable-character: true)
- [B] #lorem(5)
- [A] #lorem(5)

#show: el.config.checklist.with(enable-character: false)
- [B] #lorem(5)
- [A] #lorem(5)
```

- [B] Lorem ipsum dolor sit amet.
- [A] Lorem ipsum dolor sit amet.

- [B] Lorem ipsum dolor sit amet.
- [A] Lorem ipsum dolor sit amet.

- `symbol-map` (`dictionary`, `function`): Custom commands.
  - Can replace built-in commands or add new ones.
    - The form is: `("some-character": content)`.
  - If a function is specified, its form is: `it => dictionary`, where `it` provides three properties: `fill`, `radius`, `solid`.

Example

```
#show: el.default-enum-list.with(fill: blue, checklist: true)
#show: el.config.checklist.with(
  symbol-map: (" ": emoji.checkmark, "A": emoji.a),
)
- [ ] #lorem(5)
- [A] #lorem(5)

#let box-sym(fill: white, stroke: rgb("#616161"), radius: .1em, body) = box(
  stroke: .05em + stroke,
  fill: fill,
```

- ✓ Lorem ipsum dolor sit amet.
- 🅰 Lorem ipsum dolor sit amet.

- ☐ Lorem ipsum dolor sit amet.
- ☑ Lorem ipsum dolor sit amet.
- 🅰 Lorem ipsum dolor sit amet.

```
  height: 1em,
  width: 1em,
  radius: radius,
)[#body]
#show: el.config.checklist.with(
  fill: blue,
  solid: luma(95%),
  symbol-map: it => (
    " ": box-sym(fill: it.solid, stroke: it.fill, radius: .2em, []),
    "x": box-sym(fill: it.fill, stroke: it.fill, radius: .2em, [#text(fill: white) [#emoji.checkmark]]),
    "A": box-sym(fill: it.fill, stroke: it.fill, radius: .2em, [#text(fill: white) [A]]),
  ),
)
- [ ] #lorem(5)
- [x] #lorem(5)
- [A] #lorem(5)
```

Also, you can use other package that provides symbol mapping.

Example

```
#import "@preview/roumnd:0.1.0"
#let symbol-map = roumnd.icons.map(icon => context icon(foreground: luma(95%), background: text.fill))
#show: el.config.checklist.with(symbol-map: symbol-map)
#show: el.default-enum-list.with(checklist: true, fill: (blue, auto))
- [ ] #lorem(2)
- [x] #lorem(2)
- [/] #lorem(2)
- [>] #lorem(2)
- [!] #lorem(2)
```

- Lorem ipsum.
- Lorem ipsum.
- Lorem ipsum.
- Lorem ipsum.
- Lorem ipsum.

- `enable-format` (`bool`): Whether to enable formatting for the item body, with the formatting determined by `format-map`. The default formatting is for the character `"-"`, with the formatting:

```
it => strike(text(fill: rgb("#888888"), it))
```

Example

```
#show: el.default-enum-list.with(checklist: true)
#show: el.config.checklist.with(enable-format: true)
- [ ] #lorem(3)
- [-] #lorem(3)
```

- Lorem ipsum dolor.
- ~~Lorem ipsum dolor.~~

- `format-map` (`dictionary`): Custom formatting for the item body.
  - Can replace built-in commands or add new ones.
  - The form is: `("some-character": func)`.
    - `func` takes the form `it => content`. It applies the body of the current `some-character` item to this method.

Example

```
#show: el.default-enum-list.with(checklist: true)
#show: el.config.checklist.with(
  enable-format: true,
  format-map: (
    ">": it => highlight(it),
    "-": it => strike(text(fill: rgb("#888888"), emph(it))), // Overrides the format-function of "-"
  ),
)
- [ ] #lorem(3)
- [-] #lorem(3)
- [>] #lorem(3)
```

- [ ] Lorem ipsum dolor.
- [-] ~~*Lorem ipsum dolor.*~~
- [>] Lorem ipsum dolor.

II.12. Description list

↑ Introduced in 0.3.0

The description list is like the following:

```
[marker] / [number] [term-body] [description-body]
```

where

```
term-body: [prefix] [term] [separator.prefix]
description-body: [separator.suffix] [description] [suffix]
```

Using the syntax:

- `+ / term: description`
- `- / term: description`

Example

```
#let size = 1.4em
#let term-format = body => text(size: size, strong(body))
#let timeline = el.default-list.with(
  label-format: (circle(width: size, fill: blue), auto),
  label-baseline: ((amount: "center", relative-to: term-format([A])), auto),
  tight-item-mode: ("never-tight", auto),
  body-format: (
    whole: (
      stroke: ((left: gray + 2pt), auto),
```

2024
Product Project Approval

2025
Prototype development

2026
Officially released

```
        outset: ((left: -size / 2, bottom: .5em),
        auto),
      ),
    ),
    description-config: (
      style: "stacked",
      term-format: (term-format,),
      separator: (context block(above: 0pt,
      below: par.leading), auto),
    ),
)
#timeline[
  - / 2024: Product Project Approval
  - / 2025: Prototype development
  - / 2026: Officially released
]
```

Example

```
#let qa(body) = {
  let ask-badge = box(
    width: 1.8em,
    height: 1.8em,
    radius: 2pt,
    fill: rgb("#8a4b4b"),
    stroke: .5pt + rgb("#8a4b4b"),
    inset: 5pt,
  )[
    #align(center + horizon)[
      #text(
        fill: rgb(20, 15, 15),
        weight: "bold",
        size: 18pt,
      )[问]
    ]
  ]
  show: el.default-list.with(
    body-format: (
      whole: (
        fill: (rgb(20, 15, 15).lighten(6%), auto),
        inset: (10pt, auto),
      ),
      outer: (
        stroke: ((bottom: 0.5pt + rgb("#5a2626").lighten(5%)), auto),
        inset: ((bottom: 10pt), auto),
      ),
      style: (fill: rgb(220, 210, 210)),
    ),
    label-format: (ask-badge, auto),
    label-baseline: ("center", auto),
    description-config: (
      style: "stacked",
      term-format: (text.with(fill: rgb(180, 100, 100)), auto),
    ),
  )
```

```
  body
}

#qa[
  - / 如何第一时间了解《黑神话：钟馗》的最新消息？：每年的8月20日，游戏科学都会例行给大家汇报最新的工作进度。所以最简单的办法，就是在820当天瞅瞅B站、抖音、快手、微博、小红书、微信视频号等社媒的【黑神话】官号动态，然后顺手点个关注。
  - / 作为黑神话系列的第二作，《黑神话：钟馗》与《黑神话：悟空》会有哪些异同？：首先，从名字就能看出，《黑神话：钟馗》依然取材于中国的古代神话与民间传说。其次，它依然是一款标准的单机·动作·角色扮演游戏，商业模式与前作别无二致。再次，从最新的实机演示中大家或许能感受到：一方面，我们希望保留黑神话前作广受认可的诸多乐趣；另一方面，也乐于探索契合新主角的特色玩法、前沿的技术特性与差异化的叙事方式。特别提醒：提前公布的宣传视频难免与最终发售内容存在一定差别，有些探索目前仍是实验性的尝试，如果不是因为每年都得硬着头皮出来献艺……
  - / 能再多透露点《黑神话：钟馗》的剧情和玩法吗？：透不了，真透不了。很多玩家对视频解读的角度之刁、考据之细、脑洞之大，已远远超出开发团队自己对项目的理解，学习都来不及。
  - / 《黑神话：钟馗》将登陆哪些平台？：PC+主流主机平台，正式信息请以发售前通知为准。
  - / 《黑神话：钟馗》大概什么时候发售？：尽量快！但考虑到综合品质必须先打动内部见多识广的同事们，那就一定快不起来，嘻嘻。
]
```

问 如何第一时间了解《黑神话：钟馗》的最新消息？

每年的 8 月 20 日，游戏科学都会例行给大家汇报最新的工作进度。所以最简单的办法，就是在 820 当天瞅瞅 B 站、抖音、快手、微博、小红书、微信视频号等社媒的【黑神话】官号动态，然后顺手点个关注。

问 作为黑神话系列的第二作，《黑神话：钟馗》与《黑神话：悟空》会有哪些异同？

首先，从名字就能看出，《黑神话：钟馗》依然取材于中国的古代神话与民间传说。其次，它依然是一款标准的单机·动作·角色扮演游戏，商业模式与前作别无二致。再次，从最新的实机演示中大家或许能感受到：一方面，我们希望保留黑神话前作广受认可的诸多乐趣；另一方面，也乐于探索契合新主角的特色玩法、前沿的技术特性与差异化的叙事方式。特别提醒：提前公布的宣传视频难免与最终发售内容存在一定差别，有些探索目前仍是实验性的尝试，如果不是因为每年都得硬着头皮出来献艺……

问 能再多透露点《黑神话：钟馗》的剧情和玩法吗？

透不了，真透不了。很多玩家对视频解读的角度之刁、考据之细、脑洞之大，已远远超出开发团队自己对项目的理解，学习都来不及。

问 《黑神话：钟馗》将登陆哪些平台？

PC+主流主机平台，正式信息请以发售前通知为准。

问 《黑神话：钟馗》大概什么时候发售？

尽量快！但考虑到综合品质必须先打动内部见多识广的同事们，那就一定快不起来，嘻嘻。

II.12.1. Configuration of description list

Configure the description list through `#default-enum-list.description-config`. Its complete form is: (

- **(style): "plain"** `auto` | `"plain"` | `"hanging"` | `"stacked"` | `"horizontal"`
- **(checklist): false** `bool`
- **(term-width): auto** `auto` | `relative` | `"max"` | `dictionary`
- **(term-align): auto** `auto` | `alignment`
- **(prefix): []** `content`
- **(suffix): []** `content`
- **(separator): auto** `auto` | `content` | `dictionary`
- **(hanging-indent): auto** `auto` | `length`

```
  (term-format): none   none | function
  (description-format): none   none | function
  (format): none   none | function
  (enable-first-par): auto   auto | bool
)
```

> All the keys are `level-index-property`.

- `style`: The style of the description list.
  - `"plain" == auto`: Plain description list. The `term-body` and `description-body` are displayed in a single paragraph.
  - `"hanging"`: Hanging description list. The `term-body` and `description-body` are displayed in a single paragraph, but `hanging-indent` is `2em` in default. The same as native Typst `terms`.
  - `"stacked"`: Stacked description list. The `term-body` and `description-body` are displayed as two start-aligned paragraphs, respectively.
  - `"horizontal"`: Horizontal description list. The `term-body` and `description-body` are displayed in a single paragraph, but `description-body` will be indented and aligned according to `term-width` (by default, using the maximum width of the `term-body`).

Example

```
#let test = [
  #set par(justify: true)
  - / #box([Windows]): #lorem(10)

      #lorem(10)
  - / MacOS: #lorem(6)
  - / Linux: #lorem(6)
  + / Markdown: #lorem(10)
  + / Latex: #lorem(6)
  + / Typst: #lorem(6)
]

#let desc-list(body, ..args) = el.default-enum-list(
  description-config: (
    term-format: (strong,),
    ..args.named(),
  ),
  body,
)

#table(
  [
    *Plain style description list: *
    #desc-list[#test]
  ],
  [
    *Horizontal style description list: *
    #desc-list(style: "horizontal")[#test]
  ],
  [
    *Hanging style description list: *
    #desc-list(style: "hanging")[#test]
  ],
  [
    *Stacked style description list: *
    #desc-list(style: "stacked", separator: (prefix: context block(above: 0pt, below: par.leading)))[#test]
  ],
)
```

Plain style description list:

- **Windows** Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
- **MacOS** Lorem ipsum dolor sit amet, consectetur.
- **Linux** Lorem ipsum dolor sit amet, consectetur.

1. **Markdown** Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
2. **Latex** Lorem ipsum dolor sit amet, consectetur.
3. **Typst** Lorem ipsum dolor sit amet, consectetur.

Horizontal style description list:

- **Windows** Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
- **MacOS** Lorem ipsum dolor sit amet, consectetur.
- **Linux** Lorem ipsum dolor sit amet, consectetur.

1. **Markdown** Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
2. **Latex** Lorem ipsum dolor sit amet, consectetur.
3. **Typst** Lorem ipsum dolor sit amet, consectetur.

Hanging style description list:

- **Windows** Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
- **MacOS** Lorem ipsum dolor sit amet, consectetur.
- **Linux** Lorem ipsum dolor sit amet, consectetur.

1. **Markdown** Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
2. **Latex** Lorem ipsum dolor sit amet, consectetur.
3. **Typst** Lorem ipsum dolor sit amet, consectetur.

Stacked style description list:

- **Windows**
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
- **MacOS**
  Lorem ipsum dolor sit amet, consectetur.
- **Linux**
  Lorem ipsum dolor sit amet, consectetur.

1. **Markdown**
   Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
2. **Latex**
   Lorem ipsum dolor sit amet, consectetur.
3. **Typst**
   Lorem ipsum dolor sit amet, consectetur.

Using `description-config` or `elem-tag` in `#item.args`, you can change the style of the description list individually.

Example

```
#show: el.default-enum-list.with(
  description-config: (
    style: it => {
      if it.elem-tag == "stacked-important"
      { "stacked" } else {"horizontal"}
    },
    term-format: (strong,),
    format: it => {
      if it.elem-tag == "stacked-important" {
        show: highlight
        it.body
      } else {
        it.body
      }
    }
  ),
)
- / #lorem(2): #lorem(10)
- / #lorem(1): #lorem(10)
Paragraph
- #el.item(elem-tag: "stacked-important")
  / #lorem(2): #lorem(10)
- / #lorem(3): #lorem(10)
```

- **Lorem ipsum.** Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
- **Lorem.** Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Paragraph

- **Lorem ipsum.**

  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
- **Lorem ipsum dolor.**

  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Example

```
#let stacked = el.item(
  description-config: (
    style: "horizontal",
    term-format: (strong,),
  ),
  for-elem: true
)
#show: el.default-enum-list
- #stacked
  / #lorem(2): #lorem(10)
- / #lorem(1): #lorem(10)
```

- **Lorem ipsum.** Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
- **Lorem.** Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

- `checklist` (`bool`): Whether to use checklist style description list.
  - **Note**: In order to enable checklist style, the `checklist` in `default-*` (or `paragraph-*`) must be `true`.

Example

```
#show: el.default-enum-list.with(
  checklist: true, // need
  description-config: (
    checklist: true, // need
  ),
)
+ / [ ]: #lorem(12)
+ / [-]: #lorem(8)
- / [x]: #lorem(8)
- / [/]: #lorem(12)
```

1. ☐ Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor.
2. ⊟ Lorem ipsum dolor sit amet, consectetur adipiscing elit.

- ☑ Lorem ipsum dolor sit amet, consectetur adipiscing elit.
- ◧ Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor.

- `term-width`: The width of the `term-body`.
  - `auto`: In default,
    - for "horizontal" style, the `term-width` uses the maximum width of the `term-body` in the current level and in the "horizontal" style. That is, use `(amount: "max", style: "native")`.
    - for other styles, the `term-width` uses the real width of the `term-body`. That is, use `(amount: auto, style: "native")`.
  - `relative`, `"max"`, `dictionary`: See `#default-enum-list.label-width`.
    - **Note**: The maximum width of the `term-body` in the current level is calculated for different styles.
- `term-align` (`auto`, `alignment`): The alignment of the `term-body`.
  - **Note**: The `term-align` works for the following cases.
    - The "horizontal" and "stacked" styles;
    - For "hanging" and "plain" styles, the `term-width` is not `auto` (that is, the real width of the `term-body` is not equal to the rendered width).
  - `auto` == `start`.
  - `alignment`: The (horizontal) alignment of the `term-body`.
- `separator`: The separator between `term-body` and `description`. Its complete form is: `(`

  `(prefix): []` `content`

  `(suffix): []` `content`

  `)`
  - In default, `separator.prefix` is added after `term` and `separator.suffix` is added before `description`.
  - Shorthand rules:
    - `auto`: The `separator.prefix` is `[]` for "stacked" style and `[#h(.6em)#h(0pt, weak: true)]` for other styles. `separator.suffix` is `[]`.
    - `content`: The `separator.prefix` is `separator` and the `separator.suffix` is `[]`.

Example

```
#show: el.default-enum-list.with(
  description-config: (
    separator: it => {
      if it.style == "plain" {[*)*#h(.6em, weak: true)]} else {auto}
    },
    term-format: (strong,),
  ),
)
- / Alpha: #lorem(10)
- / Beta: #lorem(10)
```

- **Alpha)** Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
- **Beta)** Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Example

```
#show: el.default-enum-list.with(
  tight-item-mode: ("never-tight", auto),
  description-config: (
    separator: it => {
      if it.style == "hanging" {(suffix:
      [#parbreak()])} else {auto}
    },
    term-format: (strong,),
    style: "hanging",
  ),
)
- / Alpha: #lorem(10)

  #lorem(10)
- / Beta: #lorem(10)
```

- **Alpha**

  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

- **Beta**

  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

- `prefix`, `suffix` (`content`): The `prefix` is added before `term`, and the `suffix` is added after `description`.

Example

```
#show: el.default-enum-list.with(
  tight-item-mode: ("never-tight", auto),
  description-config: (
    prefix: it => {
      if it.tag == "prefix-important" {
        box(height: 1em, width: .2em, fill:
        red, baseline: .2em)
        h(.3em, weak: true)
      }
    },
    suffix: it => {
      if it.tag == "prefix-important" {
        h(.3em, weak: true)
        box(height: 1em, width: .2em, fill:
        red, baseline: .2em)
      }
    },
    term-format: (strong,),
    // style: "hanging",
  ),
)
- #el.item(tag: "prefix-important")
  / Alpha: #lorem(10)
- / Beta: #lorem(10)
```

- **Alpha** Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
- **Beta** Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

- `hanging-indent` (`auto`, `length`): The hanging indent of the description list.
  - If `auto`, `hanging-indent` is `2em` for “hanging” style, and `0pt` for other styles.
- `term-format`, `description-format` (`none`, `function`): The format of the `term` and `description`.
  - `none`: Do not apply.
  - `function` (base-property): Apply the function to the `term` (not the `term-body`) and `description` (not the `description-body`).
    - The function form: `body => content`.
    - The `function` should be wrapped in an `array`, like: `(strong, )`.

Example

```
#let desc-style = style => (description-config: (style: style), for-elem: true)
#show: el.default-enum-list.with(description-config: (
  term-format: it => {
    let format(body) = if it.style == "stacked" {
      strong(body)
      block(above: 0pt, below: par.leading)
    } else { emph(body) }
    format(it.body)
  },
  description-format: (text.with(fill: blue), none)
))
- #el.item(..desc-style("stacked"))
  / CPU: #lorem(10)
- / GPU: #lorem(10)
Paragraph
- #el.item(..desc-style("horizontal"))
  / CPU: #lorem(10)
- / GPU: #lorem(10)
```

- **CPU**
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
- **GPU**
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Paragraph

- *CPU* Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
- *GPU* Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

- `format` (`none`, `function`): The format of the description list. If the default style is not suitable, you can use `format` to customize the style.
  - `none`: Do not apply.
  - `function` (base-property): Apply the function to the whole description list.
    - The function form: `body => content`.
    - The `function` should be wrapped in an `array`, like: `(strong, )`.

Example

```
#show: el.default-enum-list.with(description-config: (
  term-format: (emph,),
  format: (text.with(fill: red), strong, none),
))
- / CPU: #lorem(10)
  + / ABC: #lorem(10)
  + / DEFF: #lorem(10)
- / GPU: #lorem(10)
```

- *CPU* Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.
  1. ***ABC*** **Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.**
  2. ***DEFF*** **Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.**
- *GPU* Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

  - `function` (level-property): Use the form `it => content` and access values on `it`, you can define any style of description list. See the implementation examples of the four styles provided by `itemize`.
- `enable-first-par` (`auto`, `bool`): To make all the paragraphs in the description list like the first paragraph.
  - `auto`: For `stacked` style, the `enable-first-par` is `true`, and for other styles, the `enable-first-par` is `false`.
  - `bool`: Whether to enable the first paragraph.

Example

```
#let test = [
  + / CPU: #lorem(6)

      *Description list body*
      #lorem(6)

    *enum body*
    #lorem(6)
  + / GPU: #lorem(6)
]
#let my-desc(enable-first-par) = el.default-enum-list.with(
  line-indent: 1em,
  body-format: (style: (fill: blue) ),
  description-config: (
    term-format: (strong,),
    format: (text.with(fill: red), none),
    enable-first-par: enable-first-par,
  )
)
#table(
  columns: (1fr, 1fr),
  table.header([`enable-first-par: true`], [`enable-first-par: false`]),
  [
    #show: my-desc(true)
    #test
  ],
  [
    #show: my-desc(false)
    #test
  ]
)
```

enable-first-par: true	enable-first-par: false
1. **CPU** Lorem ipsum dolor sit amet, consectetur. **Description list body** Lorem ipsum dolor sit amet, consectetur. **enum body** Lorem ipsum dolor sit amet, consectetur. 2. **GPU** Lorem ipsum dolor sit amet, consectetur.	1. **CPU** Lorem ipsum dolor sit amet, consectetur. **Description list body** Lorem ipsum dolor sit amet, consectetur. **enum body** Lorem ipsum dolor sit amet, consectetur. 2. **GPU** Lorem ipsum dolor sit amet, consectetur.

II.13. Set-Rule

In the current version of `Typst` (0.15.1), we cannot use custom functions with `set rule`, which prevents us from retaining previously set property values when using `default-item-list`, `paragraph-item-list`, etc. Now, with the `elembic` package, we can achieve this functionality. To this end, we have repackaged `default-item-list` and `paragraph-item-list` into `set-default` and `set-paragraph` methods.

Usage: Include the following at the beginning of your document:

```
#show: el.set-default() // need `()`!!!
```

> The key difference is that `set-default` requires **parentheses**!

Example

```
#let item = [
  + #lorem(5)
    + #lorem(5)
    + + #lorem(5)
    + - #lorem(5)
  + #lorem(5)
]

#show: el.set-default()
#set enum(numbering: "(1).(a).(i)")
#item

// change the label color
#show: el.set-default(fill: (red, blue, green, auto))
#item

// change the label size
#show: el.set-default(size: (20pt, 16pt, 14pt, auto))
#item

// change the body-format and indent
#show: el.set-default(indent: (auto, 1em), body-format: (strong,))
#item

// change the border of the item body
#show: el.set-default(body-format: (stroke: (red, auto), inset: (5pt, auto)))
#item

// use the default style
#show: el.set-default()
#item
```

(1) Lorem ipsum dolor sit amet.
  (a) Lorem ipsum dolor sit amet.
  (b) (i) Lorem ipsum dolor sit amet.
  (c) • Lorem ipsum dolor sit amet.
(2) Lorem ipsum dolor sit amet.

(1) Lorem ipsum dolor sit amet.
  (a) Lorem ipsum dolor sit amet.
  (b) (i) Lorem ipsum dolor sit amet.
  (c) • Lorem ipsum dolor sit amet.
(2) Lorem ipsum dolor sit amet.

(1) Lorem ipsum dolor sit amet.
  (a) Lorem ipsum dolor sit amet.
  (b) (i) Lorem ipsum dolor sit amet.
  (c) • Lorem ipsum dolor sit amet.
(2) Lorem ipsum dolor sit amet.

(1) Lorem ipsum dolor sit amet.
  (a) Lorem ipsum dolor sit amet.
  (b) (i) Lorem ipsum dolor sit amet.
  (c) • Lorem ipsum dolor sit amet.
(2) Lorem ipsum dolor sit amet.

(1) Lorem ipsum dolor sit amet.
  (a) Lorem ipsum dolor sit amet.
  (b) (i) Lorem ipsum dolor sit amet.
  (c) • Lorem ipsum dolor sit amet.
(2) Lorem ipsum dolor sit amet.

(1) Lorem ipsum dolor sit amet.
  (a) Lorem ipsum dolor sit amet.
  (b) (i) Lorem ipsum dolor sit amet.
  (c) • Lorem ipsum dolor sit amet.
(2) Lorem ipsum dolor sit amet.

Part III.

API Reference

III.1. Main methods

#default-enum
#default-enum-list
#default-list
#inline-enum
#inline-enum-list
#inline-list
#paragraph-enum
#paragraph-enum-list
#paragraph-list
#set-default
#set-paragraph


```
#default-enum(
  (doc),
  (indent): auto,
  (body-indent): auto,
  (label-indent): auto,
  (label-inset): auto,
  (body-margin): auto,
  (hanging-type): "classic",
  (line-indent): auto,
  (first-line-inset): auto,
  (hanging-indent): auto,
  (whole-spacing): auto,
  (item-spacing): auto,
  (tight-mode): auto,
  (tight-item-mode): auto,
  (label-width): auto,
  (label-align): auto,
  (label-baseline): auto,
  (label-format): none,
  (body-format): none,
  (auto-resuming): none,
  (auto-label-width): none,
  (auto-base-level): true,
  (step): auto,
  (ref-numbering): none,
  (supplement): auto,
  (checklist): false,
  (description-config): auto,
  ..(args)
) → content
```

Configures default styling for `enum`.

See `#default-enum-list`.

```
#default-enum-list(
  (indent): auto,
  (body-indent): auto,
  (label-indent): auto,
  (label-inset): auto,
  (body-margin): auto,
  (hanging-type): "classic",
  (line-indent): auto,
  (first-line-inset): auto,
  (hanging-indent): auto,
  (whole-spacing): auto,
  (item-spacing): auto,
  (tight-mode): auto,
  (tight-item-mode): auto,
  (label-width): auto,
  (label-align): auto,
  (label-baseline): auto,
  (label-format): none,
  (body-format): none,
  (auto-resuming): none,
  (auto-label-width): none,
  (auto-base-level): true,
  (step): auto,
  (ref-numbering): none,
  (supplement): auto,
  (checklist): false,
  (enum-config): (:),
  (list-config): (:),
  (description-config): auto,
  ..(args)
)[doc] → content
```

Configures default styling for `enum` and `list`.

Argument

`(doc)` `content`

`enum` and `list` in `doc` to be process.

Argument

`(indent): auto` `auto` | `length` | `array` | `function`

The indentation of the `enum` or `list`.

- Usage:

```
#show: el.default-enum-list.with(indent: 1em) // all items are indented by 1em
#show: el.default-enum-list.with(indent: (1em, 2em, auto)) // the first level is indented by 1em, the second level is indented by 2em, the third level and beyond are indented by the `enum.indent` or `list.indent`
#show: el.default-enum-list.with(indent: (1em, 2em, el.LOOP)) // the first level is indented by 1em, the second level is indented by 2em, the third level is indented by 1em, the fourth level is indented by 2em, and so on
```

```
  #show: el.default-enum-list.with(indent: ((2em, 1em, auto), 1em, auto)) // in
  the first level, the first item is indented by 2em, the second item is indented
4 by 1em, the third item and beyond are indented by the `enum.indent` or
  `list.indent`; the second level is indented by 1em, the third level and beyond
  are indented by the `enum.indent` or `list.indent`
5 #show: el.default-enum-list.with(indent: it => it.level * 1em)
```

- `auto`: The indentation is determined by the `enum.indent` or `list.indent`.
- `length`: The value of `indent`.
- `array` (level-property): The elements are `length`, `auto` or `array`, the item at `level`-th level will be indented by the corresponding value of the array at position `level - 1`.
  - If the last element of the array is `LOOP`, the values in the array will be used cyclically;
  - otherwise, the last value of the array will be used for residual levels.
  - The elements in the array can also be an array, where the element at position `n - 1` applies to the item of index `n`.
- `function` (level-property): The return value will be used for each level and each item.
  - The function form: `it => length | auto | array`
  - If the returned value is a `array`, then the elements in the array will be used for each item.
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n`: The index of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.tag` (since ver0.3.0): The tag of the current item.
    - `it.elem-tag` (since ver0.3.0): The tag of the current enum or list.
    - `it.e` (since ver0.3.0): The field of the current construct element (`enum` or `list`), but excluding the `children`.
    - `it.hanging-type` (since ver0.3.0): The hanging type of the current level.
    - `it.marker` (since ver0.3.0): The marker of the current label if it is a `list`.
    - `it.number` (since ver0.3.0): The number of the current label if it is a `enum`.
    - `it.label-width` (dictionary) (since ver0.3.0): The width info of the current label.
      - `it.label-width.max`: The maximum width of the labels in the current level.
      - `it.label-width.current`: The current width of the label.
    - `it.parent` (since ver0.3.0): The parameter info of the parent containing the current list — the parent's construction element `e`, the item `n` where the parent is located, and the horizontal spacing parameters: `label-indent`, `label-inset`, `body-indent`, `indent`, `body-margin`, `label-width` (`max`, `current`)
    - `it.get` (since ver0.3.0): Access the parameter info (see `it.parent`) of the current list's ancestors by level

Argument

`(body-indent): auto` — `auto` | `length` | `array` | `function`

The indentation of the item body, i.e., the space between the label and the body of each item.

- Usage: See `#default-enum-list.indent`.
- `auto`: The indentation is determined by the `enum.body-indent` or `list.body-indent`.
- `length`: The value of `body-indent`.

- `array` (level-property): The elements are `length`, `auto` or `array`, each level of the item will be indented by the corresponding value of the array at position `level - 1`.
  - See `#default-enum-list.indent`.
- `function` (level-property): The return value will be used for each level and each item.
  - The function form: `it => length | auto | array`
  - If the returned value is a `array`, then the elements in the array will be used for each item.
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n`: The index of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.tag`: The tag of the current item.
    - `it.elem-tag`: The tag of the current enum or list.
    - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
    - `it.hanging-type`: The hanging type of the current level.
    - `it.marker`: The marker of the current label if it is a `list`.
    - `it.number`: The number of the current label if it is a `enum`.
    - `it.label-width` (dictionary): The width info of the current label.
      - `it.label-width.max`: The maximum width of the labels in the current level.
      - `it.label-width.current`: The current width of the label.
    - `it.parent`: The parameter info of the parent containing the current list — the parent's construction element `e`, the item `n` where the parent is located, and the horizontal spacing parameters: `label-indent`, `label-inset`, `body-indent`, `indent`, `body-margin`, `label-width` (`max`, `current`)
    - `it.get`: Access the parameter info (see `it.parent`) of the current list's ancestors by level

Argument

`⟨label-indent⟩: auto` — `auto` | `length` | `array` | `function`

The indentation for the label (the enum's number or the list's marker).

- Usage: See `#default-enum-list.indent`.
- ⚠ **Breaking change**: The `label-indent` now does not impact the width of the label nor the first-line indentation of the body.
- `auto`: The label is not indented (equivalent to `0pt`).
- `length`: The label is indented by the given value.
- `array` (level-property): The elements are `length`, `auto` or `array`, each level of the item will be indented by the corresponding value of the array at position `level - 1`.
  - See `#default-enum-list.indent`
- `function` (level-property): The return value will be used for each level and each item.
  - The function form: `it => length | auto | array`
  - If the returned value is a `array`, then the elements in the array will be used for each item.
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n`: The index of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.tag`: The tag of the current item.

- `it.elem-tag`: The tag of the current enum or list.
- `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
- `it.hanging-type`: The hanging type of the current level.
- `it.marker`: The marker of the current label if it is a `list`.
- `it.number`: The number of the current label if it is a `enum`.
- `it.label-width` (dictionary): The width info of the current label.
  - `it.label-width.max`: The maximum width of the labels in the current level.
  - `it.label-width.current`: The current width of the label.
- `it.parent`: The parameter info of the parent containing the current list — the parent's construction element `e`, the item `n` where the parent is located, and the horizontal spacing parameters: `label-indent`, `label-inset`, `body-indent`, `indent`, `body-margin`, `label-width` (`max`, `current`)
- `it.get`: Access the parameter info (see `it.parent`) of the current list's ancestors by level

Argument

`⟨label-inset⟩: auto` — `auto` | `length` | `array` | `function`

Adds an inset to the label (default: `0pt` if `auto`).

- Note: It translates labels horizontally (without affecting the label width and the inset of the first line).
- Usage: See `#default-enum-list.indent`.
- `auto`: The label has not inset (equivalent to `0pt`).
- `length`: The inset is the specified value.
- `array` (level-property): The elements are `length`, `auto` or `array`, each level of the item will be inset by the corresponding value of the array at position `level - 1`.
  - See `#default-enum-list.indent`.
- `function` (level-property): The return value will be used for each level and each item.
  - The function form: `it => length | auto | array`
  - If the returned value is a `array`, then the elements in the array will be used for each item.
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n`: The index of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.tag`: The tag of the current item.
    - `it.elem-tag`: The tag of the current enum or list.
    - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
    - `it.hanging-type`: The hanging type of the current level.
    - `it.marker`: The marker of the current label if it is a `list`.
    - `it.number`: The number of the current label if it is a `enum`.
    - `it.label-width` (dictionary): The width info of the current label.
      - `it.label-width.max`: The maximum width of the labels in the current level.
      - `it.label-width.current`: The current width of the label.
    - `it.parent`: The parameter info of the parent containing the current list — the parent's construction element `e`, the item `n` where the parent is located, and the

horizontal spacing parameters: `label-indent`, `label-inset`, `body-indent`, `indent`, `body-margin`, `label-width` (`max`, `current`)
- `it.get`: Access the parameter info (see `it.parent`) of the current list's ancestors by level

Argument

⟨body-margin⟩: auto — `auto` | `relative` | `dictionary` | `array` | `function`

The left and right margin of the current item body. Based on the `itemize` design, `label-indent`, the label's content, `body-indent` are belong to the label's body. For `ltr`, the left margin starts from the left of the label's body; for `rtl`, the right margin starts from the right of the label's body.

- In particular, `body-margin` can control the starting alignment position of the block-level element in the item body and the actual `100%` width.
- **Note**: In general, the layout of `inline-level` elements in the item body are not affected by `body-margin` (except for the actual `100%` width).
  - The layout of `inline-level` elements usually are affected by `line-indent` and `hanging-indent`.
- With this attribute, it is now very easy to implement some lists with very special layouts.
  - Example: The body of all nested lists is aligned to the left.

```
#show: el.default-enum-list.with(
  body-margin: 0pt,
  indent: (2em, auto),
  label-indent: it => { -it.label-width.current - it.e.body-indent },
)
#set enum(full: true)
+ Level One
  + Level Two
    + #lorem(2)
  + Level Two
+ Level One
```

  - Example: Align the (first-level) item body and the first line of the paragraph, e.g., indent both by 2em.

```
#let indent = 2em
#set par(first-line-indent: (amount: indent, all: true))
#show: el.default-enum-list.with(
  body-margin: (0pt, auto),
  indent: (indent, auto),
  label-indent: it => {
    if it.level == 1 {
      -it.label-width.current - it.e.body-indent
    } else {
      auto
    }
```

```
  },
)
#lorem(15)
+ #lorem(15)
  + #lorem(15)
+ #lorem(15)
```

- Example: Align the (first level) item body and the first line of the paragraph with the `paragraph` hanging style.

```
#let indent = 2em
#set par(first-line-indent: (amount: indent, all: true))
#show: el.default-enum-list.with(
  body-margin: (0pt, auto),
  indent: (0pt, indent, auto),
  label-indent: it => {
    if it.level == 1 {
      indent - it.label-width.current - it.e.body-indent
    } else {
      auto
    }
  },
)
#lorem(15)
+ #lorem(15)
  + #lorem(15)
+ #lorem(15)
```

- For `ltr` typography, `body-margin.left` (left body margin) is called `start-margin`, `body-margin.right` (right body margin) is called `end-margin`; for `rtl` typography, `body-margin.left` (left body margin) is called `end-margin`, `body-margin.right` (right body margin) is called `start-margin`.
- `auto`: Set `start-margin` and `end-margin` to `auto`. In this case:
  - `end-margin` is `0pt`
  - `start-margin` is the label's body width, i.e., the rendered width of the label + `label-indent` + `body-indent`. (Also affected by the `label-width`.)
    - **Note**: This also works for `hanging-type` is `paragraph` (and so `paragraph-*` method); but in this case, the `hanging-indent` and `line-indent` are reset to make the paragraphs in item body aligned to the left.
- `relative`: Set `start-margin` and `end-margin` to the same specified value.
  - For `start-margin`, the ratio part of `start-margin` is relative to the `label-width`, that is, `start-margin` = `start-margin.length + start-margin.ratio × label's body width`, where **label's body width** = the rendered width of the label + `label-indent` + `body-indent`.
  - For `end-margin`, the ratio part of `end-margin` is relative to the parent container's width.
- `dictionary`: The key values are `left` and `right`, each of which can be `relative`, `auto`. Then the `start-margin` and `end-margin` are set to the corresponding value.

- `array` (level-property): The elements are `relative`, `auto`, `dictionary`, or `array`. The `level`-th of the `enum` or `list` will be set to the corresponding value of the array at position `level - 1`.
  - See `#default-enum-list.indent`.
- `function` (level-property): The return value will be used for each level.
  - The function form: `it => relative | auto | dictionary | array`.
  - See `#default-enum-list.indent`.

Argument

`⟨hanging-type⟩: "classic"` `str` | `array` | `function`

The hanging type of the paragraphs inside the item body.

- Usage:

```
#show: el.default-enum-list.with(hanging-type: "paragraph")
#show: el.default-enum-list.with(hanging-type: ("classic", "paragraph",
"classic"))
#show: el.default-enum-list.with(hanging-type: it => if calc.odd(it.level)
{"paragraph"} else {"classic"})
```

- `"classic"`: The native behavior of `enum` or `list`.
- `"paragraph"`: For each level, paragraphs inside the item body are not indented; they align to the current `body-margin`'s beginning margin (the same as the `paragraph-*` methods).
  - Note: But the current `body-margin`'s beginning margin is the same as the `"classic"` one.
  - Hint: See also the `inline-*` methods (`#inline-enum-list`, `#inline-enum`, `#inline-list`).
- `array` (level-property): The elements are `"classic"` or `"paragraph"`. Each level of `hanging-type` will be determined by the corresponding value of the array at position `level - 1`.
  - If the last element of the array is `LOOP`, the values in the array will be used cyclically;
  - otherwise, the last value of the array will be used for residual levels.
- `function` (level-property): The return value ("classic" or "paragraph") will be used for each level.
  - The function form: `it => "classic" | "paragraph"`
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.elem-tag`: The tag of the current enum or list.
    - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.

Argument

`⟨line-indent⟩: auto` `auto` | `length` | `array` | `function`

The indentation for the first line of a paragraph, excluding the first paragraph (default: `auto`).

- Note: Paragraphs inside `block-level` elements that appear in the item body are not processed (the related settings are not yet exposed).
- Usage: See `#default-enum-list.indent`.
- `auto`: It uses the first line indent of the current paragraph (`par.first-line-indent.amount`).

- `length`: The value of the line indent.
- `array` (level-property): The elements are `length`, `auto` or `array`, each level of the item will be indented by the corresponding value of the array at position `level - 1`.
  - See `#default-enum-list.indent`.
- `function` (level-property): The return value will be used for each level and each item.
  - The function form: `it => length | auto | array`
  - If the returned value is a `array`, then the elements in the array will be used for each item.
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n`: The index of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.tag`: The tag of the current item.
    - `it.elem-tag`: The tag of the current enum or list.
    - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
    - `it.hanging-type`: The hanging type of the current level.
    - `it.marker`: The marker of the current label if it is a `list`.
    - `it.number`: The number of the current label if it is a `enum`.
    - `it.label-indent`, `it.label-inset`, `it.body-indent`, `it.indent`, `it.body-margin`: All the horizontal spacing parameters.
    - `it.label-width` (dictionary): The width info of the current label.
      - `it.label-width.max`: The maximum width of the labels in the current level.
      - `it.label-width.current`: The current width of the label.
    - `it.parent`: The parameter info of the parent containing the current list — the parent's construction element `e`, the item `n` where the parent is located, and the horizontal spacing parameters: `label-indent`, `label-inset`, `body-indent`, `indent`, `body-margin`, `label-width` (`max`, `current`)
    - `it.get`: Access the parameter info (see `it.parent`) of the current list's ancestors by level

Argument

`⟨first-line-inset⟩: auto` `auto` | `length` | `array` | `function`

Add an inset to the first line of the item body (default: `0pt` if `auto`). If there is no first line, it is ignored.

- Note: Paragraphs inside `block-level` elements that appear in the item body are not processed (the related settings are not yet exposed).
- Usage: See `#default-enum-list.indent`.
- `auto`: The first line is not inset (equivalent to `0pt`).
- `array` (level-property): The elements are `length`, `auto` or `array`, each level of the item will be inset by the corresponding value of the array at position `level - 1`.
  - See `#default-enum-list.indent`.
- `function` (level-property): The return value will be used for each level and each item.
  - The function form: `it => length | auto | array`
  - If the returned value is a `array`, then the elements in the array will be used for each item.
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.

- `it.n`: The index of the item, starting from 1.
- `it.n-last`: The index of the last item in the current level.
- `it.tag`: The tag of the current item.
- `it.elem-tag`: The tag of the current enum or list.
- `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
- `it.hanging-type`: The hanging type of the current level.
- `it.marker`: The marker of the current label if it is a `list`.
- `it.number`: The number of the current label if it is a `enum`.
- `it.label-indent`, `it.label-inset`, `it.body-indent`, `it.indent`, `it.body-margin`: All the horizontal spacing parameters.
- `it.label-width` (dictionary): The width info of the current label.
  - `it.label-width.max`: The maximum width of the labels in the current level.
  - `it.label-width.current`: The current width of the label.
- `it.parent`: The parameter info of the parent containing the current list — the parent's construction element `e`, the item `n` where the parent is located, and the horizontal spacing parameters: `label-indent`, `label-inset`, `body-indent`, `indent`, `body-margin`, `label-width` (`max`, `current`)
- `it.get`: Access the parameter info (see `it.parent`) of the current list's ancestors by level

Argument

⟨hanging-indent⟩: auto — `auto` | `length` | `array` | `function`

The hanging indentation for item body.

- Note: Paragraphs inside `block-level` elements that appear in the item body are not processed (the related settings are not yet exposed).
- Usage: See `#default-enum-list.indent`.
- `auto`: It uses the hanging indent of the current paragraph (`par.hanging-indent`).
- `length`: The hanging indentation is fixed to the specified value.
- `array` (level-property): The elements are `length`, `auto` or `array`, each level of the item will be indented by the corresponding value of the array at position `level - 1`.
  - See `#default-enum-list.indent`
- `function` (level-property): The return value will be used for each level and each item.
  - The function form: `it => length | auto | array`
  - If the returned value is a `array`, then the elements in the array will be used for each item.
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n`: The index of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.tag`: The tag of the current item.
    - `it.elem-tag`: The tag of the current enum or list.
    - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
    - `it.hanging-type`: The hanging type of the current level.
    - `it.marker`: The marker of the current label if it is a `list`.
    - `it.number`: The number of the current label if it is a `enum`.

- `it.label-indent`, `it.label-inset`, `it.body-indent`, `it.indent`, `it.body-margin`: All the horizontal spacing parameters.
- `it.label-width` (dictionary): The width info of the current label.
  - `it.label-width.max`: The maximum width of the labels in the current level.
  - `it.label-width.current`: The current width of the label.
- `it.parent`: The parameter info of the parent containing the current list — the parent's construction element `e`, the item `n` where the parent is located, and the horizontal spacing parameters: `label-indent`, `label-inset`, `body-indent`, `indent`, `body-margin`, `label-width` (`max`, `current`)
- `it.get`: Access the parameter info (see `it.parent`) of the current list's ancestors by level

Argument

⟨whole-spacing⟩: auto — `auto` | `relative` | `fraction` | `dictionary` | `array` | `function`

The above and below spacing of the current `enum` or `list`.

- Usage: See `#default-enum-list.item-spacing`.
- `auto`: The above and below spacing are set to `auto` (See also `block.spacing`). In this case, it may be affected by the `tight-mode`.
- `relative`, `fraction`: The above and below spacing are set to the specified value.
- `dictionary`: The key values are `above` and `below`, each of which can be `relative`, `fraction` or `auto`. Then the above and below spacing are set to the corresponding value.
- `array` (level-property): The elements are `relative`, `fraction`, `auto`, or `dictionary`, the `level`-th of the `enum` or `list` will be set to the corresponding value of the array at position `level - 1`.
  - If the last element of the array is `LOOP`, the values in the array will be used cyclically;
  - otherwise, the last value of the array will be used for residual levels.
- `function` (level-property): The return value will be used for each level.
  - The function form: `it => auto | relative | fraction | dictionary`
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.elem-tag`: The tag of the current enum or list.
    - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
    - `it.hanging-type`: The hanging type of the current level.

Argument

⟨item-spacing⟩: auto — `auto` | `relative` | `fraction` | `dictionary` | `array` | `function`

Spacing between items.

- Usage:

```
#show: el.default-enum-list.with(item-spacing: 1em)
#show: el.default-enum-list.with(item-spacing: (below: 1fr, above: auto))
#show: el.default-enum-list.with(item-spacing: (1em, (below: 1fr), auto))
#show: el.default-enum-list.with(item-spacing: it => it.level * 1em)
```

- `auto`: The spacing is determined by the `enum.spacing` or `list.spacing`. And in this case, it may be affected by the `tight-item-mode`.
- `relative`, `fraction`: The value of spacing between each item; whose rule is the same as `block.spacing`.
- `dictionary`: The key values are `above` and `below`, each of which can be `relative`, `fraction` or `auto`.
  - Note: The `above` spacing is ignored for the first item, but the `below` spacing is added to the last item.
  - The spacing between the previous and current items is the previous `below` plus the current `above`.
    - Note: This differs from setting `item-spacing` to a concrete value.
- `array` (level-property): The elements are `relative`, `fraction`, `dictionary`, `auto`, and `array`.
  - See #`default-enum-list.indent`.
- `function` (level-property): The return value will be used for each level and each item.
  - The function form: `it => relative | fraction | dictionary | auto | array`
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n`: The index of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.tag`: The tag of the current item.
    - `it.elem-tag`: The tag of the current enum or list.
    - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
    - `it.hanging-type`: The hanging type of the current level.
    - `it.marker`: The marker of the current label if it is a `list`.
    - `it.number`: The number of the current label if it is a `enum`.

Argument

`(tight-mode): auto` — `auto` | `str` | `dictionary` | `array` | `function`

If `whole-spacing` are `auto`, then `tight-mode` is used to determine the above and below spacing of `enum` or `list`.

- Usage:

```
#show: el.default-enum-list.with(tight-mode: "always-tight")
#show: el.default-enum-list.with(tight-mode: ("never-tight", "always-tight", auto))
#show: el.default-enum-list.with(tight-mode: (tight: auto, not-tight: 2em))
```

- Background: For the current `enum` or `list`, the above spacing (denoted as `above-spacing`) and the below spacing (denoted as `below-spacing`) are determined by the following cases:
  1. The current `enum` or `list` is `tight` (i.e. `enum.tight` or `list.tight` is `true`, see #`default-enum-list.tight-item-mode`);
  2. The current `enum` or `list` is not `tight` (i.e. `enum.tight` or `list.tight` is `false`, see #`default-enum-list.tight-item-mode`);
  3. There is a `parbreak()` between the above paragraph and the current `enum` or `list`: like

```
The preceding paragraph.  // below has a `parbreak()`

+ items
```

4. There is no `parbreak()` between the above paragraph and the current `enum` or `list`: like

```
The preceding paragraph.
+ items
```

   - Combining these cases, we call
     - case 1 with case 3 **par-tight** (i.e. `tight` with a `parbreak()` above),
     - case 2 with case 3 **par-not-tight**,
     - case 1 with case 4 **tight**,
     - case 2 with case 4 **not-tight**.
   - Note: In native Typst behavior, cases **par-not-tight**, **par-tight** and **not-tight** are the same.

- `auto`: Use the native Typst behavior of `enum` or `list`.
  - for the **tight** case: `above-spacing` is `enum.spacing` or `list.spacing` (if `auto`, representing `par.leading`); for the other cases it is `auto` (representing `par.spacing`).
  - `below-spacing` is `auto` (representing `par.spacing`)
- `"always-tight"`: `above-spacing` and `below-spacing` are `par.leading` and `auto`, respectively.
- `"never-tight"`: `above-spacing` and `below-spacing` are `par.spacing` and `auto`, respectively.
- `"compact-tight"`: `above-spacing` and `below-spacing` are both `par.leading`.
- `dictionary`: The keys are: `tight`, `not-tight`, `par-tight`, `par-not-tight`, with values of `auto`, `relative`, `fraction`, `array` (with two elements representing the above and below spacing).
  - If values of the keys are taken `auto`, `relative`, `fraction`, then they represent the `above-spacing`; `below-spacing` is `auto`.
  - If values of the keys are `array`, then the first element represents the `above-spacing`, and the second element represents the `below-spacing`.
  - If `tight` is `auto`, use `enum.spacing` or `list.spacing` (if `auto`, representing `par.leading`); if `not-tight` is `auto`, use `auto` (representing `par.spacing`); if `par-tight` and `par-not-tight` are `auto`, use the value of `not-tight`
  - `tight`, `not-tight`, `par-tight`, `par-not-tight` represent the **tight** case, **not-tight** case, **par-tight** case, **par-not-tight** case, respectively.
  - **Note**: In order to set `below-spacing` from `par-tight` and `par-not-tight`, you need to add in the document:

```
#show: el.config.auto-detect-tight
```

  - Example:

```
#let first-level-tight-mode = (
  tight: (auto, 0.65em),
  par-tight: (auto, auto),
  not-tight: (1.5em, 1.5em),
  par-not-tight: (2em, 2em),
```

```
  )
#show: el.default-enum-list.with(tight-mode: (first-level-tight-mode, auto))
#show: el.config.auto-detect-tight
*Tight*: #lorem(2) *with parbreak*

+ #lorem(2) *without parbreak*
+ #lorem(2)
End paragraph

#lorem(2) *without parbreak*
+ #lorem(2) *without parbreak*
+ #lorem(2)
End paragraph
#divider()
*Not Tight*: #lorem(2) *with parbreak*

+ #lorem(2) *with parbreak*

+ #lorem(2)
End paragraph

#lorem(2) *without parbreak*
+ #lorem(2) *with parbreak*

+ #lorem(2)
End paragraph
```

- `array` (level-property): Sets `tight-mode` by level. The elements are `auto`, `"always-tight"`, `"never-tight"`, `"compact-tight"`, `dictionary`.
  - If the last element of the array is `LOOP`, the values in the array will be used cyclically;
  - otherwise, the last value of the array will be used for residual levels.
- `function` (level-property): The return value will be used for each level.
  - The function form: `it => auto | "always-tight" | "never-tight" | "compact-tight" | dictionary`
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.elem-tag`: The tag of the current enum or list.
    - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
    - `it.hanging-type`: The hanging type of the current level.

Argument

`⟨tight-item-mode⟩`: `auto` — `auto` | `str` | `dictionary` | `array` | `function`

If `item-spacing` is `auto`, then `tight-item-mode` is used to determine the spacing between items.

- Usage:

```
#show: el.default-enum-list.with(tight-item-mode: "always-tight")
#show: el.default-enum-list.with(tight-item-mode: ("never-tight", auto))
#show: el.default-enum-list.with(tight-item-mode: ((tight: auto, not-tight: 2em), auto))
```

- Background: The spacing between items is determined by the following cases:
  1. **tight**: There is no `parbreak()` between items, like:

```
+ item one // no `parbreak()` below
+ item two // no `parbreak()` below
+ item three
```

  2. **not-tight**: There is a `parbreak()` between items, like:

```
+ item one // has a `parbreak()` below

+ item two
+ item three
```

- `auto`: Use the native Typst behavior of `enum` or `list`.
  - The spacing uses `enum.spacing` or `list.spacing` (if `auto`, representing `par.leading` for the **tight** case and `par.spacing` for the **not-tight** case).
- `"always-tight"`: The spacing is `par.leading`.
- `"never-tight"`: The spacing is `par.spacing`.
- `dictionary`: The keys are `tight`, `not-tight`, with values of `auto`, `relative`, `fraction`.
  - If `tight` is `auto`, use `par.leading`
  - If `not-tight` is `auto`, use `par.spacing`
  - Use the values of `tight` and `not-tight` for the **tight** case and the **not-tight** case, respectively.
- `array` (level-property): Sets `tight-item-mode` by level. The elements are `auto`, `"always-tight"`, `"never-tight"`, `dictionary`.
  - See `#default-enum-list.tight-mode`.
- `function` (level-property): The return value will be used for each level.
  - The function form: `it => auto | "always-tight" | "never-tight" | dictionary`
  - See `#default-enum-list.tight-mode`.

Argument

`(label-width): auto` — `auto` | `relative` | `str` | `dictionary` | `array` | `function`

Render the width of the label.

- Usage:

```
#show : el.default-enum-list.with(label-width: 2em)
#show : el.default-enum-list.with(label-width: ((amount: 2em, style: "constant"), auto))
```

- **Notation**:
  - `real-width`: The real width of the current label's content (calculated by `measure`).

  - `max-width`: The maximum value of all `real-widths` in the current level.
- `auto`: Use the native behavior. That is, the current width of the label will be set to `max-width`.
- `relative`: This is the same as `(amount: len, style: "default")`.
- `"max"`: This is the same as `auto`.
- `dictionary`: The keys are `amount` and `style`.
  - `amount` can be taken `relative`, `auto` or `"max"`
    - `relative`: The value is `amount.length + amount.ratio × real-width`.
    - `auto`: `real-width`.
    - `"max"`: `max-width`.
  - `style` can be `"default"`, `"constant"`, `"auto"`, or `"native"`.
    - `"default"`: If `real-width` is less than `amount`, the label's width is rendered as `amount`; otherwise, the label's width is rendered as `real-width`.
    - `"constant"`: The label's width is rendered as `amount`.
    - `"auto"`: The label's width is rendered as `real-width`.
    - `"native"`: The label's width is rendered as `max-width`.
      - Also `amount` is set to `max-width` (since ver0.3.0).
    - **Note**: In order to make the layout more in line with expectations, the `start-margin` (i.e., `body-margin.left` if `text.dir` is `ltr`, `margin-body.right` if `text.dir` is `rtl`) will be set to `amount + body-indent` (if `start-margin` is `auto`).
- `array` (level-property): The elements are `auto`, `relative`, `string`, `dictionary` or `array`. The element at the position `level - 1` will be used for the `level`-th level.
  - See `#default-enum-list.indent`.
- `function` (level-property): The return value will be used for each level and each item.
  - The function form: `it => auto | relative | string | dictionary | array`
  - If the returned value is a `array`, then the elements in the array will be used for each item.
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n`: The index of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.tag` (since ver0.3.0): The tag of the current item.
    - `it.elem-tag` (since ver0.3.0): The tag of the current enum or list.
    - `it.e` (since ver0.3.0): The field of the current construct element (`enum` or `list`), but excluding the `children`.
    - `it.hanging-type` (since ver0.3.0): The hanging type of the current level.
    - `it.marker` (since ver0.3.0): The marker of the current label if it is a `list`.
    - `it.number` (since ver0.3.0): The number of the current label if it is a `enum`.
    - `it.max-width` (since ver0.3.0): The maximum width of the labels in the current level.
    - `it.width` (since ver0.3.0): The current width of the label.

Argument

`⟨label-align⟩: auto` `auto` | `alignment` | `array` | `function`

The `alignment` that enum numbers and list markers should have.

- Note: This is different with `enum.number-align` and `list.marker-align` (the latter are align whole enum numbers and list markers). We suggest to make `enum.number-align` and `list.marker-align` to be default when using `itemize`.

- **Note**: In general, this parameter is used to set **horizontal** alignment for label. If want 2D-alignment, then see #default-enum-list.label-baseline.
- Usage:

```
#show : el.default-enum-list.with(label-align: left)
#show : el.default-enum-list.with(label-align: (left, right))
```

- `auto`: Use `end`.
- `alignment`: The alignment of the label.
- `array` (level-property): The elements are `auto`, `alignment` or `array`, each level of the item will be set to the corresponding value of the array at position `level - 1`.
  - See #default-enum-list.indent.
- `function` (level-property): The return value will be used for each level and each item.
  - The function form: `it => auto | alignment | array`
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n`: The index of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.tag`: The tag of the item.
    - `it.elem-tag`: The tag of the `enum` or `list`.
    - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
    - `it.hanging-type`: The hanging type of the current level.

Argument

⟨label-baseline⟩: auto — `auto` | `relative` | `str` | `dictionary` | `array` | `function`

Set the baseline style of the label body.

- Usage:

```
#show : el.default-enum-list.with(label-baseline: "center")
#show : el.default-enum-list.with(label-baseline: ("top-item", auto))
#show : el.default-enum-list.with(label-baseline: (same-line-style: "top-item")) // The default behavior of native `enum` or `list` (Typst 0.15.1)
#show : el.default-enum-list.with(label-baseline: 1em)
#show : el.default-enum-list.with(label-baseline: (impact-first-line: true))
```

- **Note**: By default, `itemize` prioritizes the vertical component of `enum.number-align` or `list.marker-align`. For `label-baseline` to control the vertical alignment of the label body, set the corresponding `number-align`/`marker-align` to its default value (i.e., `end`).
- The full form of `label-baseline` is a dictionary with the following keys:
  - `amount`
  - `at`
  - `relative-to`: a dictionary with keys:
    - `to`, `at`, `shift`
  - `same-line-style`: a dictionary with keys:
    - `amount`, `at`, `relative-to` (a dictionary with keys: `to`, `at`, `shift`)
  - `impact-first-line`

- `auto`: The default of `label-baseline` configuration (i.e., using the native behavior), which is the same as `(amount: 0pt, at: auto, relative-to: "baseline", same-line-style: auto, impact-first-line: auto)`
- `relative`: An amount to shift the label baseline by.
  - The same as `(amount: label-baseline)`.
  - The ratio part is relative to the (rendered) height of the (whole) label body.
- `"center"`, `"top"`, `"bottom"`, `"baseline"`: The (first line of the) label body and the first line of the item body will be aligned at the specified (i.e., horizontal, top, bottom, baseline) position. For example, for `"top"`, it means that the top of label body and the top of item body will be aligned.
  - The same as `(amount: label-baseline, at: at)`, where `at` is `auto` if `label-baseline` is `baseline`, `top` if `label-baseline` is `top`, `bottom` if `label-baseline` is `bottom`, and `horizon` if `label-baseline` is `center`.
- `"top-item"`, `"horizon-item"`, `"bottom-item"`: The top, horizontal, or bottom of the label body will be aligned to the top, horizontal, or bottom of the whole item body, respectively.
  - The same as `(amount: label-baseline)`.
  - **Note**: `itemize` uses the height of the first character (i.e., [A]) in the first line of the item body. If you manually set the font style of the paragraph's text, this alignment may not be accurate. It is recommended to use the `style` parameter in `body-format` for adjustments.
- `dictionary`: The keys are
  - `amount` (`auto`, `relative`, `"center"`, `"top"`, `"bottom"`, `"baseline"`, `"top-item"`, `"horizon-item"`, `"bottom-item"`): The shift amount of the label body.
    - `auto` == `0pt` (default)
    - **Note**: Only when `amount` is `auto` or `relative`, the keys `at` and `relative-to` can work.
  - `at` (`auto`, `top`, `bottom`, `horizon`): Used to set what the position of the label body to be aligned; the default of `at` is `auto`, meaning the the baseline of the (first line of the) label body. See also the `baseline` parameter of `box`, https://typst.app/docs/reference/layout/box/#parameters-baseline.
  - `relative-to` (`string`, `content`, `relative`, `array`, `dictionary`): Used to set the offset relative to what position (of item body). Default is `"baseline"`.
    - `"bottom"`, `"center"`, `"top"`, `"baseline"`: Relative to the first line of the current item body.
    - `"top-item"`, `"horizon-item"`, `"bottom-item"`: Relative to the top, horizontal, or bottom of the whole item. In this case, `at` is set to `top`, `horizon`, `bottom`, respectively. Users can not set it again.
    - `content`: The passed content is used to calculate the height. The same as `(to: relative-to, at: auto, shift: 0pt)`.
    - `relative`: Representing the height of the first line of the current item body. The same as `(to: [A], at: auto, shift: relative-to)`.
      - The ratio part is relative to the (rendered) height of the (whole) item body.
    - `array` with 2 elements like `(relative-shift, relative-to)`. This means the label body should be aligned to `relative-to` with the offset of `relative-shift`.
      - The first element `relative-shift` can be `content` or `relative`.
      - The second element `relative-to` can be `"bottom"`, `"center"`, `"top"`, `"baseline"`.
    - `dictionary` with keys: `to`, `at`, `shift`. This means the label body should be aligned to the content of `to`, at the `at` position with the offset of `shift`.

- **Note**: Usually, the content of `to` should be added to the first line of the item body.
- `to` (`auto`, `content`): If `auto`, it is `[A]`. Default is `[A]`.
- `at` (`auto`, `top`, `bottom`, `horizon`): Used to set what the position of `to` to be aligned. Default is `auto`.
- `shift` (`relative`): The offset of `shift` at `at` position. The ratio part is relative to the height of the content `to`. Default is `0pt`.

‣ `impact-first-line` (`auto`, `bool`): Whether to impact the first line of the label body by `label-baseline`. Default is `auto`.

– `auto`: If `#default-enum-list.hanging-type` is `"paragraph"`, then `true`, else `false`.

‣ `same-line-style` (`auto`, `string`, `dictionary`): If labels from different levels appear on the same line, their alignment is determined by `same-line-style`. That is, it deals with like `+ + Item`. Default is `auto`.

– In Typst 0.15, like `+ + Item`, the first label is "top-aligned" to `+ Item` (since `enum` or `list` has no baseline). In default, in `itemize` (since ver0.2.0), the first label and `+ Item` is aligned at baseline.

– `auto`: Use the current setting of `label-baseline`.

– `"top"`, `"center"`, `"bottom"`, `"baseline"`: The (first line of the) label body on the same line and the first line of the item body will be aligned at the specified (i.e., top, horizontal, bottom, baseline) position.

– `"top-item"`, `"horizon-item"`, `"bottom-item"`: The top, horizontal, or bottom of the label body on the same line will be aligned to the top, horizontal, or bottom of the whole item body, respectively.

– `"top-label"`, `"center-label"`, `"bottom-label"`, `"baseline-label"`: The top, horizontal, or bottom of the label body on the same line will be aligned to the top, horizontal, or bottom of the **last** label body, respectively.

– `dictionary` with keys `amount`, `at`, `relative-to`. This means the label body on the same line will be shifted at the `at` position with the offset of `amount`, and will be aligned determined by `relative-to`.

- `amount` (`auto`, `relative`): The shift amount of the label body on the same line. Default is `0pt`.
  - `auto` == `0pt`.
  - The ratio part is relative to the (rendered) height of the (whole) label body.
- `at` (`auto`, `top`, `bottom`, `horizon`): Used to set what the position of the label body on the same line to be aligned. Default is `auto` (the baseline of the label body).
- `relative-to` (`string`, `array`, `dictionary`): Used to set the offset relative to what position (of item body). Default is `"baseline"`. See also the key `relative-to` in `#default-enum-list.label-baseline`.
  - `"top"`, `"center"`, `"bottom"`, `"baseline"`: Relative to the first line of the current item body.
  - `"top-item"`, `"horizon-item"`, `"bottom-item"`: Relative to the top, horizontal, or bottom of the whole item.
  - `"top-label"`, `"center-label"`, `"bottom-label"`, `"baseline-label"`: Relative to the top, horizontal, or bottom of the last label body.
  - `content`: The passed content is used to calculate the height. The same as `(to: relative-to, at: auto, shift: 0pt)`.

  - `relative`: Representing the height of the first line of the current item body. The same as `(to: [A], at: auto, shift: relative-to)`.
    - The ratio part is relative to the (rendered) height of the (whole) item body.
  - `array` with 2 elements like `(relative-shift, relative-to)`. This means the label body on the same line should be aligned to `relative-to` with the offset of `relative-shift`.
    - The first element `relative-shift` can be `content` or `relative`.
    - The second element `relative-to` can be `"bottom"`, `"center"`, `"top"`, `"baseline"`.
  - `dictionary` with keys `to`, `at`, `shift`. This means the label body on the line should be aligned to `to` (the last label body or content), at the `at` position with the offset of `shift`.
    - `to` (`auto`, `"label"`, `content`): If `"label"`, this means the label body on the line should be aligned to the last label body. If `auto`, it is `[A]`.
    - `at` (`auto`, `top`, `horizon`, `bottom`): Used to set what the position of `to` to be aligned.
    - `shift` (`relative`): The offset of `shift` at `at` position. The ratio part is relative to the height of the content `to` if `to` is `content` or the last label body if `to` is `"label"`.
- Example: In the following, the bottom of the first label is aligned to the bottom of second label.

```
#show: el.default-enum-list.with(
  body-format: (stroke: (red, auto)),
  label-baseline: ((same-line-style: "bottom-label"), auto),
)
#set list(marker: [#circle(width: 1em, fill: red)])
- - #el.item(label-format: (stroke: blue), [$vec(1, 1, 1)$]) #lorem(20)
  #lorem(10)
```

- `array` (level-property): The elements are `auto`, `relative`, `"center"`, `"top"`, `"bottom"`, `"top-item"`, `"horizon-item"`, `"bottom-item"`, `"baseline"`, `dictionary`, or `array`.
  - If the last element of the array is `LOOP`, the values in the array will be used cyclically;
  - otherwise, the last value of the array will be used for residual levels.
  - The elements in the array can also be an array, where the element at position `n - 1` applies to the item of index `n`.
- `function` (level-property): The return value will be used for each level and each item.
  - The function form: `it => auto | relative | "center" | "top" | "bottom" | "top-item" | "horizon-item" | "bottom-item" | "baseline" | dictionary | array`.
  - If the returned value is a `array`, then the elements in the array will be used for each item.
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n`: The index of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.tag`: The tag of the current item.
    - `it.elem-tag`: The tag of the current enum or list.
    - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
    - `it.hanging-type`: The hanging type of the current level.
    - `it.marker`: The marker of the current label if it is a `list`.

- `it.number`: The number of the current label if it is a `enum`.

Argument

`(label-format): none` `none` | `dictionary` | `function` | `array`

Customize item labels.

- Usage:

```
#show: el.default-enum-list.with(label-format: (stroke: red))
#show: el.default-enum-list.with(label-format: (fill: (red, blue, auto)))
#show: el.default-enum-list.with(label-format: (emph, strong))
#show: el.default-enum-list.with(label-format: it => {
  if it.level == 1 {
    circle(it.body, inset: 1pt)
  } else {
    strong(it.body)
  }
})
```

- `none`: Does not take effect.
- `content`: The content of the label.
- `dictionary`: The keys are
  - `format`: The format of the label.
    - `none` == `auto` (default): The same as `it => it.body`.
    - `content`: The content of the label.
    - `function` (base-property):
      - The function form: `body => content`. Apply the function to the label's content (generated by `enum.item.number` + `enum.numbering` or `list.marker`).
      - The `function` should be wrapped in an `array`, like: `(strong, )`.
    - `array` (level-property): The elements are `content`, `function` (base-property) or `array`. The element at the position `level - 1` will be used for the `level`-th level.
      - If the last element of the array is `LOOP`, the values in the array will be used cyclically;
      - otherwise, the last value of the array will be used for residual levels.
    - `function` (level-property):
      - The function form: `it => content`.
      - If the return value is `array`, then each element in the array should be `auto`, `content` or `function` (base-property), which will be applied to the label's content for each item in the current level.
      - Access the following values available on `it`:
        - `it.body`: The current label's content.
        - `it.level`: The level of the item, starting from 1.
        - `it.n`: The index of the item, starting from 1.
        - `it.n-last`: The index of the last item in the current level.
        - `it.tag`: The tag of the item.
        - `it.elem-tag`: The tag of the `enum` or `list`.
        - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.

    - `it.hanging-type`: The hanging type of the current level.
  - Example:

```
#show: el.default-enum-list.with(label-format: (format: it => {
  if it.tag == "strong" {
    set text(fill: red, weight: "bold")
    it.body
  } else {
    it.body
  }
}))
+ #el.item(tag: "strong") #lorem(2)
+ #lorem(2)
  + #lorem(2)
  + #el.item(tag: "strong") #lorem(2)
```

- `stroke`, `radius`, `outset`, `fill`, `inset`, `clip`: The **border** of the label, which are the same as `box` arguments.
  - `auto`: The default value of the argument.
  - the values of the corresponding named arguments of `box` that can be set.
  - `array` (level-property): The elements are `auto`, the values of the corresponding named arguments of `box` that can be set, or `array`. The element at the position `level - 1` will be used for the `level`-th level.
    - If the last element of the array is `LOOP`, the values in the array will be used cyclically;
    - otherwise, the last value of the array will be used for residual levels.
  - `function` (level-property): The return value will be used for each level and each item.
    - The function form: `it => auto | the value of the corresponding named argument of box that can be set | array`
    - Access the following values available on `it`:
      - `it.level`: The level of the item, starting from 1.
      - `it.n`: The index of the item, starting from 1.
      - `it.n-last`: The index of the last item in the current level.
      - `it.tag`: The tag of the item.
      - `it.elem-tag`: The tag of the `enum` or `list`.
      - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
      - `it.hanging-type`: The hanging type of the current level.
- `align` (`auto`, `alignment`, `array`, `function`): The alignment of the label. See `stroke` etc. Default is `start`.
- `width-style`, `height-style`: Set the width and height style of the label.
  - `width-style`:
    - **Note**: This does not affect the maximum width of the labels in the current level.
    - `"real-width"`, `"label-width"`, `relative`: The same as `(amount: len, stretched: false)`.
      - `"real-width"`: The real width of the label.
      - `"label-width"`: The rendered width of the label determined by the `label-width` parameter.

- `dictionary`: The keys are
  - `amount` (`"real-width"`, `"label-width"`, or `relative`): Set the width of the label to be the specified value.
    - The ratio part of `amount` is relative to the real width of the label, i.e., the value of `amount` is `amount.ratio × real-width + amount.length`, where `real-width` is the real width of the label.
  - `stretched` (bool): If `true`, the rendered width of the label will be set to `amount + inset.x`, else the rendered width of the label will be determined by the `label-width` parameter (in particular, it does not affect the layout of the item body.).
- `height-style`:
  - `"real-height"`, `relative`: The same as `(amount: len, stretched: false)`.
  - `"real-height"`: The real height of the label.
  - `dictionary`: The keys are
    - `amount` (`"real-height"`, `relative`): Set the height of the label to be the specified value.
      - The ratio part of `amount` is relative to the real height of the label, i.e., the value of `amount` is `amount.ratio × real-height + amount.length`, where `real-height` is the real height of the label.
    - `stretched` (bool): If `true`, the rendered height of the label will be set to `amount + inset.y`, else use the real height of the label (in particular, it does not affect the layout of the item body.).
  - Example

```
#show: el.default-enum-list.with(label-format: (
  stroke: (red, blue),
  align: center + horizon,
  width-style: (amount: 1.2em, stretched: true),
  height-style: (amount: 1.2em, stretched: true),
  radius: 1em,
))
#set enum(numbering: "1.A")
+ #lorem(10)
+ #lorem(10)
  + #lorem(10)
  + #lorem(10)
```

- `function` (base-property): The same as `(format: func)`.
- `array` (level-property): The same as `(format: (func1, func2,...))`.
- `function` (level-property): The same as `(format: func)`.

Argument

`⟨body-format⟩: none` — `none` | `dictionary` | `function` | `array`

Customize item bodies.

- Usage:

```
#show: el.default-enum-list.with(body-format: (stroke: red))
```

```
#show: el.default-enum-list.with(body-format: (whole: (stroke: (red, blue, auto))))
#show: el.default-enum-list.with(body-format: (
  inner: (
    stroke: (red, blue, auto),
    inset: (5pt, 5pt, auto),
  ),
))
#show: el.default-enum-list.with(body-format: (
  style: (
    fill: (red, blue, auto),
    size: (1.2em, 12pt, auto),
  )
))
#show: el.default-enum-list.with(body-format: (emph, strong, auto))
#show: el.default-enum-list.with(body-format: it => {
  if it.e.tight {
    show: strong
    set text(fill: red)
    it.body
  } else {
    it.body
  }
})
```

- `none`: Does not take effect.
- `dictionary`: The keys are
  - `style`: Any `text` named arguments (like: `fill`, `size`, `font`, etc.; see https://typst.app/docs/reference/text/text/). The **text-style** of the item body.
    - See also `#default-enum-list.args`.
  - `whole`, `outer`, `inner`: Format the item body.
    - `whole`: Wrap the entire `enum` or `list`
    - `outer`: Wrap the item (including the label)
    - `inner`: Wrap the item body (excluding the label)
      - Example:

```
#show: el.default-enum-list.with(body-format: (
  inner: (
    stroke: (red, blue, auto),
    inset: (5pt,),
  ),
  outer: (
    stroke: (yellow, green, auto),
    inset: (5pt,),
  ),
  whole: (
```

```
    stroke: (purple, maroon, auto),
    inset: (5pt,),
  ),
))
+ Item one
  + Sub Item one
  + Sub Item two
+ Item two
```

- **Note**: If `whole`, `outer`, and `inner` are omitted, the default is `outer`.
- `whole`, `outer`, or `inner` can be taken `dictionary` with the following keys:
  - `format` (`none`, `function`, `array`): The format of the item body.
    - See the `format` in `#default-enum-list.label-format`.
    - **Note**: Use the `format` function (since ver0.3.0) to replace the `item-format` (in ver0.2.0) parameter .
  - `stroke`, `radius`, `outset`, `fill`, `inset`, `clip`, `breakable`, `sticky`, `width`: The **border** of the item body, which are the same as the `block` named elements (note: `width` only works for `whole`).
    - `auto`: The default value of the argument.
    - the values of the corresponding named arguments of `block` that can be set.
    - `array` (level-property), `function` (level-property):
      - See the **border** in `#default-enum-list.label-format`.

Argument

`⟨auto-resuming⟩: none` `none` | `auto` | `bool` | `array` | `function`

Relate to the feature **Resuming Enum**.

- `none`: Disable this feature.
- `auto`: Enable this feature. In this case, the following methods `#resume`, `#resume-label`, `#resume-list`, `#auto-resume-enum`, `#isolated-resume-enum` can be used.
  - Use the method `#resume` to continue using the enum numbers from the previous enum at the same level.
    - Example:

```
#show: el.default-enum-list.with(auto-resuming: auto)
+ Item one
+ Item two
Paragraph
#el.resume()
+ new list item one
```

  - Or use `resume[/*content*/]` to explicitly continue using the enum numbers from the previous level (especially in ambiguous cases) and treat the `[/*content*/]` as new `enums`.
    - Example:

```
 #show: el.default-enum-list.with(auto-resuming: auto)
 + Item one
```

```
3   + Item two
4     + Sub item one
5   Paragraph
6   #el.resume[
7     + New list item // resume
8       + Sub item // not resume
9     + New list item
10  ]
11  + New list item one // not resume
```

- Use the method `#resume-label` to label the enum you want to resume, and then use `#resume-list` in the desired enum to continue using the labelled enum numbers.
  - If you use the following in your document:

```
1  #show: el.config.ref-resume
```

    then you can use `@some-label` instead of `resume-list(<some-label>)`.
  - Example:

```
1  #show: el.default-enum-list.with(auto-resuming: auto)
2  + Item one
3  + Item two
4    + Sub item one
5    + Sub item Two
6    + Sub item Three #el.resume-label(<resume:sub>) // The enum you want to resume
7  Paragraph
8  #el.resume-list(<resume:sub>)
9  + New Item // resume
```

- Use the method `auto-resume-enum(auto-resuming: true)[...]`, where all enum numbers within `[...]` will continue from the previous ones.
  - See the method `#auto-resume-enum` for the details of the argument `auto-resuming` (`bool`, `array`, `function`).
  - Example:

```
1   #show: el.default-enum-list.with(auto-resuming: auto)
2   + Item one
3   + Item two
4     + Sub item
5   Paragraph
6   #el.auto-resume-enum(auto-resuming: true)[
7     + New list item
8       + Sub item
9       + Sub item
10    + New list item
11      + Sub item // resume
```

```
12    Paragraph
13    + New list item // resume
14  ]
15  + New list item // not resume
```

  - The method `isolated-resume-enum[...]` allows the `enums` in `[...]` to be treated as new enums with independent numbering, without affecting other enums out of the `[...]`.
- `bool`: If `auto-resuming` is set to `true`, then all enum numbers will continue from the previous ones.
- `array` (level-property): The elements are `bool`. The `level`-th level will be set to the corresponding value of the array at position `level - 1`; now `true` means the enum numbers at the `level`-th level will continue from the previous ones.
  - If the last element of the array is `LOOP`, the values in the array will be used cyclically;
  - otherwise, the last value of the array will be used for residual levels.
- `function` (level-property): The return value will be used for each level.
  - The function form: `it => bool`
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.elem-tag`: The tag of the `enum` or `list`.
    - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
    - `it.hanging-type`: The hanging type of the current level.
  - Example: Using `elem-tag`, specify which `enum` continues to use the previous enum's numbers.

```
1   #show: el.default-enum-list.with(auto-resuming: it => {
2     if (it.elem-tag == "resume") { true } else { false }
3   })
4   + Item one
5   + Item two
6     + Sub item one
7     + Sub item two
8   New paragraph
9   + Item one // not resume
10    + #el.item(elem-tag: "resume") Sub item // resume
```

- **Note**: If `auto-resuming` is not `none` or `false`, the the methold `default-enum-list(auto-resuming: ...)` can not be used nested.
  - For large documents like books or articles, set `auto-resuming` to `auto` in the required sublists and use it with methods like `resume`, `resume-label`, `resume-list`, or `auto-resume-enum`.
  - For small documents like exams, exercises, or CVs, if need to resume enum numbers throughout the document, then use the following at the beginning:
    - `#show : el.default-enum-list.with(auto-resuming: true),`

or if specify certain enums to continue using the number of the previous enum (with the help of `item(elem-tag: ...)`)

– `#show : el.default-enum-list.with(auto-resuming: it => /* ... */)`.

Argument

`⟨auto-label-width⟩: none` `none` | `auto` | `str` | `array` | `function`

Used to share the same maximum label width for different `enums` and `lists`.

- `none`: Disable this feature.
- `auto`: Enable this feature. In this case, use the method `auto-label-item[...]` to share the same maximum label width for `enum` or `list` in `[...]`.
  - Example: In the following, `List A` and `List B` share the same maximum label width.

```
#show: el.default-enum-list.with(auto-label-width: auto)
+ Item one
#el.auto-label-item(form: "each")[
   + Item two // List A
   Paragraph
   + New Item one // List B
   10. Item
]
+ New enum
```

  - See the method `#auto-label-item` for the details of the argument `form` (Similar to `auto-label-width`).
- `"each"`: The maximum widths of `enum`'s labels and `list`'s labels are considered separately.
  - Example: In the following, `List A` and `List B` share the same maximum label width, `Enum A` and `Enum B` share the same maximum label width.

```
#show: el.default-enum-list.with(auto-label-width: "each")
- item // List A
- item
+ item // Enum A
10. item
- #el.item([#square(width: 1em)]) item // List B
Paragraph
100. item // Enum B
```

- `"list"`: Only the maximum widths of `list`'s labels are considered.
- `"enum"`: Only the maximum widths of `enum`'s labels are considered.
- `"all"`: The maximum widths of `enum`'s labels and `list`'s labels are both considered.
- `array` (level-property): The elements are `none`, `auto`, `"each"`, `"list"`, `"enum"`, `"all"`. The `level`-th level will be set to the corresponding value of the array at position `level - 1`.
  - For array elements, `"each"` is equivalent to `auto`.
  - If the last element of the array is `LOOP`, the values in the array will be used cyclically;
  - otherwise, the last value of the array will be used for residual levels.
- `function` (level-property): The return value will be used for each level.
  - The function form: `it => none | auto | "each" | "list" | "enum" | "all"`

- For returned values, `"each"` is equivalent to `auto`.
- Access the following values available on `it`:
  - `it.level`: The level of the item, starting from 1.
  - `it.n-last`: The index of the last item in the current level.
  - `it.elem-tag`: The tag of the `enum` or `list`.
  - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
  - `it.hanging-type`: The hanging type of the current level.
- Example: In the following, for the same level of `enums` or `lists`, if they are marked as "auto-label" (with the help of `el.item(elem-tag: ...)`), then they will share the same maximum label width.

```
#show: el.default-enum-list.with(auto-label-width: it => {
  if it.elem-tag == "auto-label" { auto }
})
+ Item one
  + #el.item(elem-tag: "auto-label") Sub Item one
  + Sub item two
+ Item one
  10. #el.item(elem-tag: "auto-label") Sub Item one
  + Sub item two
```

Argument

`⟨auto-base-level⟩: true` `bool`

By default (since ver0.3.0), `auto-base-level` is set to `true`, meaning each `default-*` (`inline-*`, `paragraph-*`) method now maintains its own level counter, starting from 1. This behavior almost only affects the display of `enum.number` (by `enum.numbering`).

- If want to all `enum` use the same counter (as in ver0.2.x), set `auto-base-level` to `false` (but then at least three layout iterations are required).

Argument

`⟨step⟩: auto` `auto` | `int` | `function` | `array`

Used to set the step size for `enum`'s number.

- Usage:

```
#show: el.default-enum-list.with(step: 2) // the next enum's number is 2 greater
than the current enum's number.
```

- `auto`: The step size is `1` if `enum.reversed` is `false`, otherwise it is `-1`.
- `int`: The step size is the specified value.
- `function` (base-property): The next enum's number is the return value of the function. If the return value is `auto` or `none` (usually used as an initial value), then the step size is `1` if `enum.reversed` is `false`, otherwise it is `-1`.
  - The function form: `(..nums) => int | auto | none`
  - Here is an example to make enum's number like factorial:

```
#let factorial-step = (..nums) => {
  let numbers = nums.pos()
  let n = numbers.len()
  if n >= 2 {
    return numbers.at(n - 2) + numbers.at(n - 1)
  }
}
#show: el.default-enum-list.with(step: (factorial-step,))
1. #lorem(2)
1. #lorem(2)
+ #lorem(2)
+ #lorem(2)
+ #lorem(2)
```

- `array` (level-property): The elements are `int`, `function`, `auto` and `none`. The element at position `level - 1` sets the step size for the `level`-th level.
  - `int`: The step size is the specified value.
  - `auto` or `none`: The step size is `1` if `enum.reversed` is `false`, otherwise it is `-1`.
  - If the last element of the array is `LOOP`, the values in the array will be used cyclically;
  - otherwise, the last value of the array will be used for residual levels.
- `function` (level-property): The return value will be used for each level.
  - The function form: `it => int | auto | function`
  - If the returned value is a `function`, it should be the above form of `function` (base-property).
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.elem-tag`: The tag of the current enum or list.
    - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
    - `it.hanging-type`: The hanging type of the current level.
  - Example: for all odd levels, the `enum` numbers are reversed.

```
#show: el.default-enum-list.with(step: it => {
  if calc.odd(it.level) {
    (..nums) => {
      let numbers = nums.pos()
      let n = numbers.len() // 1-based count of previous numbers
      if n == 0 { it.n-last } else { numbers.at(n - 1) - 1 }
    }
  } else { auto }
})
```

Argument

`(ref-numbering): none` — `function` | `str` | `none`

How to number the `enum`'s reference (default: `none`, determined by `enum.numbering` or `#config.ref.numbering`).

- See also `enum.numbering`.

Argument

`(supplement): auto` `auto` | `content` | `dictionary` | `function` | `array`

Used to set supplementary content when referencing enum labels.

- Note: This is independent of the `#config.ref.supplement` setting, meaning the supplement content and the referenced enum or list label will be passed as a whole to `#config.ref.supplement`.
- Usage:

```
#show: el.default-enum-list.with(supplement: [Item])
#show: el.default-enum-list.with(supplement: (prefix: [第], suffix: [项]))
#show: el.default-enum-list.with(supplement: it =>
[#numbering(heading.numbering, ..counter(heading).at(it.target)).#it.body])
```

- `auto`: No supplementary content will be added.
- `content`: Uses `content` as supplementary content. The effect is that when referencing `enum` labels, `content` is added before the label.
- `dictionary`: The keys are: `prefix`, `suffix`, with values of `content`. The effect is that when referencing `enum` labels, `prefix` content is added before the label, and `suffix` content is added after the label.
- `array` (level-property): Sets `supplement` by level. The elements are `content`, `dictionary`, `auto`, `array`.
  - See `#default-enum-list.indent`.
- `function` (level-property): The return value will be used for each level and each item.
  - The function form: `it => content`
  - Access the following values available on `it`:
    - `it.body`: The referenced `enum` labels.
    - `it.level`: The level of the item, starting from 1.
    - `it.n`: The index of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.tag` (since ver0.3.0): The tag of the item.
    - `it.elem-tag` (since ver0.3.0): The tag of the `enum` or `list`.
    - `it.e` (since ver0.3.0): The field of the current construct element (`enum` or `list`), but excluding the `children`.
    - `it.hanging-type` (since ver0.3.0): The hanging type of the current level.
    - `it.number` (since ver0.3.0): The number of the current label if it is a `enum`.
    - `it.target` (since ver0.3.0): The target of the referenced label.
    - `it.numbers` (since ver0.3.0): The full numbers of the referenced item (including the parent `enum`'s numbers).

Argument

`(checklist): false` `bool` | `array`

Whether to use checklist (default: `false`). If set to `true` (since ver0.3.0), then configure checklist-related features using the method `#config.checklist`.

- Usage:

```
#show: el.default-enum-list.with(checklist: true)
#show: el.config.checklist.with(
  // configure
)
```

  See the method `#config.checklist` for more details.
- `array` (level-property): The elements are `bool`, each level of the item will be set to the corresponding value of the array at position `level - 1`; now `true` means to use checklist for the current level.
  - If the last element of the array is `LOOP`, the values in the array will be used cyclically;
  - otherwise, the last value of the array will be used for residual levels.

Argument

⟨enum-config⟩: (:) `dictionary`

Configure `enum` in `doc`.

- The keys are:
  - any named arguments of the function `text` (like: `fill`, `size`, `font`, etc.)
  - `indent`
  - `body-indent`
  - `label-indent`
  - `label-inset`
  - `body-margin`
  - `hanging-indent`
  - `line-indent`
  - `first-line-inset`
  - `whole-spacing`
  - `item-spacing`
  - `tight-mode`
  - `tight-item-mode`
  - `label-width`
  - `label-baseline`
  - `label-format`
  - `label-align`
  - `body-format`
  - `step`
  - `ref-numbering`
  - `supplement`
  - `description-config`
  - `hanging-type`
- Rules: If both `*-enum-list` and `enum-config` have the same property set, the settings in `enum-config` take precedence.
  - For `label-format` and `body-format`, the settings are folded.
- The level used in `enum-config` is the relative level of `enum` (i.e., ignore `list`).

- The values allowed to be passed in the keys are consistent with the corresponding parameters in the method `default-enum-list`.

Argument

`⟨list-config⟩: (:)` `dictionary`

Configure `list` in `doc`.

- See `#default-enum-list.enum-config`.

Argument

`⟨description-config⟩: auto` `dictionary` | `none`

Configure the description list.

- Using the syntax:
  - `+ / term: description`
  - `- / term: description`
- The description list is like the following:

```
[marker] / [number] [term-body] [description-body]
term-body: [prefix] [term] [separator.prefix]
description-body: [separator.suffix] [description] [suffix]
```

Note: Usually, when the `term` and `description` are paragraphs, this will work as expected.

- Provide four styles: horizontal description list, hanging description list, stacked description list, plain description list.
- Support checklist style for description list (requires setting `checklist` to `true` in `description-config` and `#default-enum-list.checklist`).
- Usage:

```
#show: el.default-enum-list.with(description-config: {
  style: "horizontal",
  term-format: (strong,),
})
- / Windows: Windows is a family of graphical operating systems, developed by Microsoft.
- / macOS: macOS is a series of graphical operating systems developed and marketed by Apple Inc.
- / Linux: Linux is a family of free and open-source operating systems.
+ / Windows: Windows is a family of graphical operating systems, developed by Microsoft.
+ / macOS: macOS is a series of graphical operating systems developed and marketed by Apple Inc.
+ / Linux: Linux is a family of free and open-source operating systems.
```

- `none`: Disable the description list.
- `dictionary`: The keys are:
  - `style` (`auto`, `string`, `array`, `function`): The style of the description list.
    - `"plain"` == `auto`: Plain description list. The `term-body` and `description-body` are displayed in a single paragraph.

- `"hanging"`: Hanging description list. The `term-body` and `description-body` are displayed in a single paragraph, but `hanging-indent` is `2em` in default. The same as native Typst `terms`.
- `"stacked"`: Stacked description list. The `term-body` and `description-body` are displayed as two start-aligned paragraphs, respectively.
- `"horizontal"`: Horizontal description list. The `term-body` and `description-body` are displayed in a single paragraph, but `description-body` will be indented and aligned according to `term-width` (by default, using the maximum width of the `term-body`).
- `array` (level-property): The elements are `auto`, `"plain"`, `"hanging"`, `"stacked"`, `"horizontal"`, or `array`. The element at the position `level - 1` will be used for the `level`-th level.
  - If the last element of the array is `LOOP`, the values in the array will be used cyclically;
  - otherwise, the last value of the array will be used for residual levels.
  - The elements in the array can also be an array, where the element at position `n - 1` applies to the item of index `n`.
- `function` (level-property): The return value will be used for each level and each item.
  - The function form: `it => auto | "plain" | "hanging" | "stacked" | "horizontal" | array`
  - If the returned value is a `array`, then the elements in the array will be used for each item.
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n`: The index of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.tag`: The tag of the current item.
    - `it.elem-tag`: The tag of the current enum or list.
    - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
    - `it.hanging-type`: The hanging type of the current level.
    - `it.marker`: The marker of the current label if it is a `list`.
    - `it.number`: The number of the current label if it is a `enum`.
    - `it.term-width` (dictionary): The width info of the current label.
      - `it.term-width.max`: The maximum width of the `term-body` in the current level and in the current style.
      - `it.term-width.current`: The current width of the `term-body`.
- `term-width` (`auto`, `relative`, `string`, `dictionary`, `array`, `function`): The width of the `term-body`.
  - `auto`: In default,
    - for "horizontal" style, the `term-width` uses the maximum width of the `term-body` in the current level and in the "horizontal" style. That is, use `(amount: "max", style: "native")`.
    - for other styles, the `term-width` uses the real width of the `term-body`. That is, use `(amount: auto, style: "native")`.
  - `relative`, `"max"`, `dictionary`, `array` (level-property): See `#default-enum-list.label-width`.

    - The maximum width of the `term-body` in the current level is calculated for different styles.
  - `function` (level-property): The return value will be used for each level and each item.
    - The function form: `it => auto | relative | string | dictionary | array`.
    - Access the following values available on `it`:
      - `it.level`: The level of the item, starting from 1.
      - `it.n`: The index of the item, starting from 1.
      - `it.n-last`: The index of the last item in the current level.
      - `it.tag`: The tag of the current item.
      - `it.elem-tag`: The tag of the current enum or list.
      - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
      - `it.hanging-type`: The hanging type of the current level.
      - `it.marker`: The marker of the current label if it is a `list`.
      - `it.number`: The number of the current label if it is a `enum`.
      - `it.style`: The description list style of the current item.
      - `it.max-width`: The maximum width of the `term-body` in the current level and in the current style.
      - `it.width`: The width of the `term-body` of the current item.
- `term-align` (`auto`, `alignment`, `array`, `function`): The alignment of the `term-body`.
  - **Note**: The `term-align` works for the following cases.
    - The "horizontal" and "stacked" styles;
    - For "hanging" and "plain" styles, the `term-width` is not `auto` (that is, the real width of the `term-body` is not equal to the rendered width).
  - `auto` == `start`.
  - `alignment`: The alignment of the `term-body`.
  - `array` (level-property): The elements are `auto`, `alignment`, or `array`.
    - See `style`.
  - `function` (level-property): The return value will be used for each level and each item.
    - The function form: `it => auto | alignment | array`.
    - Access the following values available on `it`: See `separator`.
- `separator` (`auto`, `content`, `dictionary`, `array`, `function`): The separator between `term-body` and `description`.
  - **Note**: `separator` contains two parts: `prefix` and `suffix`.
    - In default, `separator.prefix` is added after `term` and `separator.suffix` is added before `description`.
  - `auto`: The `separator.prefix` is `[]` for "stacked" style and `[#h(.6em)#h(0pt, weak: true)]` for other styles. `separator.suffix` is `[]`.
  - `content`: The `separator.prefix` is `separator` and the `separator.suffix` is `[]`.
  - `dictionary`:
    - The keys are `prefix` and `suffix`, with `content` values.
      - In default, `prefix` is `[]` and `suffix` is `[]`.
    - Now, `separator.prefix` is `prefix` and `separator.suffix` is `suffix`.
  - `array` (level-property): The elements are `auto`, `content`, `dictionary`, or `array`.
    - See `style`.
  - `function` (level-property): The return value will be used for each level and each item.

    - The function form: `it => auto | content | dictionary | array`.
    - Access the following values available on `it`:
      - `it.style`: The description list style of the current item.
      - For others, see `style`.
- `prefix` (`content`, `array`, `function`): The prefix of the `term`. Default is `[]`. The `prefix` is added before `term`.
  - `array` (level-property): The elements are `content` or `array`.
    - See `style`.
  - `function` (level-property):
    - The function form: `it => content | array`.
    - Access the following values available on `it`: See `separator`.
- `suffix` (`content`, `array`, `function`): The suffix of the `description`. Default is `[]`. The `suffix` is added after `description`.
  - **Suggestion**: Using it, control the description list's below spacing (like `block(above: 0pt, below: len)`), or add special symbols, and so on.
- `hanging-indent` (`auto`, `length`, `array`, `function`): The hanging indent of the description list.
  - `auto`: The `hanging-indent` is `2em` for "hanging" style, and `0pt` for other styles.
  - `length`: The value of hanging indent.
  - `array` (level-property): The elements are `auto`, `length`, or `array`.
    - See `style`.
  - `function` (level-property): The return value will be used for each level and each item.
    - The function form: `it => auto | length | array`.
    - Access the following values available on `it`: See `separator`.
- `term-format` (`none`, `function`, `array`): The format of the `term`.
  - `none`: Do not apply.
  - `function` (base-property): Apply the function to the `term` (not the `term-body`).
    - The function form: `body => content`.
    - The `function` should be wrapped in an `array`, like: `(strong, )`.
    - Example: In the following, the `Terms` will be colored by `red` and formatted by `strong`.

```
#let term-f = body => {
  show: strong
  set text(fill: red)
  body
}
#show: el.default-enum-list.with(description-config: (
  term-format: (term-f,),
))
/ Terms: #lorem(10)
```

  - `array` (level-property): The elements are `none`, `function` or `array`.
    - See `style`.
    - Example: In the following, the `Terms` and `Sub Terms` will be formatted by `strong` and `emph`, respectively.

```
#show: el.default-enum-list.with(description-config: (
```

```
  term-format: (strong, emph),
))
- / Terms: #lorem(10)
  - / Sub terms: #lorem(10)
```

– `function` (level-property): The return value will be used for each level and each item.
  - The function form: `it => content | array`.
  - Access the following values available on `it`:
    - `it.body`: The `term` of the current item.
    - `it.level`: The level of the item, starting from 1.
    - `it.n`: The index of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.tag`: The tag of the current item.
    - `it.elem-tag`: The tag of the current enum or list.
    - `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
    - `it.hanging-type`: The hanging type of the current level.
    - `it.marker`: The marker of the current label if it is a `list`.
    - `it.number`: The number of the current label if it is a `enum`.
    - `it.term-width` (dictionary): The width info of the current label.
      – `it.term-width.max`: The maximum width of the `term-body` in the current level and in the current style.
      – `it.term-width.current`: The current width of the `term-body`.
  - Example: Format `term` for different styles.

```
#let desc-style = style => (description-config: (style: style), for-elem: true)
#show: el.default-enum-list.with(description-config: (
  term-format: it => {
    let format(body) = if it.style == "stacked" {
      strong(body)
      block(above: 0pt, below: par.leading)
    } else { emph(body) }
    format(it.body)
  },
))
- #el.item(..desc-style("stacked"))
  / CPU: #lorem(10)
- / GPU: #lorem(10)
Paragraph
- #el.item(..desc-style("horizontal"))
  / CPU: #lorem(10)
- / GPU: #lorem(10)
```

- `description-format` (`function`, `array`): The format of the `description`.
  – See `term-format`.
- `checklist` (`bool`, `array`, `function`): Whether to use checklist style description list.

  - **Note**: In order to enable checklist style, the `checklist` in `default-*` (or `paragraph-*`) must be `true`.
  - Usage:

```
#show: el.default-enum-list.with(checklist: true, description-config:
(checklist: true))
- / [ ] : #lorem(2)
+ / [-] : #lorem(2)
```

  - `true`: Enable checklist style.
  - `false`: Disable checklist style.
  - `array` (level-property): The elements are `bool` or `array`.
    - See `style`.
  - `function` (level-property): The return value will be used for each level and each item.
    - The function form: `it => bool | array`.
    - Access the following values available on `it`: See `separator`.
- `enable-first-par` (`auto`, `bool`, `array`, `function`): Whether to make all the paragraphs in the description list like the first paragraph.
  - `auto`: For `stacked` style, the `enable-first-par` is `true`, and for other styles, the `enable-first-par` is `false`.
  - `bool`: Whether to enable the first paragraph.
  - `array`: The elements are `bool` or `array`.
    - See `style`.
  - `function` (level-property): The return value will be used for each level and each item.
    - The function form: `it => bool | array`.
    - Access the following values available on `it`: See `separator`.
- `format` (`none`, `function`, `array`): The format of the description list.
  - `none`: Do not apply.
  - If the default style is not suitable, you can use `format` to customize the style.
  - `function` (base-property): Apply the function to the whole description list.
    - The function form: `body => content`.
    - The `function` should be wrapped in an `array`, like: `(strong, )`.
    - Example: In the following, the whole description list will be colored by `red`.

```
#show: el.default-enum-list.with(description-config: (
  format: (text.with(fill: red),),
))
- / CPU: #lorem(10)
- / GPU: #lorem(10)
```

  - `array` (level-property): The elements are `none`, `function` or `array`. See `term-format`.
  - `function` (level-property): The return value will be used for each level and each item.
    - The function form: `it => content`.
    - Access the following values available on `it`:
      - `it.body`: The current description list.
      - `it.level`: The level of the item, starting from 1.
      - `it.n`: The index of the item, starting from 1.

- `it.n-last`: The index of the last item in the current level.
- `it.tag`: The tag of the current item.
- `it.elem-tag`: The tag of the current enum or list.
- `it.e`: The field of the current construct element (`enum` or `list`), but excluding the `children`.
- `it.hanging-type`: The hanging type of the current level.
- `it.marker`: The marker of the current label if it is a `list`.
- `it.number`: The number of the current label if it is a `enum`.
- `it.term`, `it.description`: The `term` and the `description` of the current item.
- `it.separator`: The `separator`.
- `it.prefix`, `it.suffix`: The `prefix` and `suffix` of the current item.
- `it.hanging-indent`: The hanging indent of the current item.
- `it.style`: The current description list style.
- `it.enable-first-par`: The `enable-first-par` of the current item.
- `it.term-align`: The `term-align` of the current item.
- `it.term-width` (dictionary): The width info of the current label.
  - `it.term-width.max`: The maximum width of the `term-body` in the current level and in the current style.
  - `it.term-width.current`: The current width of the `term-body`.
- `it.process-par`: Usually use like `show: process-par.with(enable-strong-par: it.enable-first-par)`. Used for internal processing of paragraph models. Advanced usage.
- `it.dir`: Current text direction.

Argument

`..(args)` `arguments`

Any `text` named arguments (like: `fill`, `size`, `font`, etc.; see https://typst.app/docs/reference/text/text/). Format the current item label.

- Usage:

```
#show: el.default-enum-list.with(fill: red) // All labels are colored red.
#show: el.default-enum-list.with(size: (2em, 1.2em, auto)) // The label's size for first level is 2em, the label's size for second level is 1.2em, and the label's size for other levels is `auto` (use the current `text.size`).
#show: el.default-enum-list.with(fill: it => {
  if it.e.tight { // tight enum or list use `red` for label
    red
  } else { // otherwise, use `black`
    black
  }
})
```

- **Note**: If some named argument of `text` allows being set as an `array` (base-property), in order to apply it to the current level, you need to further wrap it in an `array`.
  - Example: In the following, the first level of labels use the font `(name: "DejaVu Sans Mono", covers: "latin-in-cjk")`, and other levels use the font `"Noto Serif CJK SC"`.

```
#let label-font = ((name: "DejaVu Sans Mono", covers: "latin-in-cjk"), "Noto Serif CJK SC")
#show: el.default-enum-list.with(font: label-font)
#set enum(numbering: " (1) ")
+ #lorem(2)
+ #lorem(2)
  + #lorem(2)
  + #lorem(2)
```

- Example: In the following, for each level, the first index of label uses the font `(name: "DejaVu Sans Mono", covers: "latin-in-cjk")`, and other indices use the font `"Noto Serif CJK SC"`.

```
#let label-font = ((name: "DejaVu Sans Mono", covers: "latin-in-cjk"), "Noto Serif CJK SC")
#show: el.default-enum-list.with(font: (label-font,))
#set enum(numbering: " (1) ")
+ #lorem(2)
+ #lorem(2)
  + #lorem(2)
  + #lorem(2)
```

- Example: In the following, all the levels of labels use the font `((name: "DejaVu Sans Mono", covers: "latin-in-cjk"), "Noto Serif CJK SC")`.

```
#let label-font = ((name: "DejaVu Sans Mono", covers: "latin-in-cjk"), "Noto Serif CJK SC")
#show: el.default-enum-list.with(font: ((label-font,),))
#set enum(numbering: " (1) ")
+ #lorem(2)
+ #lorem(2)
  + #lorem(2)
  + #lorem(2)
```

- `array`: All the named arguments of `text` are supported for `array`. The `level`-th of the `enum` or `list` will be set to the corresponding value of the array at position `level - 1`.
  - The elements of the array are the values of the named arguments of `text` that can be set.
  - If the last element of the array is `LOOP`, the values in the array will be used cyclically;
  - otherwise, the last value of the array will be used for residual levels.
  - Then elements of the array can be `array`, meaning the element at position `n-1` will apply to the `n`-th index of the label for the current level.
- `function`: The return value will be used for each level and each item.
  - The function form: `it => value | array`, where `value` is any value accepted by the corresponding `text` named argument.
  - If the return value is `array`, the element at position `n-1` will apply to the `n`-th index of the label for the current level.
  - Access the following values available on `it`:

- `it.level`: The level of the item, starting from 1.
- `it.n`: The index of the item, starting from 1.
- `it.n-last`: The index of the last item in the current level.
- `it.tag` (since ver0.3.0): The tag of the current item.
- `it.elem-tag` (since ver0.3.0): The tag of the current enum or list.
- `it.e` (since ver0.3.0): The field of the current construct element (`enum` or `list`), but excluding the `children`.
- `it.hanging-type` (since ver0.3.0): The hanging type of the current level.
- `it.marker` (since ver0.3.0): The marker of the current label if it is a `list`.
- `it.number` (since ver0.3.0): The number of the current label if it is a `enum`.

```
#default-list(
  (doc),
  (indent): auto,
  (body-indent): auto,
  (label-indent): auto,
  (label-inset): auto,
  (body-margin): auto,
  (hanging-type): "classic",
  (line-indent): auto,
  (first-line-inset): auto,
  (hanging-indent): auto,
  (whole-spacing): auto,
  (item-spacing): auto,
  (tight-mode): auto,
  (tight-item-mode): auto,
  (label-width): auto,
  (label-align): auto,
  (label-baseline): auto,
  (label-format): none,
  (body-format): none,
  (auto-resuming): none,
  (auto-label-width): none,
  (auto-base-level): true,
  (supplement): auto,
  (checklist): false,
  (description-config): auto,
  ..(args)
) → content
```

Configures default styling for `list`.

See `#default-enum-list`.

```
#inline-enum(
  (doc),
  (indent): auto,
  (body-indent): auto,
  (label-indent): auto,
  (label-inset): auto,
  (line-indent): auto,
  (first-line-inset): auto,
  (hanging-indent): auto,
  (whole-spacing): auto,
  (item-spacing): auto,
  (tight-mode): auto,
  (tight-item-mode): auto,
  (label-width): auto,
  (label-align): auto,
  (label-format): none,
  (body-format): none,
  (auto-base-level): true,
  (step): auto,
  (ref-numbering): none,
  (supplement): auto,
  (checklist): false,
  (description-config): auto,
  ..(args)
) → content
```

Inline layout for `enum`

See `#default-enum-list` and `#inline-enum-list`.

```
#inline-enum-list(
  (doc),
  (indent): auto,
  (body-indent): auto,
  (label-indent): auto,
  (label-inset): auto,
  (line-indent): auto,
  (first-line-inset): auto,
  (hanging-indent): auto,
  (whole-spacing): auto,
  (item-spacing): auto,
  (tight-mode): auto,
  (tight-item-mode): auto,
  (label-width): auto,
  (label-align): auto,
  (label-format): none,
  (body-format): none,
  (auto-base-level): true,
  (step): auto,
  (ref-numbering): none,
  (supplement): auto,
  (checklist): false,
  (enum-config): (:),
  (list-config): (:),
  (description-config): auto,
  ..(args)
) → content
```

Inline layout for `enum` and `list`. That is, it does not use block elements and instead interprets the list content as paragraphs.

- In particular, any block elements appearing in the list are laid out flush left.
- Content at every level of the list spans the full width of the parent container.
- Because of this paragraph behavior, some features of the method `default-enum-list` no longer take effect:
  - `label-baseline` is not supported.
  - `Border` of the `body-format` is not supported.
  - `body-margin` is not supported.

See #default-enum-list.

```
#inline-list(
  ⟨doc⟩,
  ⟨indent⟩: auto,
  ⟨body-indent⟩: auto,
  ⟨label-indent⟩: auto,
  ⟨label-inset⟩: auto,
  ⟨line-indent⟩: auto,
  ⟨first-line-inset⟩: auto,
  ⟨hanging-indent⟩: auto,
  ⟨whole-spacing⟩: auto,
  ⟨item-spacing⟩: auto,
  ⟨tight-mode⟩: auto,
  ⟨tight-item-mode⟩: auto,
  ⟨label-width⟩: auto,
  ⟨label-align⟩: auto,
  ⟨label-format⟩: none,
  ⟨body-format⟩: none,
  ⟨auto-base-level⟩: true,
  ⟨supplement⟩: auto,
  ⟨checklist⟩: false,
  ⟨description-config⟩: auto,
  ..⟨args⟩
) → content
```

Inline layout for `list`.

See `#default-enum-list` and `#inline-enum-list`.

```
#paragraph-enum(
  (doc),
  (indent): auto,
  (body-indent): auto,
  (label-indent): auto,
  (label-inset): auto,
  (body-margin): auto,
  (line-indent): auto,
  (first-line-inset): auto,
  (hanging-indent): auto,
  (whole-spacing): auto,
  (item-spacing): auto,
  (tight-mode): auto,
  (tight-item-mode): auto,
  (label-width): auto,
  (label-align): auto,
  (label-baseline): auto,
  (label-format): none,
  (body-format): none,
  (auto-resuming): none,
  (auto-label-width): none,
  (auto-base-level): true,
  (step): auto,
  (ref-numbering): none,
  (supplement): auto,
  (checklist): false,
  (description-config): auto,
  ..(args)
) → content
```

Configures paragraph styling for `enum`.

See `#default-enum-list`.

```
#paragraph-enum-list(
  (doc),
  (indent): auto,
  (body-indent): auto,
  (label-indent): auto,
  (label-inset): auto,
  (body-margin): auto,
  (line-indent): auto,
  (first-line-inset): auto,
  (hanging-indent): auto,
  (whole-spacing): auto,
  (item-spacing): auto,
  (tight-mode): auto,
  (tight-item-mode): auto,
  (label-width): auto,
  (label-align): auto,
  (label-baseline): auto,
  (label-format): none,
  (body-format): none,
  (auto-resuming): none,
  (auto-label-width): none,
  (auto-base-level): true,
  (step): auto,
  (ref-numbering): none,
  (supplement): auto,
  (checklist): false,
  (enum-config): (:),
  (list-config): (:),
  (description-config): auto,
  ..(args)
) → content
```

Configures paragraph styling for `enum` and `list`.

See `#default-enum-list`.

```
#paragraph-list(
  (doc),
  (indent): auto,
  (body-indent): auto,
  (label-indent): auto,
  (label-inset): auto,
  (body-margin): auto,
  (line-indent): auto,
  (first-line-inset): auto,
  (hanging-indent): auto,
  (whole-spacing): auto,
  (item-spacing): auto,
  (tight-mode): auto,
  (tight-item-mode): auto,
  (label-width): auto,
  (label-align): auto,
  (label-baseline): auto,
  (label-format): none,
  (body-format): none,
  (auto-resuming): none,
  (auto-label-width): none,
  (auto-base-level): true,
  (supplement): auto,
  (checklist): false,
  (description-config): auto,
  ..(args)
) → content
```

Configures paragraph styling for `list`.

See `#default-enum-list`.

`#set-default(..(args)) → content`

Configures default styling for `enum` and `list`. Support `show-rule`.

- Usage:

```
1 #show: el.set-default()
2 #show: el.set-default(fill: red)
3 #show: el.set-default(body-format: (strong,))
```

See `#default-enum-list`.

`#set-paragraph(..(args)) → content`

Configures paragraph styling for `enum` and `list`.

See `#default-enum-list` and `#set-default`.

III.2. Tool methods

`#auto-label-item`	`#isolated-resume-enum`	`#resume-label`
`#auto-resume-enum`	`#item`	`#resume-list`
`#elabel`	`#resume`	

#auto-label-item((form): auto)[doc] → `content`

Used to share the same maximum label width for different `enums` and `lists` in `doc`.

- **Note**: The method `auto-label-item` cannot be nested.
- **Note**: In order to make `auto-label-item` work, should set the argument `auto-label-width` in ***-enum-list** (***-enum**) to `auto`.

Argument

⟨doc⟩ `content`

The `enum` or `list` in the `doc` to process

Argument

⟨form⟩: `auto` `none` | `auto` | `str` | `array` | `function`

- `none`: No processing.
- `"each" == auto`: The maximum widths of `enum`'s labels and `list`'s labels are considered separately.
- `"enum"`: Only the maximum widths of `enum`'s labels are considered.
- `"list"`: Only the maximum widths of `list`'s labels are considered.
- `"all"`: The maximum widths of `enum`'s labels and `list`'s labels are both considered.
- `array` (level-property): The elements are `none`, `auto`, `"each"`, `"list"`, `"enum"`, `"all"`. The `level`-th level will be set to the corresponding value of the array at position `level - 1`.
  - **Note**: The level in `doc` uses the relative or absolute level depending on which ***-enum-list** or ***-enum** is used in the document; but starting from **1**.
  - `"each"` is equivalent to `auto`.
  - If the last element of the array is `LOOP`, the values in the array will be used cyclically;
  - otherwise, the last value of the array will be used for residual levels.
- `function` (level-property):
  - The function form: `it => none | auto | "each" | "list" | "enum" | "all"`
  - Access the following values available on `it`:
    - `it.level`: The level of the item, starting from 1.
    - `it.n-last`: The index of the last item in the current level.
    - `it.elem-tag`: The tag of the `enum` or `list`.
    - `it.e`: The construct function (`enum` or `list`) of the item.
    - `it.hanging-type`: The hanging type of the current level.
  - Example: In the following, for the same level of `enums` or `lists`, if they are marked as "auto-label" (with the help of `el.item(elem-tag: ...)`), then they will share the same maximum label width.

```
#show: el.default-enum-list.with(auto-label-width: auto)
+ List one
#el.auto-label-item(form: it => {
  if it.elem-tag == "auto-label" { auto }
})[
  + Item one
  + Item two
    + #el.item(elem-tag: "auto-label") Sub item one
    + Sub item two
```

```
10    + Item one
11      10. #el.item(elem-tag: "auto-label") Sub item
12  ]
13  + List two
```

#auto-resume-enum(⟨auto-resuming⟩: none)[doc] → content

Resume enum in doc.

- **Note**: The method auto-resume-enum cannot be nested.
- **Note**: In order to make auto-resume-enum work, should set the argument auto-resuming in ***-enum-list** (***-enum**) to auto.

Argument

⟨doc⟩ content

The enum in the doc to process

Argument

⟨auto-resuming⟩: none none | bool | array | function

Resume mode

- none == false: Disable this feature.
- true: All enum numbers within doc will continue from the previous ones (for each level).
- array (level-property): The elements are bool or none. The level-th level will be set to the corresponding value of the array at position level - 1; now true means the enum numbers at the level-th level will continue from the previous ones.
  - **Note**: The enum's level in doc uses the relative or absolute level depending on which ***-enum-list** or ***-enum** is used in the document; but starting from **1**.
  - If the last element of the array is LOOP, the values in the array will be used cyclically;
  - otherwise, the last value of the array will be used for residual levels.
- function (level-property): The return value will be used for each level.
  - The function form: it => bool | none
  - Access the following values available on it:
    - it.level: The level of the item, starting from 1.
    - it.n-last: The index of the last item in the current level.
    - it.elem-tag: The tag of the enum or list.
    - it.e: The construct function (enum or list) of the item.
    - it.hanging-type: The hanging type of the current level.
  - Example: Using it.elem-tag, specify which enum continues to use the previous enum's numbers.

```
1  #show: el.default-enum-list.with(auto-resuming: auto)
2  + List one
3  #el.auto-resume-enum(auto-resuming: it => {
4    if it.elem-tag == "resume" { true } else { false }
5  })[
6    + Item one
```

```
7     + Item two
8       + Sub item one
9       + Sub item two
10    New paragraph
11    + Item one // not resume
12      + #el.item(elem-tag: "resume") Sub item // resume
13  ]
14  + List two // not resume
```

#elabel(⟨name⟩) → content

Creates a labeled reference for enum items.

- **Note**: Use this when regular `label(<...>)` won't work with enum references

Argument

⟨name⟩ — str | label | content

Label identifier

#isolated-resume-enum[doc] → content

Recovery operations within an isolation scope, allows the `enum` in the `doc` to be treated as a new enum with independent numbering, without affecting other enums.

Argument

⟨doc⟩ — content

The `enum` in the `doc` to process

#item(..⟨args⟩)

Configure for the current item

Argument

..⟨args⟩ — arguments

Arguments for the current item.

- Positional arguments.
  - body (`content`): Content for the current label body (`enum` or `list`); defaults to `none` to preserve existing label body
    - **Note**: Both `enum` and `list` support referencing the content of the `body`. However, the `label` needs to be placed after this method, for example:

```
1  #show: el.default-enum-list
2  #show: el.config.ref // Enable to support referencing
3  + #el.item([Enum])<enum:a> item <enum:b>
4  - #el.item([List])<list:a> item
5  Test:
6  @enum:a // Output: Enum
7  @list:a // Output: List
8  @enum:b // Output: 1
```

  - For `enum`, even if the content display of the item label is changed, the current item still has a label number (`enum`'s number) — and the label number will be the same as the previous one (if `item(skipped: true)`) or the same as the current one (if `item(skipped: false)`).
- Named arguments.
  - Support the following arguments:
    - label formatting: any `text` named arguments, label-width, label-format, label-baseline, label-align, label-inset, label-indent
    - body formatting: body-margin, body-format, body-indent
    - horizontal spacing: indent
    - vertical spacing: whole-spacing, item-spacing, tight-mode, tight-item-mode
    - paragraph: hanging-indent, line-indent, first-line-inset, hanging-type
    - reference: ref-numbering, supplement
    - enum's numbering: step, skipped
    - description-list config: description-config
    - identifier: tag, elem-tag
    - others: absolute, for-elem.
  - Details:
  - any `text` named arguments (like: `fill`, `size`, `font`, etc.): Format the current item label.
  - indent (`auto`, `length`, `array`, `function`, `none`): The indentation of the `enum` or `list`.
  - body-indent (`auto`, `length`, `function`, `array`, `none`): The indentation of the body, i.e., the space between the label and the body of each item.
  - label-indent (`auto`, `length`, `array`, `function`, `none`): The indentation for the label (the enum's number or the list's marker).
  - label-inset (`auto`, `length`, `array`, `function`, `none`): Adds an inset to the label. It translates labels horizontally (without affecting the label width and the inset of the first line).
  - item-spacing (`auto`, `relative`, `fraction`, `dictionary`, `array`, `function`, `none`): Spacing between items.
  - whole-spacing (`auto`, `relative`, `fraction`, `dictionary`, `array`, `function`, `none`): The above and below spacing of the current `enum` or `list`. Only used when `item` is in the first item.
  - tight-mode (`auto`, `"always-tight"`, `"never-tight"`, `"compact-tight"`, `dictionary`, `array`, `function`, `none`): If `whole-spacing` are `auto`, then `tight-mode` is used to determine the above and below spacing of `enum` or `list`. Only used when `item` is in the first item.
  - tight-item-mode (`auto`, `"always-tight"`, `"never-tight"`, `dictionary`, `array`, `function`, `none`): If `item-spacing` is `auto`, then `tight-item-mode` is used to determine the spacing between items. Only used when `item` is in the first item.
  - hanging-indent (`auto`, `length`, `array`, `function`, `none`): The hanging indentation for item body.
  - line-indent (`auto`, `length`, `array`, `function`, `none`): The indentation for the first line of a paragraph, excluding the first paragraph (default: `auto`).
  - first-line-inset (`auto`, `length`, `array`, `function`, `none`): Adds an inset to the first line of the item body (default: `0pt` if `auto`). If there is no first line, it is ignored.
  - hanging-type (`"classic"`, `"paragraph"`, `array`, `function`): Hanging type for multiline content; defaults to "classic". Only used when `item` is in the first item.

- label-align (`auto`, `alignment`, `array`, `function`, `none`): The `alignment` that enum numbers and list markers should have. In general, this parameter is used to set **horizontal** alignment for label. If want 2D-alignment, then see `#default-enum-list.label-baseline`.
- label-width (`auto`, `relative`, `"max"`, `dictionary`, `function`, `array`, `none`): Render the width of the label.
- body-format (`dictionary`, `array`, `function`, `none`): Customize item bodies.
- label-format (`dictionary`, `array`, `function`, `none`): Customize item labels.
- label-baseline (`auto`, `relative`, `"center"`, `"top"`, `"bottom"`, `"baseline"`, `"top-item"`, `"horizon-item"`, `"bottom-item"`, `dictionary`, `array`, `function`, `none`): Set the baseline style of the label body.
- body-margin (`auto`, `relative`, `dictionary`, `array`, `function`, `none`): The left and right margin of the current item body. Based on the `itemize` design, `label-indent`, the label's content, `body-indent` are belong to the label's body. For `ltr`, the left margin starts from the left of the label's body; for `rtl`, the right margin starts from the right of the label's body.
- description-config (`dictionary`, `none`): Configure the description list.
- step (`auto`, `int`, `function`, `array`, `none`): Used to set the step size for `enum`'s number.
- ref-numbering (`string`, `function`, `dictionary`, `none`): How to number the `enum`'s reference (default: `none`, determined by `enum.numbering` or `#config.ref.numbering`). Only used when `item` is in the first item of `enum`.
  - **Note**: Only work for the current `enum` (not for the nested `enum`).
  - Example:

```
#show: el.default-enum-list
#set enum(numbering: "1.")
#show: el.config.ref
+ #el.item(ref-numbering: "(1)") item <enum:a>
  + Sub item <enum:b>
Test:
@enum:a // Output: (1)
@enum:b // Output: 1.
```

- supplement (`auto`, `content`, `dictionary`, `function`, `array`, `none`): Used to set supplementary content when referencing labels.
  - **Note**: This only work when referencing the label body.
  - Example:

```
#show: el.default-enum-list
#show: el.config.ref
+ #el.item([Enum], supplement: "Item")<enum:a> item <enum:b>
Test:
@enum:a // Output: Item Enum
@enum:b // Output: 1
```

- skipped (`bool`): Only for `enum`, whether to skip current label numbering; if `true`, the label's number will use the previous one, otherwise, the label's number will remain unchanged; defaults to `false`.

- absolute (`bool`): Whether to use absolute nesting levels.
  - `true`: When using the method **`*-enum-list`**, the meaning of the nesting levels includes `enum` and `list`.
  - `false` (default): In `list`, the meaning of levels only includes `list`; and in `enum`, the meaning of levels only includes `enum`.
- for-elem (`bool`): Whether the properies set by `item` is applied for current `enum` or `list` (not only for the current item); defaults to `false`. Only used when `item` is in the first item.
  - Example: In the following, for `List A`, all the labels of are colored by `red`, but for `List B`, only the first item is colored by `red`:

```
#show: el.default-enum-list
// List A
+ #el.item(for-elem: true, fill: red) List A item one
+ List A item two
Paragraph:
// List B
+ #el.item(fill: red) List A item one
+ List A item one
```

- tag (`any`): The identifier of the item.
- elem-tag (`any`): The identifier of the lists (`list` or `enum`).

`#resume(..⟨body⟩)` → **`content`**

Continue using the enum numbers from the previous ones (for the same level).

- **Note**: In order to make this method work, set the argument `auto-resuming` in the method **`*-enum-list`** (**`*-enum`**) to `auto`.

Argument

`..⟨body⟩` — `content`

The `enum` in the body to resume

`#resume-label(⟨key⟩)` → **`content`**

Label current `enum` for reference by `key`

Argument

`⟨key⟩` — `label` | `str` | `content`

Unique identifier for the `enum`

`#resume-list(⟨key⟩, ..⟨body⟩)` → **`content`**

Resumes an enum labelled with `key` (by using the method `resume-label`)

- **Note**: In order to make this method work, set the argument `auto-resuming` in the method **`*-enum-list`** (**`*-enum`**) to `auto`.

Argument

`⟨key⟩` — `label` | `str` | `content`

Key of the `enum` to resume

Argument

`..(body)` content

The `enum` in the body to resume

#LOOP

Mark the last element in the array for cyclic usage. I.e., for `array` parameters, if the last element of the array is `LOOP`, then the values in the array will be used cyclically.

#itemize-prevent-label

Prevent recursion label

Add this label to `enum` or `list` to prevent `itemize` to process.

- Usage:

```
#enum(
  enum.item([#lorem(2)]),
  // other items
)#el.itemize-prevent-label
```

#new-enum-list-flag

Display an enum or list as a new enum or list. Usage:

```
+ item 1
+ item 2
#el.new-enum-list-flag
// The following content will be treated as a new enum or list
+ item 3
+ item 4
```


#config.auto-detect-tight #config.ref
#config.checklist #config.ref-resume

#config.auto-detect-tight((enable): true)[doc] → content

Configure auto-detect-tight settings for a document (in **module** `config`).

If `enable` is `true` and the current `whole-spacing` is `auto`, enable a compact mode between the list and the paragraph below it.

- **Note**: Related to the parameter `tight-mode` in `*-enum-list` (`*-enum`, `*-list`).
- Usage:

```
#show: el.config.auto-detect-tight
```

Argument

`(doc)` content

The document to apply.

Argument

`(enable): true` `bool`

Enable or disable a compact mode between the list and the paragraph below it, controlled by `tight-mode` settings (in ***-enum-list**, ***-enum**, ***-list**).

`#config.checklist(..(args))[doc]` → `content`

Configure checklist settings for a document (in **module `config`**).

Argument

`(doc)` `content`

The document to apply the checklist settings to.

Argument

`..(args)` `argument`

Contain the following arguments:

- baseline (`auto`, `"center"`, `"top"`, `"bottom"`, `"baseline"`, `array`): The baseline alignment for checklist labels (same as `label-baseline` with `"center"`, `"top"`, `"bottom"`, `"baseline"`). Default is `auto`, controlled by `label-baseline` of the method ***-enum-list** (***-enum**, ***-list**).
- fill (`auto`, `color`, `none`, `array`): The fill color for checklist items. If set to `auto`, it uses the current label's text style. Default is `auto`.
- radius (`auto`, `length`, `dictionary`, `array`): The border radius for checklist items (default: `.1em`). See also `block.radius`.
- solid (`none`, `color`, `array`): The solid border style for checklist items (default: `none`).
- extras (`bool`, `array`): Whether to enable extra features (default: `false`). If `true`, then use the following additional commands:

```
">": "➡",
"<": "🗓",
"?": "❓",
"!": "❗",
"*": "⭐",
"\"": "“",
"l": "📍",
"b": "🚀",
"i": "i",
"S": "💰",
"I": "💡",
"p": "👍",
"c": "👎",
"f": "🔥",
"k": "🔑",
"w": "⏳",
"u": "🔼",
"d": "🔽",
```

- enable-character (`bool`, `array`): Whether to enable character-based labels (default: `true`). When set to `true`, if the character in `[...]` is not among `x`, , `-`, `/` or the extras characters (if `extras` is `true`), the character in `[...]` will be displayed.

- enable-format (`bool`, `array`): Whether to enable formatting for the item body, with the formatting determined by `format-map` (default: `false`). The default formatting is for the character `"-"`, with the formatting function:

```
1  it => strike(text(fill: rgb("#888888"), it))
```

- symbol-map (`dictionary`, `function`, `array`): A map of symbols for checklist items (default: `(:)`).
  - `dictionary`: Can replace built-in commands or add new ones.
    - The key-value pair is: `("some-character": content)`.
  - `function`: The function form is: `it => dictionary` (the `dictionary` is the above form), where `it` provides three properties: `fill`, `radius`, `solid`.
- format-map (`dictionary`, `array`): A map of formats for checklist items (default: `(:)`). Formatting for item body.
  - The key-value pair is: `("some-character": func)`.
  - `func` takes the form `it => content`. It applies the body of the current `some-character` item to this method.

`#config.ref((full): auto, (numbering): auto, (supplement): auto, (no-label-warning): false)[doc]`
→ `content`

Configure enum reference settings for a document (in **module** `config`).

Argument

`(doc)` `content`

The document to apply the reference settings to.

Argument

`(full): auto` `auto` | `bool` | `str`

Determine to display the full enum numbers.

- `auto`: Use the `enum.full` setting.
- `true` Display the full number (including parent levels).
- `false` Display only the current item's number.
- `"ref"` Display the reference number items in a relative manner (i.e., If the current item and the reference item have the same parent level (in the same parent enum), the same parent level is not displayed.)

Argument

`(numbering): auto` `auto` | `str` | `function`

Numbering pattern.

- `auto`: Use the `enum.numbering`.
- `string`, `function`: Customize the style of the referenced item number. See also `enum.numbering`.

Argument

`(supplement): auto` `auto` | `content` | `dictionary` | `function` | `array`

Supplemental content for the reference.

- `auto`: No supplementary content will be added.
- `content`: Use `content` as supplementary content. The effect is that when referencing `enum` or `list` labels, then content is added before the label.
- `dictionary`: The keys are: `prefix`, `suffix`, with values of `content`. The effect is that when referencing enum (list) labels, `prefix` content is added before the label, and `suffix` content is added after the label.
- `array` (level-property): The elements are `auto`, `content`, `dictionary` or `array`. The item at `level`-th level will be indented by the corresponding value of the array at position `level - 1`.
  - If the last element of the array is `LOOP`, the values in the array will be used cyclically;
  - otherwise, the last value of the array will be used for residual levels.
  - The elements in the array can also be an array, where the element at position `n - 1` applies to the item of index `n`. **Note**: Not support for `list`.
- `function` (level-property): Customize the supplementary content.
  - The function form: `it => content`.
  - Access the following values available on `it`:
    - `it.body`: The referenced `enum` labels.
    - `it.level` (Only for `enum`): The level of the item, starting from 1.
    - `it.n` (Only for `enum`): The index of the item, starting from 1.
    - `it.n-last` (Only for `enum`): The index of the last item in the current level.
    - `it.tag` (since ver0.3.0) (Only for `enum`): The tag of the item.
    - `it.elem-tag` (since ver0.3.0) (Only for `enum`): The tag of the `enum` or `list`.
    - `it.target` (since ver0.3.0): The target of the referenced label.
    - `it.number` (since ver0.3.0) (Only for `enum`): The number of the referenced item.
    - `it.numbers` (since ver0.3.0): The full numbers of the referenced item (including the parent `enum`'s numbers). For `list`, this is `()`.

Argument

`⟨no-label-warning⟩: false` `bool`

Whether to disable the warning when the reference label is not found (default: `false`); note that this applies to all references in the document, so we recommend *not* setting it to `true`.

`#config.ref-resume[doc]` → `content`

Configure resume reference settings for a document (in **module `config`**).

- If you use the following in your document:

```
1 #show: el.config.ref-resume
```

then you can use `@some-label` instead of `resume-list(<some-label>)`.

Argument

`⟨doc⟩` `content`

The document to apply the resume reference settings to.

```
#adv.reset-resume
```

`#adv.reset-resume`

Resets all resume counter (in **module** `adv`)

If these records are no longer needed in the document, you can call the `adv.reset-resume()` method to clear this information. One common use case is:

```
// New enum-before
#el.adv.reset-resume()
// New enum-here:
#el.adv.reset-resume()
// New enum-after
```

- **Note**: Improper use of `adv.reset-resume()` may break the `resuming enum` functionality (**not** recommended to use).

`#version`

`itemize` version (in **module** `adv`)